Epidemiología: Nivel Avanzado

Instituto Nacional de Epidemiología Dr. Juan H. Jara

¡Les damos la bienvenida al curso!

Es un gusto recibirles en esta nueva edición del curso, diseñado para profundizar en el análisis de datos provenientes de distintos diseños de estudio mediante el uso de modelos estadísticos.

A lo largo del curso, abordaremos herramientas analíticas avanzadas que nos permitirán modelar adecuadamente la relación entre exposiciones y desenlaces en estudios transversales, de casos y controles, de cohortes y experimentales. Nos enfocaremos en la aplicación práctica de estos modelos, en la interpretación de resultados y en los supuestos que los sustentan, promoviendo una mirada crítica sobre su uso en la investigación epidemiológica.

Esperamos que este espacio sea una experiencia formativa enriquecedora y un nuevo impulso en su desarrollo como investigadoras e investigadores en salud.

El equipo docente

Christian Ballejo

Andrea Silva

Tamara Ricardo

Ramiro Dana Smith

Julia Maxwell

Declaración de uso de IA generativa

Se utilizaron herramientas de inteligencia artificial generativa (ChatGPT, versión GPT-5.3) para la revisión gramatical y ortográfica del manuscrito. El contenido final fue evaluado, validado y editado manualmente por los/as autores/as, quienes asumen la responsabilidad por su integridad.

Parte I

Unidad 1

1 Introducción a R



Figura 1.1: Artwork por @allison_horst

⚠ Estos materiales, que forman parte del **Módulo 1 del Curso de Epidemiología Avanzada**, no pretenden cubrir exhaustivamente todos los temas relevantes de un curso de lenguaje R. Su objetivo es introducir únicamente aquellos aspectos más importantes y necesarios para su aplicación en los módulos siguientes.

1.1. ¿Qué es R?

El sitio oficial **r-project.org** define a R como *"un entorno de software libre para gráficos y computación estadística. Se compila y se ejecuta en una amplia variedad de plataformas UNIX, Windows y MacOS."*

Profundizando en su descripción, podemos decir que R es un *lenguaje de programación interpretado, orientado a objetos, multiplataforma y de código abierto (open source) aplicado al manejo y análisis de datos estadísticos.*

A continuación, detallamos cada una de sus características:

R es un lenguaje de programación estadístico

Si bien posee un entorno y se puede utilizar como calculadora avanzada o para simulaciones, R es fundamentalmente un lenguaje de programación. Presenta una estructura y reglas de sintaxis propias, así como gran variedad de funciones desarrolladas con fines estadísticos.

R es un lenguaje orientado a objetos

R implementa conceptos de la programación orientada a objetos, lo cual le permite ofrecer simpleza y flexibilidad en el manejo de datos. En R, todo con lo que trabajamos —variables, funciones, datos, resultados— son objetos que pueden ser modificados o combinados con otros objetos.

R es un lenguaje interpretado

R no requiere compilación previa. Los scripts se ejecutan directamente mediante el intérprete del lenguaje, que devuelve resultados de forma inmediata.

R es un lenguaje multiplataforma

R puede instalarse y utilizarse en sistemas operativos Linux, Windows y MacOS. En todos ellos funciona de la misma manera, lo que garantiza que los scripts pueden correr en cualquier plataforma sin necesidad de modificaciones.

R es software libre y de código abierto

R se distribuye gratuitamente bajo licencia GNU - GPL (*General Public License*), lo que otorga a los usuarios la libertad de usar, estudiar, compartir y modificar el software. Esto ha favorecido el crecimiento de una comunidad global activa y colaborativa.

1.2. Breve historia del lenguaje

R tiene su origen en el lenguaje **S**, desarrollado en los años 70 en los laboratorios Bell de AT&T (actualmente Lucent Technologies). Posteriormente, S dio lugar a una versión comercial llamada **S-Plus**, distribuida por Insightful Corporation.

En 1995, los profesores de estadística **Ross Ihaka** y **Robert Gentleman**, de la Universidad de Auckland (Nueva Zelanda) iniciaron el **"Proyecto R"**, con la intención de desarrollar un programa estadístico inspirado en el lenguaje S pero de dominio público.

Aunque R es considerado un dialecto de S, existen diferencias importantes en el diseño de ambos lenguajes.

El software está desarrollado principalmente en lenguaje C++, con algunas rutinas en Fortran. El nombre "R" hace referencia a las iniciales de sus creadores: Ross y Robert. Actualmente, R es mantenido por un grupo internacional de desarrolladores voluntarios conocido como el **Core Development Team**.

1.3. Scripts de R

Un script es un archivo de texto plano que contiene una secuencia de instrucciones para ser ejecutadas por el intérprete de R.

El término *script* puede traducirse como guión, archivo de órdenes, archivo de procesamiento por lotes o archivo de sintaxis.

Puede crearse con cualquier editor de texto o con entornos especializados, y permite ser leído, modificado, guardado y ejecutado de forma completa o línea por línea.

Una de sus principales ventajas es su **reutilización**: los scripts pueden adaptarse fácilmente a distintos análisis o contextos, lo que facilita la replicabilidad del trabajo.

1.4. Características generales del lenguaje

R posee una **sintaxis textual precisa**. Como en otros lenguajes de programación, la escritura debe ser exacta: distingue entre mayúsculas y minúsculas (*case sensitive*), y cada línea escrita en la consola comienza con el símbolo > (*prompt*[1]).

[1] Símbolo que aparece en la pantalla de la computadora indicando que el sistema está esperando información del usuario o que el sistema está listo para recibir instrucciones del usuario.

1.5. Documentación del código

La documentación es una tarea fundamental en cualquier lenguaje de programación, ya que nos permite entender el propósito del script, facilita su mantenimiento y posibilita su reutilización tanto por quien lo creó como por otras personas.

En R, la documentación de los scripts se realiza a través de **comentarios**, indicados con el símbolo #. Todo lo que sigue a ese símbolo es ignorado por el intérprete cuando se ejecute el código:

```
# esto es una línea de comentario y el intérprete no la ejecuta
```

Así que a la hora de documentar es preferible abusar de estos comentarios que no utilizarlos.

1.6. Funciones

Las órdenes elementales en R se denominan comandos o **funciones**. Algunas se conocen como "integradas" ya que están incluidas en el núcleo del lenguaje (R `base`) y otras provienen de **paquetes** adicionales.

La mayoría de las ofrecidas en los paquetes o librerías están elaboradas con R, dado que todos los usuarios podemos crear nuevas funciones con el mismo lenguaje.

Toda función tiene un nombre y puede recibir argumentos (obligatorios u opcionales), que se colocan entre **paréntesis y separados por comas**. Incluso algunas funciones que no tienen asociado argumentos necesitan, en su sintaxis, a los paréntesis `()`.

Siempre una función devuelve un resultado, un valor o realiza una acción:

```
nombre_de_la_función(arg_1, arg_2, arg_n)
```

Como el interprete de R no permite errores en la sintaxis de las expresiones, debemos atender a los siguientes puntos a la hora de escribirlas:

La sintaxis habitual de una función es la siguiente:

```
funcion(arg1, arg2, arg3, ...)
```

Los títulos de los argumentos pueden escribirse y mediante un igual agregar el valor correspondiente:

```
funcion(arg1 = 32, arg2 = 5, arg3 = 65, ...)
```

Se puede omitir el título del argumento y escribir directamente el valor, pero en este caso, hay que respetar el orden definido por la función:

```
funcion(32, 5, 65, ...)
```

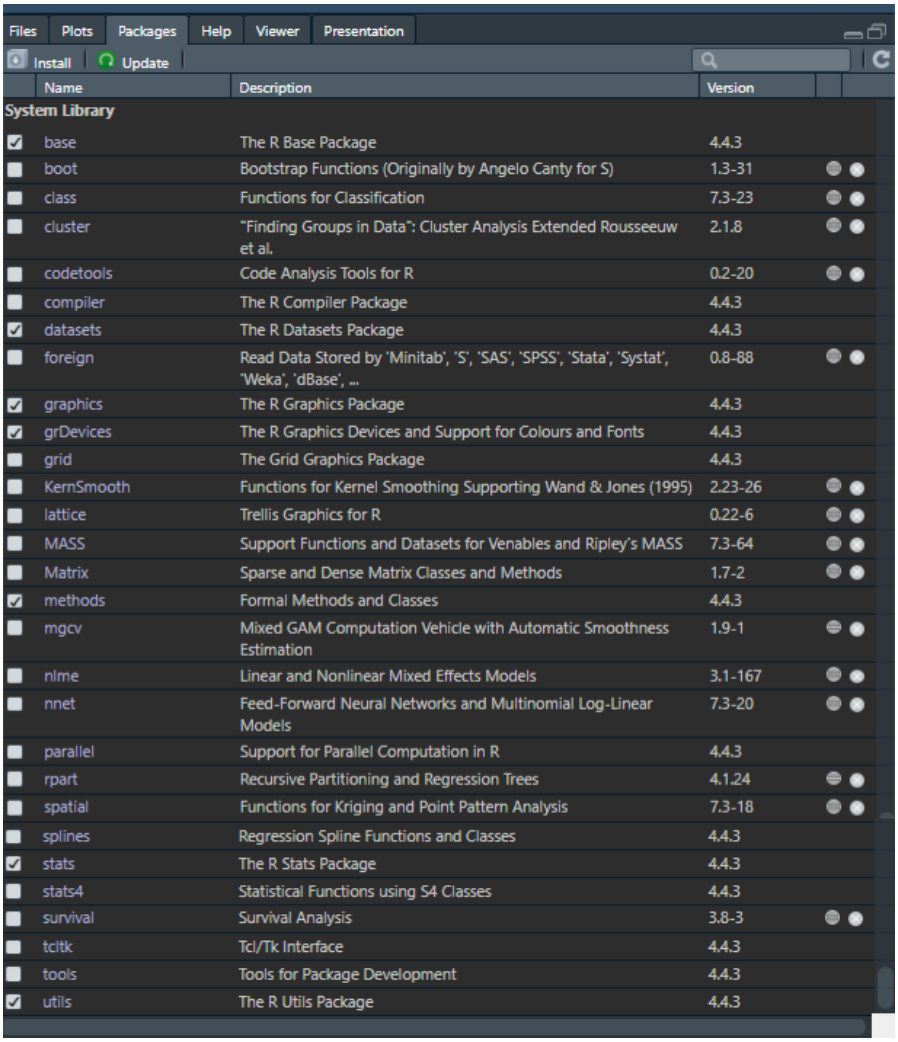
Los valores numéricos, lógicos, especiales y objetos van escritos en forma directa y cuando escribimos caracteres (texto) van necesariamente encerrados entre comillas:

```
funcion(arg1 = 3, arg2 = NA, arg3 = TRUE, arg4 = "less", arg5 = x, ...)
```

1.7. Paquetes o librerías

Los paquetes, también conocidos como *librerías*, son conjuntos de funciones, datos y documentación organizados organizadas en torno a una temática específica, que permiten ampliar las capacidades del sistema base de R.

Al instalar R, se incorpora un núcleo básico que queda activo automáticamente en cada sesión (`base`, `datasets`, `graphics`, `grDevices`, `methods`, `stats` y `utils`). Junto a este núcleo, se incluye un conjunto de paquetes recomendados que forman parte de la distribución oficial y pueden activarse manualmente cuando se los necesita. No obstante, la verdadera potencia de R reside en la posibilidad de incorporar nuevos paquetes desarrollados por la comunidad, lo que permite extender continuamente sus funcionalidades.



En la actualidad, existen más de 17.000 paquetes disponibles para una amplia variedad de aplicaciones, número que crece mes a mes gracias a la naturaleza *open source* del lenguaje. Cualquier persona puede desarrollar y compartir sus propios paquetes, aunque no todos llegan a publicarse en el repositorio oficial CRAN (Comprehensive R Archive Network), que actúa como principal fuente de distribución.

Los paquetes pueden descargarse directamente desde CRAN o instalarse a partir de archivos comprimidos locales (como `.zip` o `.tar.gz`). Una vez instalados, se los puede activar en cualquier análisis.

1.7.1. Dependencias

En muchos casos, al utilizar un paquete, este necesita de otros para funcionar correctamente. Esta relación se conoce como **dependencia**, y es una característica central en el ecosistema de R.

La mayoría de las funciones incluidas en los paquetes están desarrolladas en el propio lenguaje R y, durante su construcción, es habitual que recurran a funciones ya existentes en otros paquetes. Por ejemplo, una función nueva puede hacer uso de una función auxiliar que no pertenece al sistema base, sino a otro paquete externo.

Cuando intentamos ejecutar una función que depende de otras no disponibles en nuestra instalación, R no podrá encontrar esas funciones y nos devolverá un mensaje de error indicando que no reconoce el nombre solicitado. Este tipo de errores suele alertar sobre funciones "desconocidas" o "no encontradas", lo que indica que falta instalar o activar alguno de los paquetes requeridos.

Para evitar estos inconvenientes, R intenta resolver automáticamente las dependencias cuando instalamos un paquete desde el repositorio oficial CRAN. Si el paquete necesita otros para funcionar, el sistema detecta esta relación y se encarga de instalar también aquellos paquetes auxiliares. De todos modos, si alguno no se instala correctamente o no se activa en la sesión, será necesario hacerlo de forma manual.

1.8. Errores y advertencias

El lenguaje R es muy preciso en su sintaxis. Cometer errores al escribir funciones u objetos es común durante el aprendizaje, y el intérprete de R devuelve mensajes de error cuando detecta algo incorrecto.

Uno de los aspectos clave a tener en cuenta es que R **distingue entre mayúsculas y minúsculas** (*case sensitive*), por lo que no es lo mismo escribir `a` que `A`.

Existen tres grupos de mensajes de error:

- Errores de sintaxis
- Error de objeto no encontrado
- Otros errores

1.8.1. Errores de sintaxis

Ocurre cuando R no puede interpretar una línea de código porque algo está mal escrito. Los errores de sintaxis más frecuentes incluyen:

- Paréntesis, comas, comillas, corchetes o llaves mal colocados o ausentes.
- Argumentos mal separados o incorrectamente definidos.

Por ejemplo la función `rep()` repite valores una cantidad de veces. Tiene dos argumentos, `x` donde se coloca el valor a repetir y `times` donde se define la cantidad de veces:

```
rep(x = 3, times = 4) # repetimos 4 veces 3 con rep()
```

```
[1] 3 3 3 3
```

Si nos olvidamos de cerrar el paréntesis:

```
rep(x = 3, times = 4
```

```
Error in parse(text = input): <text>:2:0: unexpected end of input
1: rep(x = 3, times = 4
   ^
```

Si omitimos la coma entre argumentos:

```
rep(x = 3 times = 4)
```

```
Error in parse(text = input): <text>:1:11: unexpected symbol
1: rep(x = 3 times
              ^
```

Si usamos un nombre de argumento no válido:

```
rep(y = 3, times = 4)
```

```
Error in `rep()`:
! attempt to replicate an object of type 'symbol'
```

Si escribimos mal el nombre de la función:

```
rop(x = 3, times = 4)
```

```
Error in `rop()`:
! could not find function "rop"
```

Este último error se asemeja a un error de objeto no encontrado, aunque tiene origen en un problema de sintaxis.

Los mensajes de error en general y sobre todo al principio pueden parecer extraños y difíciles de entender, pero con un poco de práctica podemos inferir donde está el problema.

1.8.2. Error de objeto no encontrado

Este tipo de error se produce cuando R no reconoce un objeto utilizado en el código. Las causas más frecuentes incluyen:

- El nombre del objeto está mal escrito (por ejemplo, errores de sintaxis o uso incorrecto de mayúsculas/minúsculas).
- El objeto pertenece a un paquete o archivo que no fue cargado.
- Faltan comillas al declarar caracteres.
- Otras causas similares.

Volvamos al ejemplo anterior, ahora repitiendo un valor tipo `character`:

```
rep(x = "A", times = 4) # repetimos 4 veces "A" con rep()
```

```
[1] "A" "A" "A" "A"
```

Si olvidamos las comillas:

```
rep(x = A, times = 4) # repetimos 4 veces A con rep()
```

```
Error:
! object 'A' not found
```

1.8.3. Otros errores

Son aquellos que se generan por causas distintas a errores de sintaxis o de objeto no encontrado. Por ejemplo:

```
"x" + 10
```

```
Error in `"x" + 10`:
! non-numeric argument to binary operator
```

El código anterior genera un mensaje de error porque intenta sumar objetos de tipos incompatibles entre sí.

Veamos otro ejemplo:

```r
t.test(1)
```

```
Error in `t.test.default()`:
! not enough 'x' observations
```

En este caso, el error se debe a que no hay suficientes observaciones para realizar una prueba t de Student.

Otro caso frecuente ocurre cuando se intenta acceder a una posición inexistente en un vector:

```r
x <- c(1, 2, 3)
x[[5]]
```

```
Error in `x[[5]]`:
! subscript out of bounds
```

Este mensaje de error indica que el subíndice está fuera de los límites del objeto (`subscript out of bounds`).

1.8.4. Advertencias

Las advertencia no son tan serias como un error, o al menos no lo parece, ya que permiten el código se ejecute igual. Pero puede ocurrir que ignorar una advertencia llegue a ser algo muy serio, si esto implica que la salida de la función es equivocada.

Ignorar advertencias puede llevar a conclusiones erróneas, por lo que es recomendable prestarles atención y entender su causa.

Por ejemplo:

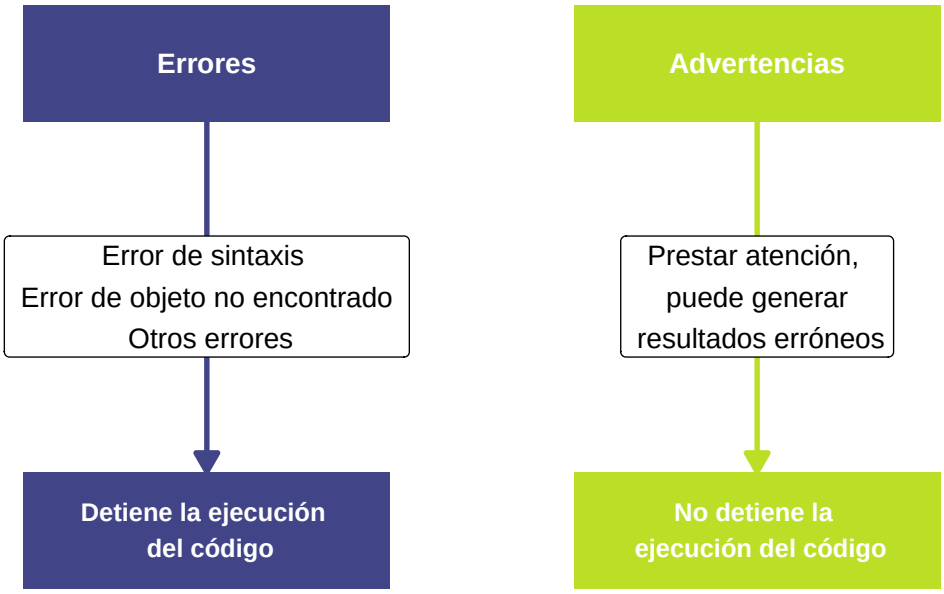
```r
log(-1)
```

```
Warning in log(-1): NaNs produced
```

```
[1] NaN
```

la función genera una advertencia porque el logaritmo de un número negativo no está definido en los reales, y R devuelve un valor `NaN` (*Not a Number*).

Resumiendo...



1.9. Creación de objetos

Todas las declaraciones donde se crean objetos usan el símbolo de asignación <−:

```
nombre_objeto <- valor
```

Veámoslo en un ejemplo:

```
a <- 1
```

En este caso asignamos el valor 1 al objeto a. El objeto a es un vector de una posición (un solo valor). Si llamamos al objeto, el intérprete devuelve el valor asignado previamente:

```
a
```

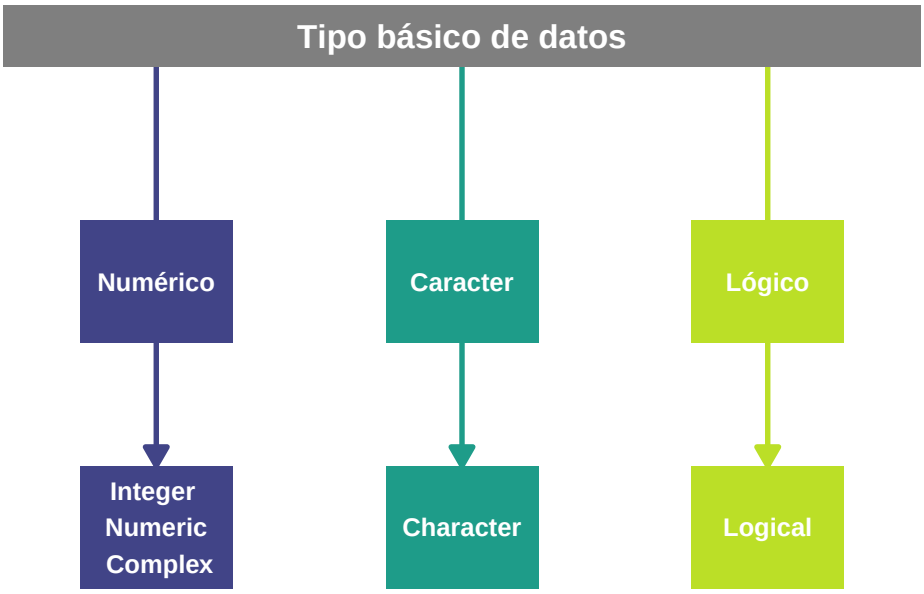
```
[1] 1
```

Observemos que, además de devolvernos el valor, aparece delante un número entre corchetes [1]. Este número es la ubicación o **índice** del comienzo del objeto. En este caso, como el vector tiene una sola posición, indica que el primer valor mostrado empieza en la posición 1.

1.10. Estructuras de datos

Los objetos contenedores de datos más simples pertenecen a cinco clases atómicas, que son:

- `integer` (números enteros)
- `numeric` (números reales)
- `complex` (números complejos)
- `character` (cadenas de caracteres)
- `logical` (valores lógicos: `TRUE` o `FALSE`)



Sin embargo, cada una de estas clases de datos no se encuentran de manera aislada, sino encapsuladas dentro de la clase de objeto más básica de R, a la que se denomina **vector**.

En RStudio, el sistema de colores ayuda a distinguir los tipos de valores:

```
# Número
1111
```

```
[1] 1111
```

```r
# Caracter
"esto es una cadena de texto"
```

```
[1] "esto es una cadena de texto"
```

```r
# Lógico
TRUE
```

```
[1] TRUE
```

```r
# Objeto
a <- 1
```

1.10.1. Vectores

Un vector es un conjunto de valores (números o símbolos), del mismo tipo ordenados de la forma (elemento 1, elemento 2, … , elemento n), donde n es la longitud o tamaño del vector.

Los atributos principales son:

- **Tipo**: puede ser `integer`, `numeric`, `character`, `complex` o `logical`.
- **Longitud**: cantidad de elementos que contiene el objeto,

El vector más simple contiene un solo dato, por ejemplo un vector de longitud 1 y tipo `numeric`:

```r
vec1 <- 1
vec1
```

```
[1] 1
```

Otro vector más grande por ejemplo podría ser `(1, 5, 2)`. En este caso también es del tipo `numeric` pero tiene una longitud de 3 elementos:

```r
vec2 <- c(1, 5, 2)
vec2
```

```
[1] 1 5 2
```

Para concatenar elementos usamos la función `c()`, dentro de la cual van los valores separados por comas. El orden de los elementos importa, en nuestro ejemplo la primera posición la ocupa el 1, la segunda el 5 y la tercera el 2. Si tuvieramos otro vector `(5, 1, 2)`, no sería lo mismo porque los valores están ordenados de forma diferente.

Para conocer la longitud del vector usamos:

```
length(vec2)
```

```
[1] 3
```

Nos informa que `vec2` tiene 3 elementos.

Para conocer el tipo de dato ejecutamos:

```
class(vec2)
```

```
[1] "numeric"
```

Podemos ver que los datos almacenados en este segundo ejemplo cumplen con la definición en lo que respecta al tipo de dato, ya que cada elemento es del mismo tipo (`numeric`).

Veamos un ejemplo de asignación de otro tipo de dato atómico, como es el `character`:

```
vec3 <- "Hola"
vec3
```

```
[1] "Hola"
```

Siempre que escribamos contenido de tipo `character` debemos hacerlo entre comillas. En este caso generamos el vector `vec3` con el contenido `"Hola"`, que, a pesar de ser una palabra compuesta de varios caracteres, dentro del vector `vec3` esta ocupa una sola posición:

```
length(vec3)
```

```
[1] 1
```

Respecto al tipo de dato si usamos la función `class()` tendremos:

```r
class(vec3)
```

```
[1] "character"
```

1.10.2. Factores

Un factor es un objeto especialmente diseñado para contener datos categóricos y se asocia particularmente con las *variables cualitativas*.

En su estructura interna está compuesto por dos vectores:

- Un vector de índices enteros.
- Un vector de categorías (niveles) de tipo `character`, a los que hace referencia el primer vector.

Existen de dos tipos: factores nominales y factores ordinales. En el caso del segundo se establece un orden en los niveles.

Normalmente, obtenemos un tipo `factor` de convertir un vector u otro tipo de objeto con caracteres, pero para mostrar un ejemplo lo realizamos con la función `factor()`:

```r
factor1 <- factor(
  x = c("a", "b", "a", "c", "b", "a"),
  levels = c("a", "b", "c")
)
factor1
```

```
[1] a b a c b a
Levels: a b c
```

Creamos el objeto `factor1` con 6 elementos caracteres y tres niveles sin orden.

Además de la practicidad de trabajar con factores, muchas funciones de R requieren que las variables categóricas estén en formato factor para funcionar correctamente.

1.10.3. *Dataframe*

Un *dataframe* es un objeto diseñado para contener conjuntos de datos y representa una estructura bidimensional similar a una tabla, con filas y columnas. Cada columna puede almacenar elementos de distintos tipos (por ejemplo, numéricos, de texto o lógicos), siempre que todos tengan la misma longitud.

Las columnas suelen tener nombres únicos, lo que permite referenciarlas fácilmente como si fueran variables individuales dentro del conjunto de datos.

Este es el tipo de objeto que se utiliza habitualmente para almacenar información importada desde archivos externos (por ejemplo, archivos de texto separados por comas o planillas de Excel), y con el que más frecuentemente trabajamos en los análisis.

Desde el punto de vista estructural, un *dataframe* está compuesto por una serie de vectores de igual longitud dispuestos verticalmente, uno al lado del otro, conformando las columnas de la tabla.

Veamos un ejemplo:

```r
# Historia clínica
HC <- c("F324", "G21", "G34", "F231")

# Edad
edad <- c(34, 32, 34, 54)

# Sexo
sexo <- c("M", "V", "V", "M")

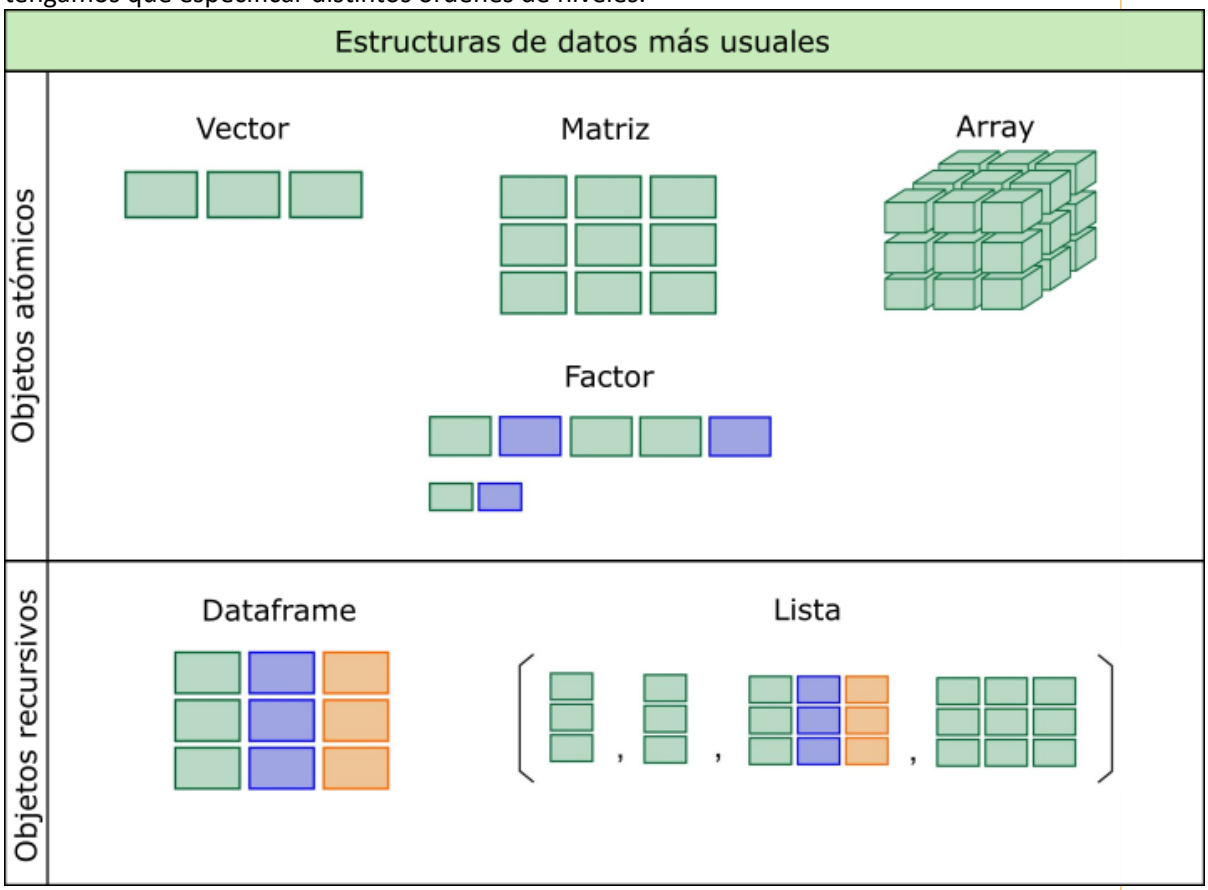
# dataframe
df1 <- data.frame(HC, edad, sexo)

df1
```

```
    HC edad sexo
1 F324   34    M
2  G21   32    V
3  G34   34    V
4 F231   54    M
```

Creamos tres vectores con datos de supuestos individuos, su historia clinica, la edad y el sexo. Luego mediante la función `data.frame()` “unimos” esos vectores en forma vertical para formar un *dataframe* de 3 variables y 4 observaciones.

Existen otras estructuras de datos que aparecen en la siguiente figura. Las más habituales en nuestro trabajo son los vectores y los *dataframes*. Los factores serán necesarios cuando tengamos que especificar distintos órdenes de niveles.



1.11. Operadores

Además de funciones, el lenguaje R cuenta con operadores de uso relativamente intuitivo, que permiten realizar operaciones de diferentes tipos con los objetos que contienen datos.

1.11.1. Operadores aritméticos

Operador	Descripción
+	Suma
-	Resta

Operador	Descripción
*	Multiplicación
/	División
^	Potencia
%%	Módulo
%/%	División de enteros

Los operadores aritméticos se utilizan como si el lenguaje fuese una calculadora:

```
# Suma
2 + 5
```

```
[1] 7
```

```
# Resta
3 - 2
```

```
[1] 1
```

```
# Multiplicación
9 * 3
```

```
[1] 27
```

```
# División
10 / 2
```

```
[1] 5
```

```
# Potencia
5^2
```

```
[1] 25
```

También se pueden hacer operaciones con los objetos que almacenan valores numéricos:

```r
a <- 3

b <- 6

(a + b) * b
```

```
[1] 54
```

Y funciona con objetos con más de un elemento, aplicando aritmética vectorial, donde las operaciones se realizan elemento a elemento:

```r
a <- c(1, 2, 3)

a * 3
```

```
[1] 3 6 9
```

O bien, con operaciones entre los objetos, donde se realiza entre los elementos de la misma posición:

```r
a <- c(1, 2, 3)

a * a
```

```
[1] 1 4 9
```

1.11.2. Operadores relacionales

Operador	Descripción
<	Menor que
>	Mayor que
<=	Menor o igual que
>=	Mayor o igual que
==	Igual que
!=	No igual que

Habitualmente estos operadores se utilizan asiduamente en expresiones para indicar relaciones entre valores.

Podemos ver su funcionamiento en el ejemplo siguiente:

```r
a <- c(3, 8, 2)

a == c(3, 4, 5)
```

```
[1]  TRUE FALSE FALSE
```

El lenguaje evalúa las comparaciones que hace el operador relacional igual (en este caso) y en aquellos valores que coinciden devuelve `TRUE` y en los que no hay coincidencia devuelve `FALSE`.

Lo mismo sucede con los otros operadores relacionales:

```r
a <- c(4, 8, 10)

a > 8
```

```
[1] FALSE FALSE  TRUE
```

```r
a < 8
```

```
[1]  TRUE FALSE FALSE
```

```r
a != 8
```

```
[1]  TRUE FALSE  TRUE
```

1.11.3. Operadores lógicos

Operador	Descripción
!	NOT
&	AND booleano
&&	AND booleano para vectores de longitud 1
|	OR booleano
||	OR booleano para vectores de longitud 1

Cuando queremos conectar algunas de las expresiones relacionales hacemos uso de estos operadores lógicos típicos (AND, OR, NOT).

Para ejemplificar podemos hacer:

```r
a <- c(1:8)

a
```

```
[1] 1 2 3 4 5 6 7 8
```

```r
(a > 3) & (a < 7)
```

```
[1] FALSE FALSE FALSE  TRUE  TRUE  TRUE FALSE FALSE
```

En el caso anterior usamos el operador & como conector AND de dos expresiones relacionales donde el lenguaje devuelve `TRUE` en el rango mayor a 3 y menor a 7 (valores 4,5 y 6).

1.12. Valores especiales

Existen algunos valores especiales para datos con expresiones reservadas en R, entre ellos encontramos los valores `NA`, `NaN`, `Inf` y `NULL`.

Operador	Significado	Descripción
NA	Not available	Es la forma de expresar a los valores perdidos o faltantes (missing values)
NaN	Not a number	Utilizado para resultados de operaciones que devuelven error numérico
Inf	Infinity	Valor infinito (positivo)
-Inf	Infinity	Valor infinito (negativo)
NULL	Null	Valor nulo

El más relevante de estos valores especiales es el `NA` que sirve para indicar que no hay valor en esa posición o elemento de un objeto.

1.13. Secuencias regulares

Además de concatenar elementos con la función `c()`, existen tres formas comunes de generar secuencias regulares.

La primera es mediante un **operador secuencial** (`:`), que genera una secuencia de enteros entre dos valores, ya sea en forma ascendente o descendente:

```r
1:10
```

```
 [1]  1  2  3  4  5  6  7  8  9 10
```

```r
10:1
```

```
 [1] 10  9  8  7  6  5  4  3  2  1
```

La segunda forma es mediante la función `seq()`, cuyos argumentos principales son `from`, `to` y `by`. Esta función permite mayor flexibilidad:

```r
seq(from = 1, to = 20, by = 2)
```

```
 [1]  1  3  5  7  9 11 13 15 17 19
```

El ejemplo anterior genera una secuencia de números que comienza en 1 y llega hasta 20, avanzando de dos en dos.

Algunos otros ejemplos de la misma función:

```r
seq(from = 0.1, to = 0.9, by = 0.1)
```

```
[1] 0.1 0.2 0.3 0.4 0.5 0.6 0.7 0.8 0.9
```

```r
seq(from = -5, to = 5, by = 1)
```

```
 [1] -5 -4 -3 -2 -1  0  1  2  3  4  5
```

```r
seq(from = 300, to = 0, by = -50)
```

```
[1] 300 250 200 150 100  50   0
```

La tercera opción es la función `rep()`, que permite duplicar valores. Su forma más básica es `rep(x, times = n)`, donde se repite el valor `x` la cantidad de veces indicada por `n`.

Algunos ejemplos:

```r
rep(x = 2, times = 5)
```

```
[1] 2 2 2 2 2
```

```r
# combinada con el operador :
rep(1:4, 5)
```

```
 [1] 1 2 3 4 1 2 3 4 1 2 3 4 1 2 3 4 1 2 3 4
```

```r
# combinada con la función c()
rep(c(4.5, 6.8, 7.2), 2)
```

```
[1] 4.5 6.8 7.2 4.5 6.8 7.2
```

1.14. Índices

R implementa una forma eficiente y flexible de acceder selectivamente a elementos de un objeto basado en una indexación interna.

Para acceder a un índice se utiliza la notación de corchetes. Por ejemplo, si tenemos un vector `x` con varios elementos y queremos acceder al segundo, usamos `x[2]`.

Esta forma de indexar los elementos de los distintos objetos nos permite realizar muchas operaciones desde el simple llamado, hasta seleccionar, eliminar o modificar valores.

Veamos algunos ejemplos:

```r
# generamos un vector x con 5 letras
x <- c("a", "b", "c", "d", "e")

# llamamos a la primer posición y nos devuelve su valor
x[1]
```

```
[1] "a"
```

```r
# llamamos a la tercer posición y nos devuelve su valor
x[3]
```

```
[1] "c"
```

```r
# llamamos a las posiciones 1 y 3 juntas mediante c()
x[c(1, 3)]
```

```
[1] "a" "c"
```

```r
# llamamos a las posiciones menos la 1 y la 4
x[-c(1, 4)]
```

```
[1] "b" "c" "e"
```

```r
# creamos otro vector y con los valores 1,2 y 5
y <- c(1, 2, 5)

# utilizamos el vector y como índice
x[y]
```

```
[1] "a" "b" "e"
```

```r
# asignamos el valor "h" a la posición 2 del vector x
x[2] <- "h"

x
```

```
[1] "a" "h" "c" "d" "e"
```

```r
# creamos el vector z eliminando la posición 5 de x
z <- x[-5]

z
```

```
[1] "a" "h" "c" "d"
```

Estos ejemplos muestran las múltiples posibilidades que ofrece el uso de índices para manipular objetos en R, lo que hace al lenguaje muy poderoso y versátil.

Cuando se aplican índices a estructuras bidimensionales, como matrices o *dataframes*, la notación general es:

```
nombre[índice de fila, índice de columna]
```

1.15. Gestión de factores

Al presentar las distintas estructuras de datos mencionamos que los factores suelen generarse a partir de vectores u otros objetos de tipo carácter.

Para entender cómo funcionan, partamos de un vector con datos categóricos:

```
respuesta <- c("Si", "No", "No", "Si", "Si", "Si", "No")
```

Creamos un vector llamado `respuesta` con 7 elementos de tipo carácter, en el que se repiten las categorías `"Si"` y `"No"`.

Podemos confirmar que se trata de un vector y que su contenido es de tipo carácter:

```
# preguntamos si sexo es un vector
is.vector(respuesta)
```

```
[1] TRUE
```

```
# visualizamos el tipo de dato de sexo
class(respuesta)
```

```
[1] "character"
```

Para crear un factor a partir de este vector debemos utilizar la función `factor()`:

```
respuesta <- factor(respuesta)

respuesta
```

```
[1] Si No No Si Si Si No
Levels: No Si
```

En la salida observamos los siete elementos y, debajo, una línea con los niveles del factor, indicados por `Levels`. Estos niveles son identificados automáticamente.

Podemos verificar cómo cambió el tipo de objeto:

```r
# preguntamos si respuesta es un vector
is.vector(respuesta)
```

```
[1] FALSE
```

```r
# preguntamos si respuesta es un factor
is.factor(respuesta)
```

```
[1] TRUE
```

```r
# visualizamos el tipo de dato de respuesta
class(respuesta)
```

```
[1] "factor"
```

Aunque visualmente vemos palabras, internamente R trata al factor como un tipo especial basado en números. ¿Por qué sucede esto? Veámoslo con más detalle:

```r
str(respuesta)
```

```
 Factor w/ 2 levels "No","Si": 2 1 1 2 2 2 1
```

La función `str()` devuelve la estructura interna del objeto. En este caso, muestra que `respuesta` tiene dos niveles y que cada elemento del vector es representado por un número (1 o 2), donde `1` corresponde a `"No"` y `2` a `"Si"`.

Esto significa que la estructura de los factores está compuesta por dos vectores: uno numérico que funciona como índice de enteros, que sustituye al vector de caracteres original, y el otro es un vector de caracteres, que contiene los niveles o categorías, a los que hace referencia el primer vector.

Para ver solo los niveles o categorías del factor podemos usar:

```r
levels(respuesta)
```

```
[1] "No" "Si"
```

En los factores nominales donde no importa el orden, la función `factor()` implementa el orden alfabético para determinar a qué índice numérico pertenece cada categoría. Es claro en el ejemplo que a `No` le asigna el `1` y a `Si` el `2`.

Pero si nos encontramos frente a una variable cualitativa ordinal vamos a necesitar indicarle a la función cual es el orden de las categorías.

Vamos al siguiente ejemplo:

```
salud <- c(4, 3, 1, 3, 2, 2, 3, 3, 1)
salud
```

```
[1] 4 3 1 3 2 2 3 3 1
```

Tenemos en el vector salud algunos códigos numéricos que representan nivel de salud de personas registradas por una encuesta donde `1` significa mala salud, `2` regular, `3` buena y `4` muy buena.

Procedemos a crear el factor `nivsalud` a partir de este vector:

```
nivsalud <- factor(
  salud,
  label = c("Mala", "Regular", "Buena", "Muy buena"),
  levels = 1:4
)

nivsalud
```

```
[1] Muy buena Buena     Mala      Buena     Regular   Regular   Buena
[8] Buena     Mala
Levels: Mala Regular Buena Muy buena
```

Aquí utilizamos dos argumentos adicionales:

- `labels`: define las etiquetas que queremos mostrar para cada categoría.
- `levels`: indica el orden de los niveles originales (en este caso, 1 a 4).

Aquí es necesario definir estos argumentos porque, a diferencia del factor `respuesta`, el vector `salud` contiene números en lugar de las palabras correspondientes a las categorías.

Hasta aquí hemos creado un factor pero si miramos sus niveles no encontraremos señales que sigan un orden específico:

```r
levels(nivsalud)
```

```
[1] "Mala"      "Regular"   "Buena"     "Muy buena"
```

Sin embargo, aún no hemos definido un **orden** explícito entre las categorías. Para hacerlo, agregamos el argumento `ordered = TRUE`:

```r
nivsalud <- factor(
  salud,
  label = c("Mala", "Regular", "Buena", "Muy buena"),
  levels = 1:4,
  ordered = TRUE
)

nivsalud
```

```
[1] Muy buena Buena     Mala      Buena     Regular   Regular   Buena
[8] Buena     Mala
Levels: Mala < Regular < Buena < Muy buena
```

Al incorporar este argumento, estamos diciendo que los niveles tienen orden natural. Esto puede observarse en la salida donde se muestra que `"Mala"` < `"Regular"` < `"Buena"` < `"Muy buena"`.

1.16. Gestión de *dataframes*

En R, la estructura utilizada para almacenar datos provenientes de fuentes externas (como archivos .csv, planillas de cálculo, bases SQL, etc.) es el *dataframe*.

A modo de ejemplo, vamos a construir un *dataframe* llamado `datos` con 4 variables y 5 observaciones. Las variables serán:

- `id`: identificador numérico entero y correlativo.
- `edad`: edad en años.
- `sexo`: codificado como `"V"` para varón y `"M"` para mujer.
- `trabaja`: variable lógica, donde `TRUE` (`T`) representa que la persona trabaja y `FALSE` (`F`) que no lo hace.

```r
# construimos el vector id
id <- 1:5

# construimos el vector edad
edad <- c(23, 43, 12, 65, 37)

# construimos el vector sexo
sexo <- c("V", "M", "M", "V", "M")

# construimos el vector trabaja
trabaja <- c(T, T, F, F, T)

# construimos el dataframe datos
datos <- data.frame(id, edad, sexo, trabaja)
```

Aunque para el ejemplo construimos un *dataframe* manualmente, lo habitual en la práctica es leer archivos externos que se importan directamente como *dataframes*.

Algunas de las funciones generales que podemos aplicar son:

```r
# pedimos el número de columnas (variables)
ncol(datos)
```

```
[1] 4
```

```r
# pedimos el número de filas (registros u observaciones)
nrow(datos)
```

```
[1] 5
```

```r
# pedimos las dimensiones del dataframe (observaciones,variables)
dim(datos)
```

```
[1] 5 4
```

También podemos visualizar como se compone el objeto `datos` aplicando `str()` que devuelve la estructura interna de cualquier objeto en R.

```r
str(datos)
```

```
'data.frame':   5 obs. of  4 variables:
 $ id     : int  1 2 3 4 5
 $ edad   : num  23 43 12 65 37
 $ sexo   : chr  "V" "M" "M" "V" ...
 $ trabaja: logi  TRUE TRUE FALSE FALSE TRUE
```

Esta salida nos informa que:

- `datos` es un *dataframe* con 5 observaciones y 4 variables.
- `id` es un entero (`int`).
- `edad` es numérica (`num`).
- `sexo` es de tipo carácter (`chr`).
- `trabaja` es lógica (`logi`).

Además, `str()` muestra los primeros valores de cada variable, que en este caso son todos, ya que el *dataframe* tiene solo 5 registros.

Cuando necesitemos llamar al contenido de alguna columna o variable del *dataframe*, utilizamos la siguiente notación:

```r
nombre_del_dataframe$nombre_de_la_variable
```

Por ejemplo, si queremos mostrar el contenido de la variable `sexo` del objeto `datos` hacemos:

```r
datos$sexo
```

```
[1] "V" "M" "M" "V" "M"
```

Esto devuelve todos los valores de la variable `sexo`.

También podemos acceder a los elementos del *dataframe* usando **indexación por filas y columnas**:

```r
# pedimos la tercer variable (sexo)
datos[, 3]
```

```
[1] "V" "M" "M" "V" "M"
```

Observemos que las dos salidas son idénticas, dado que muestran todas las observaciones de la variable `sexo`, aunque la solicitud sea de manera diferente.

Algunos otros ejemplos de uso de índices:

```
# observación 1 de la variable 2
datos[1, 2]
```

```
[1] 23
```

```
# observación 4 de todas las variables
datos[4, ]
```

```
  id edad sexo trabaja
4  4   65    V   FALSE
```

```
# observación 1,2 y 3 de la variable 3
datos[1:3, 3]
```

```
[1] "V" "M" "M"
```

```
# observación 5 de las variables 1 y 4
datos[5, c(1, 4)]
```

```
  id trabaja
5  5    TRUE
```

Por último, vamos podemos mostrar y gestionar los nombres de las variables con la función `names()`:

```
names(datos)
```

```
[1] "id"      "edad"    "sexo"    "trabaja"
```

1.17. Fórmulas

En R, las fórmulas se utilizan principalmente para describir modelos estadísticos, y las vamos a emplear a lo largo de toda la cursada.

Las fórmulas se escriben utilizando operadores como `~`, `+`, `*`, entre otros. Se usan para especificar modelos como regresiones, ANOVA, pruebas de hipótesis y, en algunos casos, también para generar ciertos tipos de gráficos.

Formula genérica

Aquí la formula está dada por el operador `~`, a la izquierda está la variable de respuesta o dependiente, a la derecha la o las variables explicativas o independientes. El esquema general es el siguiente:

```
variable_dependiente ~ variable_independiente
```

El símbolo `~` (llamado virgulilla) puede interpretarse como *"es modelada por"* o *"en función de"*.

Por ejemplo, en una regresión lineal se utiliza la función base `lm()` y el argumento principal de esta función es una formula:

```
regresion_lineal <- lm(variable_respuesta ~ variable_explicativa, data = datos)
```

Además de la fórmula, la función `lm()` incluye el argumento `data =`, que es fundamental: allí se le indica a R el *dataframe* en el que debe buscar las variables involucradas en el modelo.

La siguiente tabla muestra los usos de los los operadores más comunes dentro de una formula:

Operador	Ejemplo	Descripción
+	+x	Incluye la variable x
-	-x	Excluye la variable x
:	x : z	Incluye la interacción de la variable x con z
*	x * z	Incluye ambas variables y la interacción entre ellas

2 Introducción a RStudio

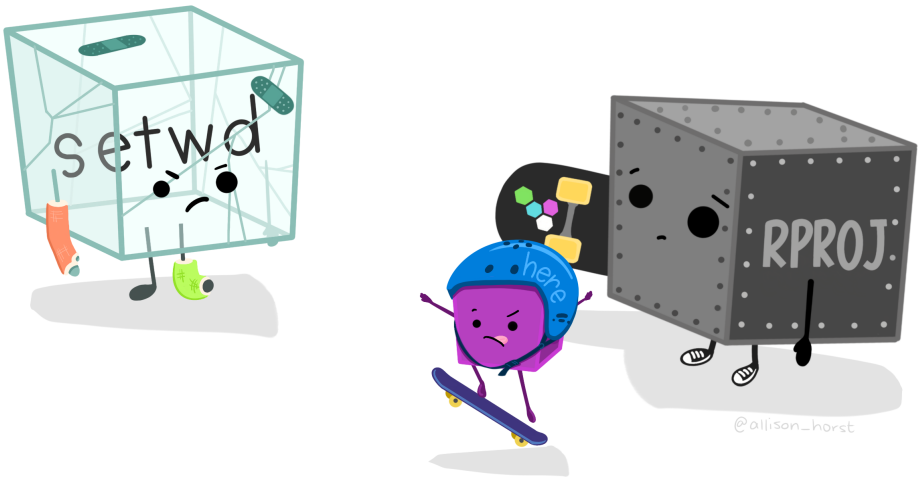


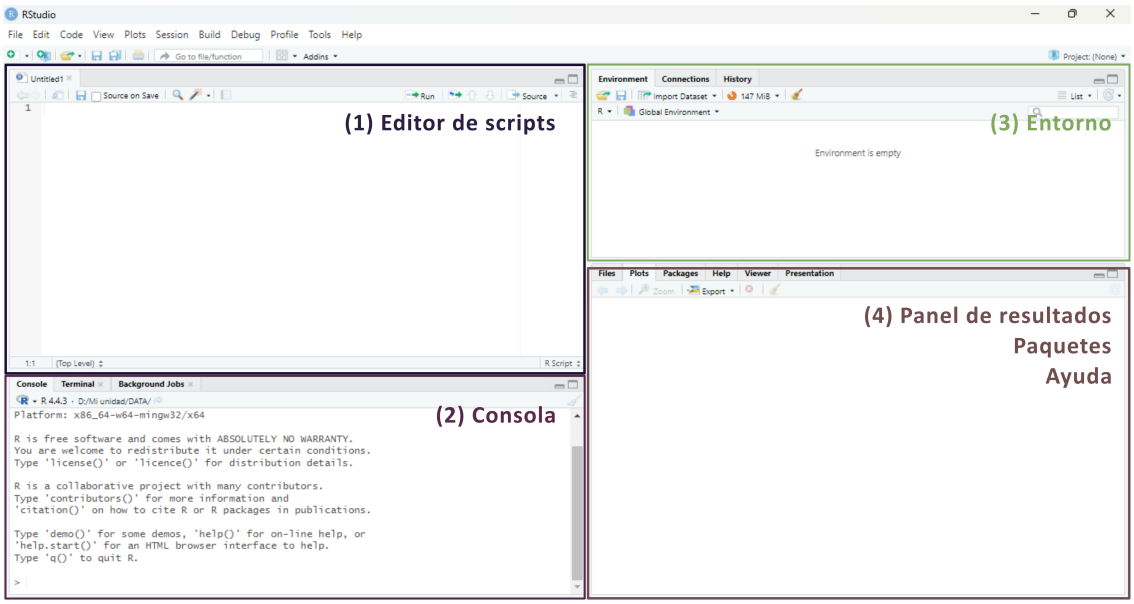
Figura 2.1: Artwork por @allison_horst

2.1. Primeros pasos y generalidades

RStudio Desktop (2025, Posit Software) es un **entorno de desarrollo integrado** (*IDE*, por sus siglas en inglés) diseñado específicamente para trabajar con el lenguaje R.

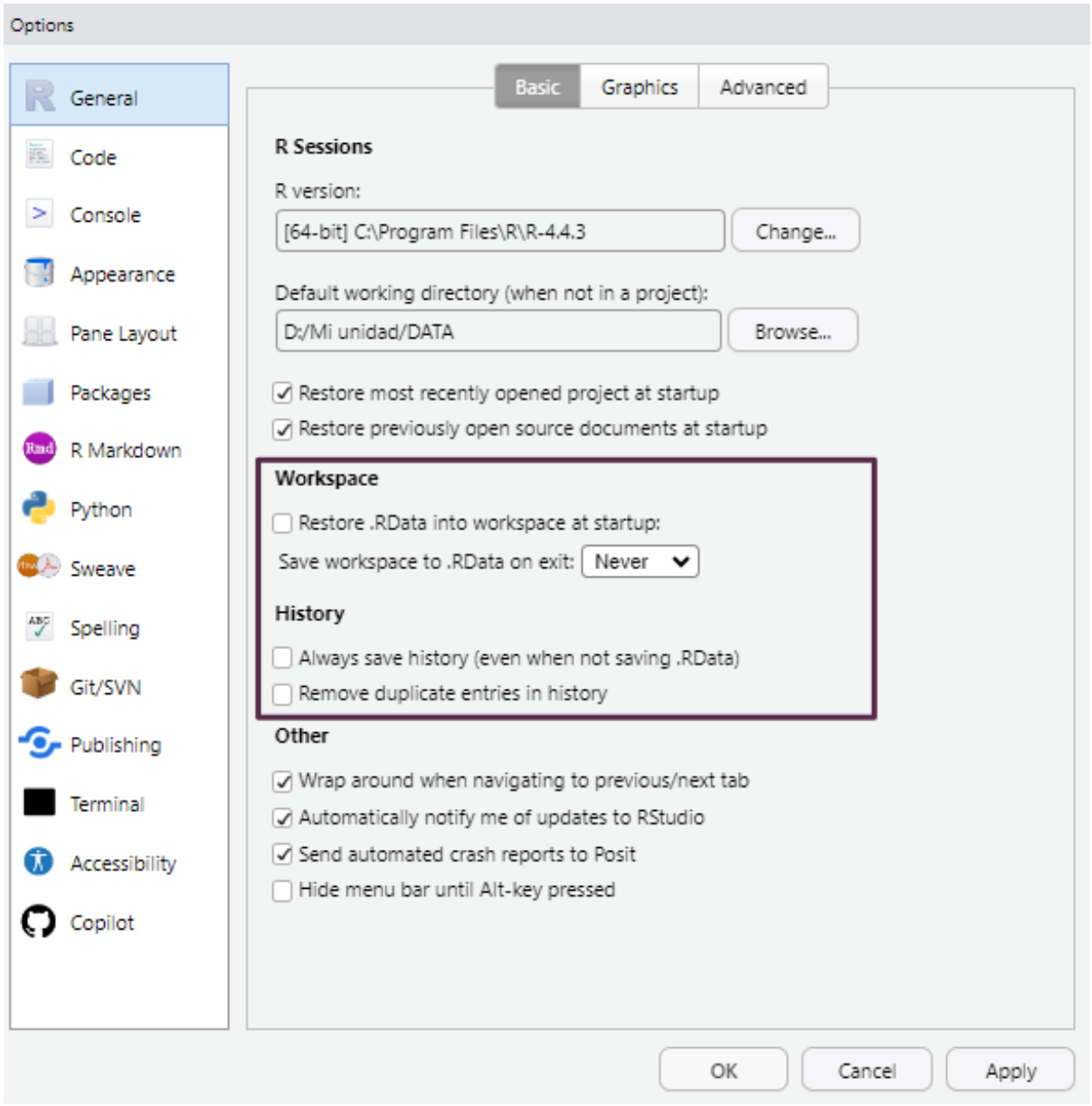
Es una herramienta **multiplataforma y de código abierto** que facilita la programación, el análisis de datos y la elaboración de informes científicos. Ofrece una integración fluida con otros componentes del ecosistema de R, como R Markdown, Quarto, control de versiones (por ejemplo, Git) y la gestión de proyectos.

RStudio presenta una interfaz unificada compuesta por distintos paneles, lo que permite organizar el trabajo de forma clara y eficiente:



1. **Editor de scripts** (*Source*): permite crear y editar scripts de R, así como documentos R Markdown o Quarto.
2. **Consola de R** (*Console*): muestra la salida del código ejecutado y ejecuta código de R de forma inmediata.
3. **Entorno** (*Environment*): muestra los objetos creados durante la sesión, como vectores, *dataframes* o funciones.
4. **Panel de resultados** (*Output*): presenta los gráficos, tablas, visualizaciones en HTML y también incluye un explorador de archivos, visor de paquetes instalados y panel de ayuda.

Para garantizar la reproducibilidad de los resultados, es recomendable evitar el guardado automático del historial de objetos entre sesiones, ya que puede generar confusión. Para desactivar esta opción, ir a `Tools > Global Options` y desmarcar las casillas de las opciones **Workspace** y **History** como se muestra en la siguiente imagen:



2.2. Proyectos de RStudio

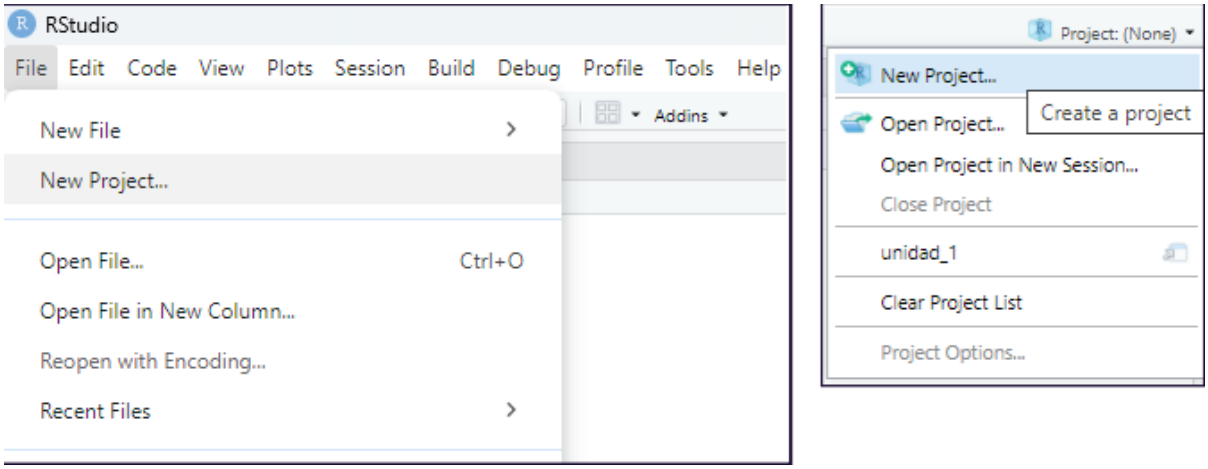
Los proyectos de RStudio permiten organizar de forma estructurada todo el material asociado a un análisis: scripts, informes, bases de datos, imágenes, etc. Cada proyecto se vincula a una carpeta específica del sistema de archivos, y RStudio la utiliza como directorio de trabajo por defecto. Esta carpeta puede estar ubicada en cualquier parte del sistema de almacenamiento que deseemos (disco

rígido, pendrive, disco externo, etc).

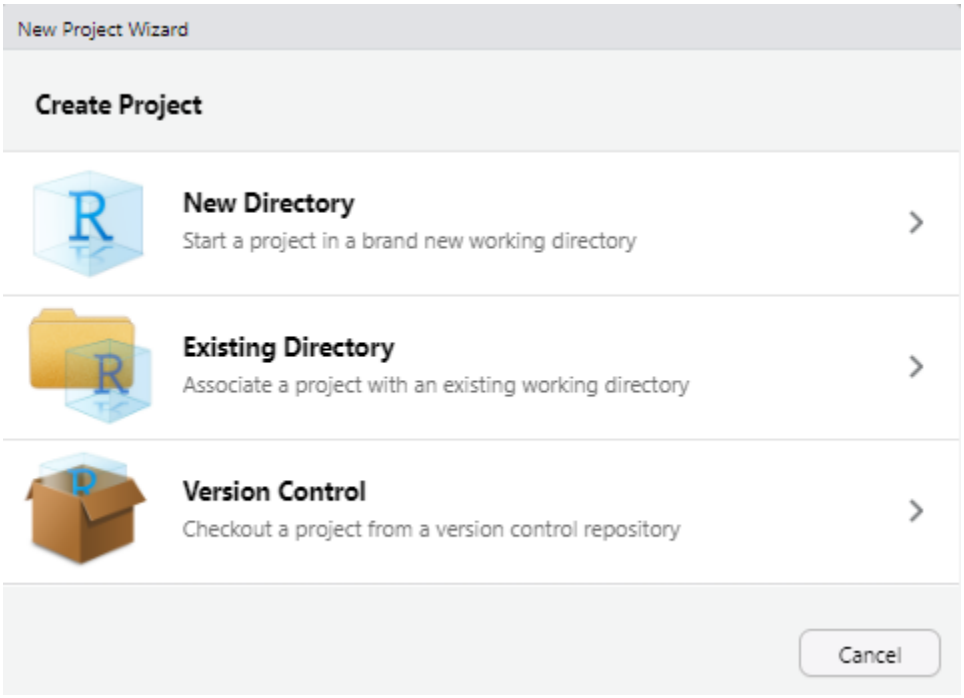
Trabajar con proyectos facilita la importación de datos y evita errores relacionados con rutas relativas o absolutas.

2.2.1. Crear un proyecto

Para crear un nuevo proyecto, se puede utilizar el menú `File > New Project...`. También accedemos a generar un proyecto nuevo a partir de pulsar el acceso directo `New Project...` ubicado en la esquina superior derecha de la interfaz:

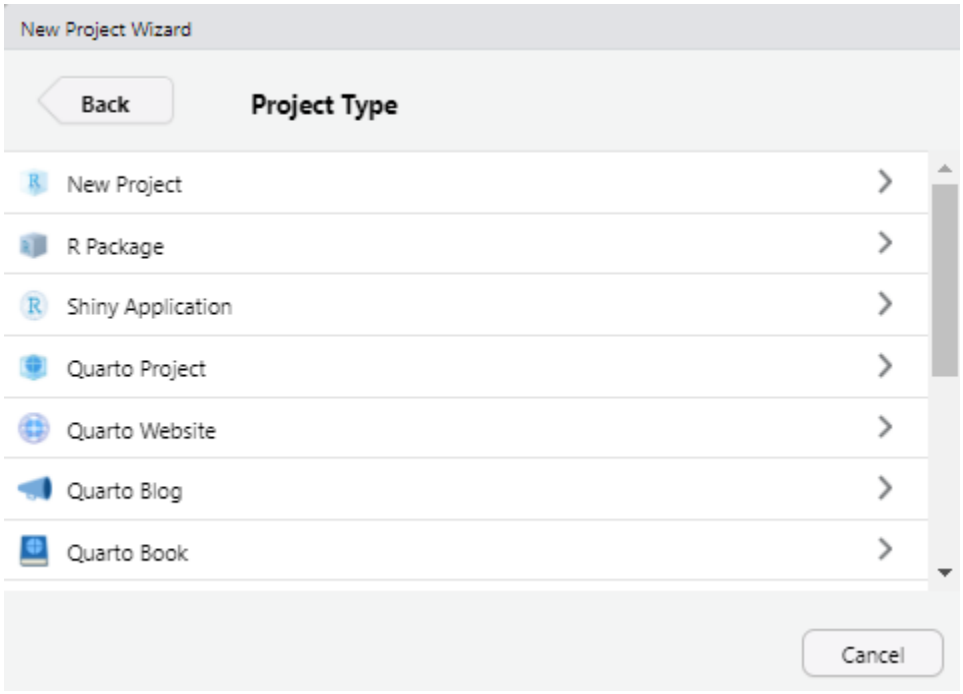


En cualquiera de los dos casos aparecerá un cuadro de diálogo que presenta tres opciones para crear el **nuevo proyecto** de RStudio:



- **New Directory**: crea una nueva carpeta para el proyecto a la que deberemos asignarle un nombre, es la opción más habitual. Todos los archivos de configuración aparecerán asociados a esta nueva carpeta.
- **Existing Directory**: vincula el proyecto a una *carpeta ya existente* que contenga archivos previos con los que deseamos trabajar.
- **Version Control**: permite clonar un repositorio (Git o SVN). Esta opción no se utilizará durante el curso.

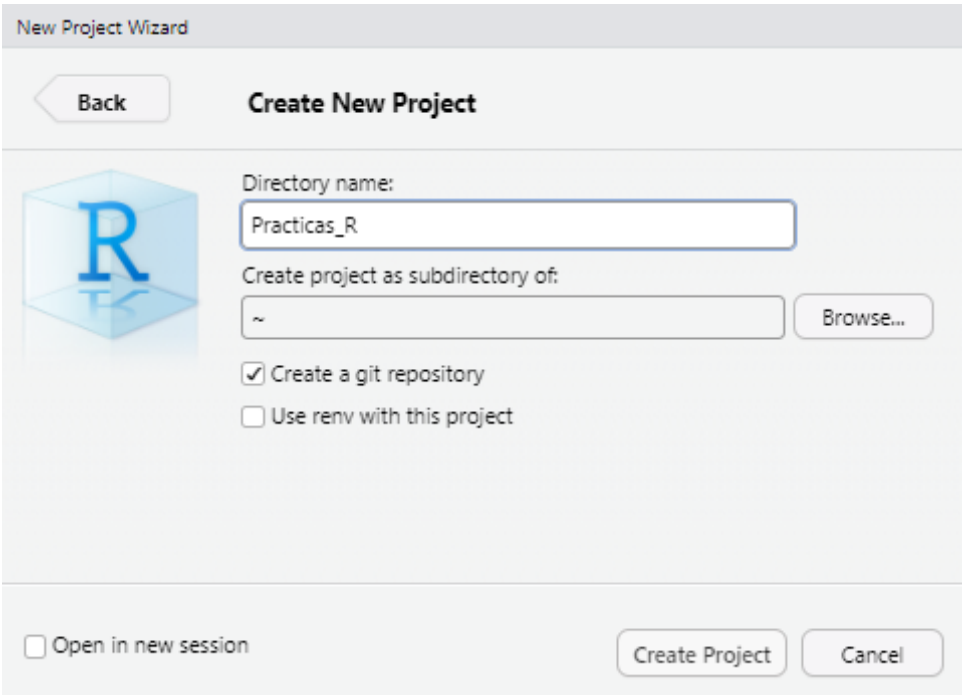
Una vez que creamos un nuevo proyecto con la opción **New Directory**, aparecerá una pantalla con una lista de tipos de proyectos que se pueden crear en RStudio:



Durante este curso utilizaremos siempre la **primera opción** (**New Project**). Las demás opciones están pensadas para usos más específicos, como el desarrollo de paquetes de R, sitios web o documentos con Quarto o R Markdown.

Al seleccionar **New Project**, se abrirá una nueva ventana con los siguientes campos:

- **Directory name:** aquí debemos escribir el nombre del nuevo directorio (carpeta) que también será el nombre del proyecto. Por ejemplo, podemos llamarlo `Practicas_R`.
- **Create project as subdirectory of:** este campo permite definir en qué ubicación del sistema de archivos se guardará el proyecto. Podemos hacer clic en el botón **Browse…** para abrir el explorador de archivos y seleccionar la carpeta contenedora. En nuestro ejemplo, lo ubicaremos dentro de **Mis Documentos**.



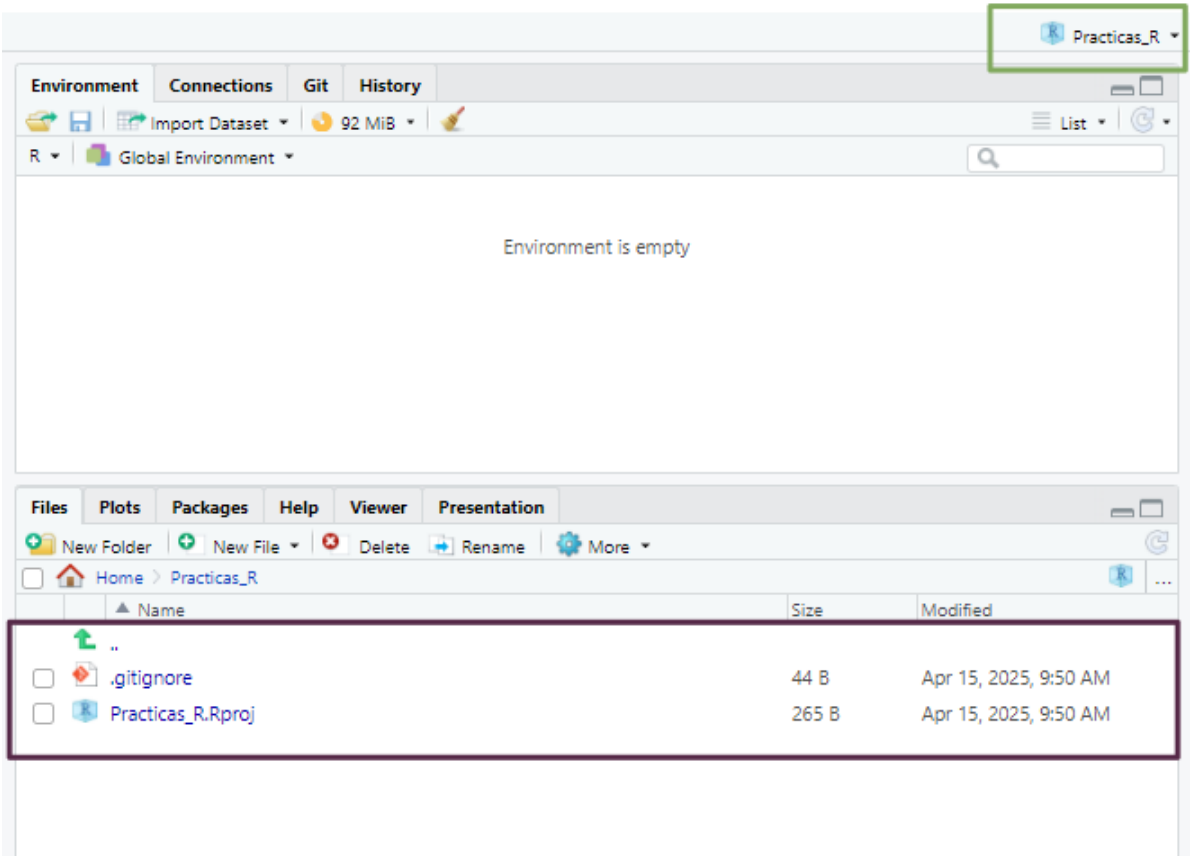
Una vez completados estos campos, hacemos clic en **Create Project**.

Los proyectos en RStudio tienen entornos independientes, lo que significa que si cerramos un proyecto o cambiamos a otro, la configuración de cada uno se mantendrá inalterable sin interferir con los demás.

Esto incluye los scripts abiertos, el directorio de trabajo, las pestañas que dejamos activas y otros elementos del entorno que puedan ser necesarios para continuar con un análisis. Este sistema permite mantener organizados los distintos trabajos que llevamos adelante.

Echemos un vistazo a lo que RStudio realizó:

- En primer lugar el panel **Files** (pantalla inferior derecha) apunta a la nueva carpeta **Practicas_R** y dentro de ella vemos un nuevo archivo el nombre del proyecto y la extensión `.Rproj`. Este archivo contiene todas las configuraciones para su proyecto.
- El otro cambio se observa en la parte superior derecha, que muestra el nombre del proyecto. Si hacemos click en él, se desplegará el menú de proyectos. Desde aquí se puede abrir y cerrar proyectos, navegar rápidamente a proyectos que se han abierto recientemente y configurar las opciones de RStudio para cada uno de ellos.



2.2.2. Abrir un proyecto existente

Cuando el proyecto ya existe, sea porque lo creamos nosotros o porque alguien nos pasó una carpeta con un proyecto de RStudio creado vamos a visualizar dentro de esa carpeta un archivo con extensión `.Rproj`.

La forma más veloz para abrir el proyecto es ejecutar este archivo (debería abrir una sesión de RStudio con el proyecto activo). La otra forma es desde el menú superior derecho de RStudio en la opción **Open project…** y luego buscando en nuestro directorio el mismo archivo `.Rproj`.

Nota

El menú de proyectos del área superior derecha va guardando como elementos recientes los proyectos que se van abriendo y también es una forma rápida de acceder a ellos pulsando sobre estos atajos.

2.3. Scripts en RStudio

Como vimos **anteriormente**, un script es un archivo de código que contiene una secuencia de instrucciones escritas en R. Estos scripts pueden ser reutilizados, modificados y compartidos, lo que los convierte en una herramienta fundamental para garantizar la reproducibilidad del trabajo.

2.3.1. Crear un nuevo script

Tenemos varias formas de crear un script nuevo:

- Desde el menú superior: `File > New File > R Script`
- Con el atajo de teclado: `Ctrl + Shift + N`
- Desde la barra de herramientas: presionando el ícono 

2.3.2. Ejecutar scripts

La forma habitual de ejecutar el contenido de un script es línea por línea, usando alguna de las siguientes opciones:

- Presionando el botón 

del editor de código de RStudio
- Mediante el atajo de teclado `Ctrl + Enter`

Para ejecutar una línea, simplemente ubicamos el cursor en cualquier parte de ella y presionamos el comando correspondiente. Luego de ejecutarse, el cursor avanzará automáticamente a la siguiente línea de código.

Mientras ejecutamos cada línea debemos ir observando la salida en la consola (panel inferior izquierdo) y también los cambios que se dan en el panel *Environment* (panel superior derecho) donde aparecerán los objetos que vayamos creando y manipulando.

2.3.3. Edición de scripts

Modificar o agregar líneas al script puede hacerse directamente en el editor. Cada vez que realizamos un cambio, es necesario **volver a ejecutar la línea** o bloque modificado para que los cambios se reflejen en el entorno de trabajo.

Podemos probar y modificar tantas veces como sea necesario. Sin embargo, debemos tener presente que cada manipulación en los objetos se mantiene hasta que se vuelvan a cambiar y a veces, cuando los objetos están vinculados con otras líneas de código posteriores tenemos que tener cuidado que se mantenga la consistencia del script.

Por ejemplo: si definimos un vector numérico para realizar cálculos matemáticos, pero luego lo sobrescribimos con un valor de tipo caracter, los cálculos posteriores producirán un error y RStudio nos informará de esto en la consola.

Por eso, es clave observar el contenido de los objetos en el panel **Environment**, lo que nos ayuda a evitar errores y operaciones incoherentes.

2.3.4. Guardado de scripts

Cualquier agregado o modificación que hayamos realizado al script y nos interese mantener nos obligará a guardar el archivo de código editado.

Existen distintas formas de guardar un script:

- Desde el menú superior: `File > Save`
- Con el atajo de teclado: `Ctrl + S`
- Presionando el ícono del disquete azul

Si en cambio quisiera guardarlo como otro archivo para mantener el script original, podemos guardarlo con diferente nombre o en otra ubicación mediante `File > Save As...`

2.3.5. Abrir scripts existentes

Los scripts que construyamos o nos compartan siempre tendrán extensión `.R` y, por lo general, se encontrarán dentro de un proyecto.

Para abrir estos archivos `.R` podemos:

- Desde el menú superior: `File > Open file...`
- Con el atajo de teclado: `Ctrl + O`
- Haciendo click sobre el archivo desde el panel *Files*
- Presionando el botón de la carpeta amarilla

Esto abrirá el script en una nueva pestaña dentro del editor de código.

2.3.6. ¿Cómo trabajaremos en este curso?

En general, utilizaremos scripts dentro de **proyectos de RStudio**. La secuencia recomendada será:

1. **Descargar** desde el Aula Virtual un archivo comprimido conteniendo la carpeta, el proyecto, los scripts y archivos de datos.
2. **Descomprimir** el archivo en la ubicación que deseamos (recomendamos crear una carpeta destinada al curso).
3. Abrir la carpeta y ejecutar el archivo de proyecto `.Rproj`.
4. Una vez abierto RStudio con el proyecto activo, ubicamos los scripts desde el panel **Files**.
5. **Ejecutar** cada línea del script, leyendo la documentación del código y observando la salida en la consola y los cambios en el entorno

2.4. Herramientas de RStudio

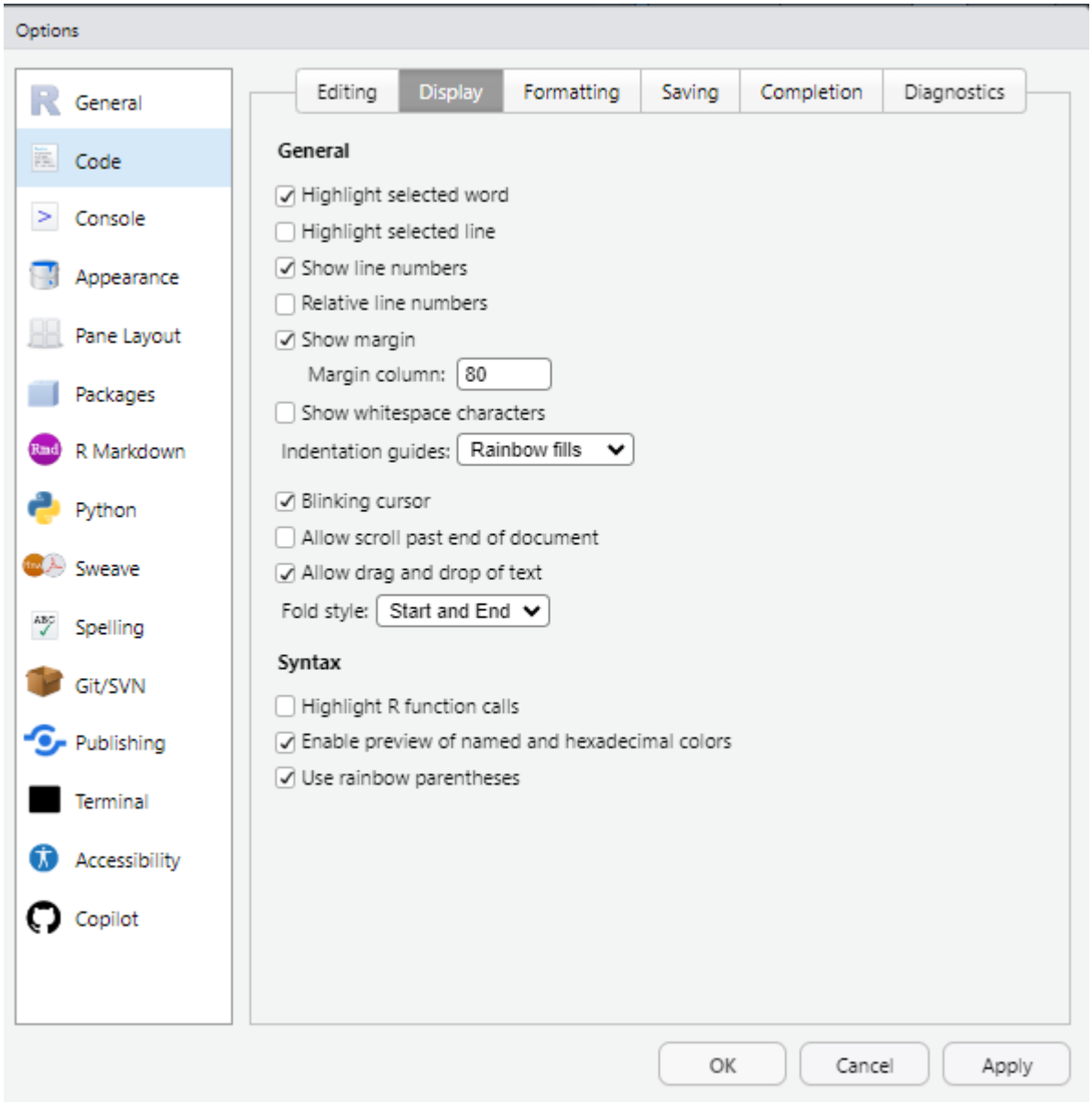
Algunas de las herramientas fundamentales de RStudio son el asistente de código, la ayuda en línea y el historial de comandos.

2.4.1. Asistente de código

Al escribir en el editor o la consola, la tecla `Tab` activa el autocompletado de funciones, nombres de objetos y argumentos, agilizando la escritura y reduciendo errores de sintaxis. En versiones recientes, el asistente también permite la previsualización de colores en los gráficos, resaltar los paréntesis de cierre en funciones anidadas con distintos colores y gestionar automáticamente la indentación del código.

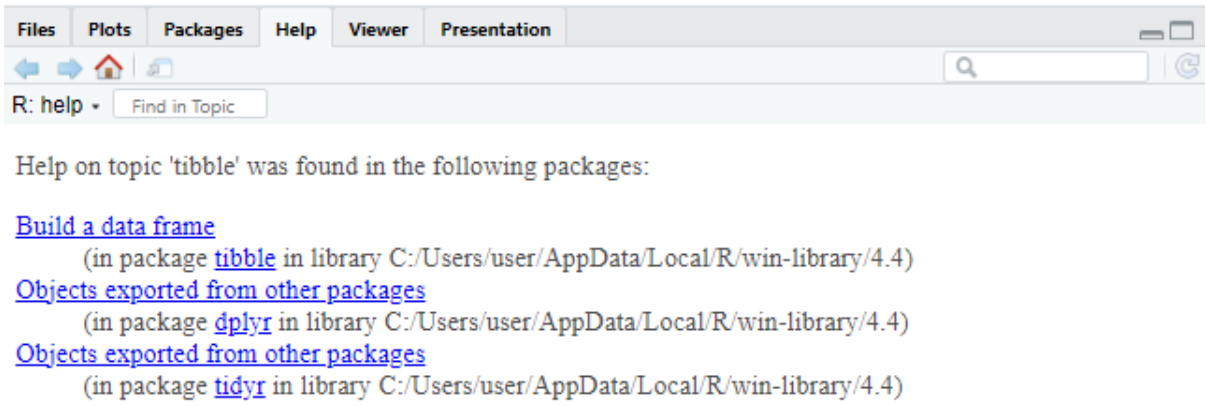


Muchas de estas opciones se pueden configurar desde el menú `Code` y desde `Tools > Global Options > Code`(pestañas **Editing** y **Display**).



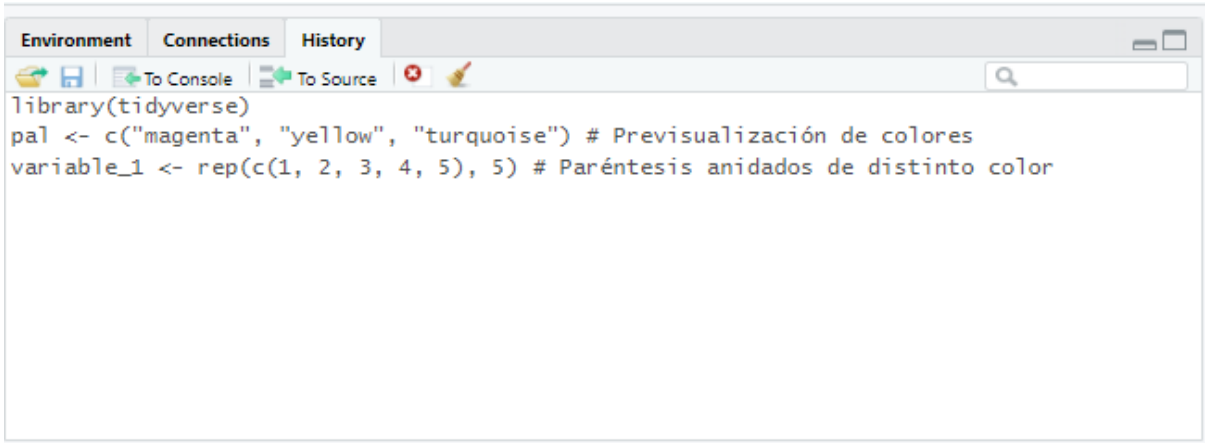
2.4.2. Ayuda en línea

Al posicionar el cursor sobre el nombre de una función en el editor y presionar `F1`, se accede directamente a la documentación correspondiente en el panel **Help** (habitualmente ubicado en la esquina inferior derecha).



2.4.3. Historial de comandos

En la consola, al usar las teclas de flecha arriba/abajo, se puede navegar por los comandos ejecutados durante la sesión actual. Además, el panel **History** (parte superior derecha) almacena los comandos de todas las sesiones previas, permitiendo reutilizarlos con un clic en **To Console** (`Enter`) o **To Source** (`Shift + Enter`), según se desee insertarlos en la consola o en el script activo.



2.5. Atajos de teclado (Windows)

Menú	Descripción
Archivo (File)	
Ctrl + Shift + N	Crea un nuevo script
Ctrl + O	Abre un script guardado
Ctrl + S	Guarda el script activo
Ctrl + W	Cierra el script activo
Ctrl + Q	Sale del programa RStudio
Edición (Edit)	
Ctrl + F	Abre la ventana de búsqueda (para buscar palabras dentro de un script)
Ctrl + L	Limpia la consola
Código (Code)	
Ctrl + Enter	Ejecuta la línea de código donde está situado el cursor
Ctrl + Alt + R	Ejecuta todo el código del script activo
Ctrl + Shift + N	Inserta nueva sección de código
Ctrl + Shift + R	Inserta nueva sección de comentarios de texto
Sesión (Session)	
Ctrl + Shift + H	Abre la ventana para establecer directorio de trabajo
Ctrl + Shift + F10	Reinicia la sesión de R
Herramientas (Tools)	
Alt + Shift + K	Abre la lista de ayuda de atajos de teclado

2.6. Paquetes

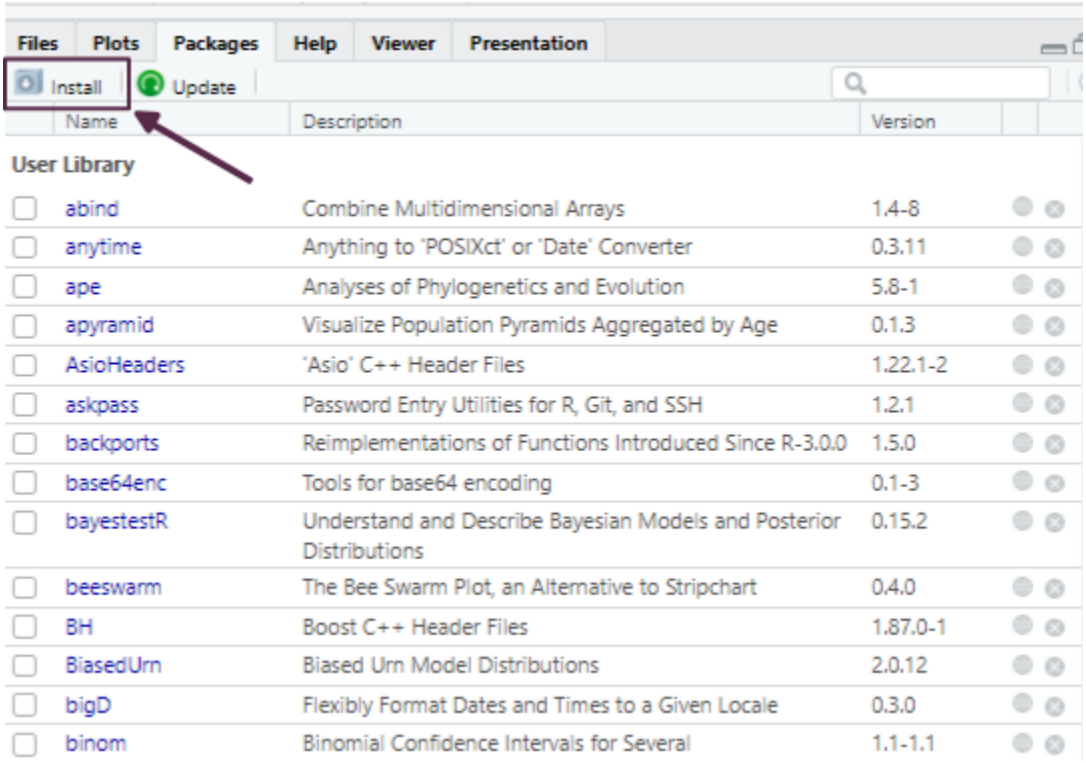
Existen dos formas principales de descargar paquetes: directamente desde RStudio o desde el **sitio web de CRAN**, descargándolos como archivos comprimidos. Si el equipo cuenta con conexión a Internet, lo más práctico es realizar la descarga directamente desde RStudio. En cambio, si no se dispone de acceso permanente a la red, es posible descargar los paquetes desde otro equipo y luego transferirlos como archivos `.zip` o `.tar.gz` al equipo donde se encuentra instalado R.

Cuando accedemos al sitio web de CRAN y buscamos un paquete específico, encontraremos información útil como una breve descripción del paquete, el número de versión, la fecha de publicación, el nombre del autor, documentación asociada y enlaces de descarga específicos para cada sistema operativo.

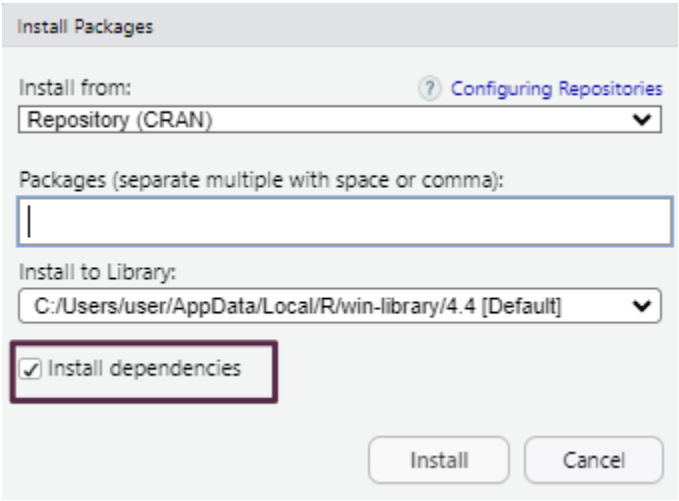
Dado que la mayoría de las computadoras hoy en día cuentan con acceso a Internet, en este curso nos enfocaremos en la instalación y activación de paquetes directamente desde RStudio.

Dentro del entorno de RStudio, la gestión de paquetes se realiza desde la pestaña *Packages*, ubicada en el panel inferior derecho. Esta interfaz facilita tareas como la instalación, actualización y activación de paquetes, y cada acción que realizamos desde la interfaz gráfica se traduce internamente en la ejecución de funciones del lenguaje R que pueden observarse en la consola.

Para instalar un paquete nuevo, simplemente pulsamos el botón *Install* dentro de la pestaña *Packages*, lo cual abrirá una ventana emergente.



Allí podemos escribir el nombre del paquete que deseamos instalar. Para asegurarnos de que también se descarguen e instalen automáticamente los paquetes de los que depende, es importante tildar la opción *Install dependencies*.



Como alternativa, también podemos realizar la instalación desde el editor de scripts mediante el siguiente comando:

```
install.packages("nombre_del_paquete", dependencies = TRUE)
```

> Para facilitar la instalación de los paquetes requeridos durante el curso, recomendamos descargar el archivo **"Paquetes y datos para el curso"**, descomprimirlo y ejecutar el script que se encuentra en su interior.

2.7. Lectura de archivos de datos

R permite importar tablas de datos desde diversos formatos, tanto utilizando funciones de **R base** como funciones provistas por paquetes específicos.

El formato más común es el **texto plano** (ASCII), donde los valores están organizados en columnas separadas por caracteres delimitadores. Los separadores más habituales incluyen:

- Coma (`,`)
- Punto y coma (`;`)
- Tabulación (`\t`)
- Barra vertical (`|`)

Estos archivos suelen tener una **cabecera** (*header*) en la primera fila con los nombres de las variables, y cada columna debe contener datos del mismo tipo (números, texto, lógicos, etc.).

Para importar correctamente un archivo es importante conocer su estructura:

- Si incluye o no cabecera.
- Qué carácter se usa como separador.
- El tipo de codificación (UTF-8, Latin1, etc.).

Dado que son archivos de texto, pueden visualizarse con editores simples como el Bloc de Notas o desde RStudio, lo que facilita su inspección previa.

Para cargar los datos desde un archivo de texto plano usamos el código:

```
datos <- read.xxx("mis_datos.xxx")
```

(Se debe reemplazar `read.xxx()` por la función correspondiente: `read.table()`, `read.csv()`, `read_delim()`, `read_excel()`, etc., según la extensión del archivo).

R también permite cargar bases de datos incluidas en paquetes instalados mediante:

```
data(nombre_datos)

datos <- nombre_datos
```

2.8. Buenas prácticas

Adoptar buenas prácticas desde el inicio mejora la reproducibilidad, facilita el trabajo colaborativo y reduce errores. Algunas recomendaciones clave son:

- Trabajar **siempre** dentro de un **proyecto de RStudio** (`.Rproj`). Esto permite organizar los archivos, mantener rutas relativas consistentes y acceder a funcionalidades específicas como control de versiones o panel de archivos integrados.
- Incluir al comienzo de cada script las líneas de **activación de paquetes** necesarios, utilizando la función `library()`.
- **Cargar los datos** una vez activados los paquetes, para garantizar que todas las funciones requeridas estén disponibles.
- Documentar el código mediante **comentarios** iniciados con #. Esto permite entender qué hace cada bloque de código, facilitando futuras modificaciones o revisiones.
- **Usar espacios e indentación adecuada** para mejorar la legibilidad. Esto es especialmente importante en estructuras anidadas (como condicionales, bucles o funciones).

3 Introducción a `tidyverse`



3.1. Introducción

Tidyverse es el nombre que recibe el conjunto de paquetes desarrollados y/o promovidos por **Hadley Wickham** (jefe científico en Posit/RStudio) y su equipo, orientado al trabajo de ciencia de datos con R (Wickham et al. 2019). Estos paquetes están diseñados para integrarse de manera coherente, compartiendo una misma filosofía de diseño conocida como ***The tidy tools manifesto***.

Los cuatro principios básicos sobre los que se construye *tidyverse* son:

- Reutilización de estructuras de datos
- Resolución de problemas complejos combinando varias piezas sencillas
- Uso de programación funcional
- Diseño orientado a las personas

Los paquetes incluidos cubren todas las etapas del análisis de datos dentro de R: importación y ordenamiento de los datos (*tidy data*), transformación, visualización, modelado y la posterior comunicación de resultados.

La palabra *tidy* se traduce como “ordenado”, y hace referencia a una estructura específica que deben cumplir los datos:

- Cada ***variable*** es una *columna* de la tabla de datos.
- Cada ***observación*** es una *fila* de la tabla de datos.
- Cada ***tabla*** responde a una *unidad de observación o* análisis.



Además de los paquetes principales, al instalar `tidyverse` se incluyen otros que permiten trabajar con fechas, cadenas de caracteres o factores, también siguiendo los mismos principios.

Uno de los objetivos de los desarrolladores fue dotar a la sintaxis de estos paquetes de una **gramática** clara: funciones cuyos nombres y argumentos permiten construir “*frases*” que sean semánticamente comprensibles. Un ejemplo de esto se ve en el paquete `dplyr`, donde la mayoría de las funciones son verbos en inglés como `filter()`, `mutate()`, `summarise()`, lo que facilita su lectura y comprensión.

El paquete `tidyverse` puede instalarse desde el repositorio oficial CRAN mediante el menú *Packages* de RStudio, o ejecutando el siguiente código:

```
install.packages("tidyverse")
```

Una vez instalado, se activa mediante:

```
library(tidyverse)
```

Al activarlo, se muestra un mensaje con la versión instalada, la lista de paquetes que se cargan automáticamente y posibles conflictos de nombres entre funciones. Esto es habitual cuando se utilizan múltiples paquetes, ya que algunas funciones pueden llamarse igual. Por ejemplo, la función `filter()` existe tanto en el paquete `stats` como en `dplyr`. Al cargar `tidyverse`, R avisa de esta superposición:

```
dplyr::filter() masks stats::filter()
```

Cuando necesitamos asegurarnos de que estamos usando la función de un paquete específico, se recomienda usar la notación `::`, por ejemplo:

```r
# Función filter() del paquete stats
stats::filter()

# Función filter() del paquete tidyverse
dplyr::filter()
```

Una estrategia útil cuando trabajamos con varios paquetes es cargar `tidyverse` al final de la lista de paquetes, para que sus funciones sobrescriban las de otros paquetes si fuese necesario:

```r
library(stats)
library(tidyverse)
```

Los paquetes que se instalan con la versión actual de `tidyverse` pueden consultarse ejecutando:

```r
tidyverse_packages()
```

```
 [1] "broom"         "conflicted"    "cli"           "dbplyr"
 [5] "dplyr"         "dtplyr"        "forcats"       "ggplot2"
 [9] "googledrive"   "googlesheets4" "haven"         "hms"
[13] "httr"          "jsonlite"      "lubridate"     "magrittr"
[17] "modelr"        "pillar"        "purrr"         "ragg"
[21] "readr"         "readxl"        "reprex"        "rlang"
[25] "rstudioapi"    "rvest"         "stringr"       "tibble"
[29] "tidyr"         "xml2"          "tidyverse"
```

Además, existen muchos otros paquetes que siguen la misma filosofía pero no están incluidos por defecto. En esos casos, deben instalarse y activarse individualmente.



Para profundizar el uso de `tidyverse`, se recomienda consultar:
Sitio oficial: https://www.tidyverse.org/
Libro *R para Ciencia de Datos*: https://es.r4ds.hadley.nz/
Nueva versión (*r4ds 2e*): https://r4ds.hadley.nz/
EpiRhandbook en español: https://www.epirhandbook.com/es/index.es.html



3.2. *Dataframes* con `tibble`

Uno de los paquetes que forman parte del núcleo básico de *tidyverse* es `tibble` (Müller y Wickham 2026), que introduce una versión moderna del objeto `data.frame`. Todas las funciones que generan tablas de datos en *tidyverse* devuelven objetos *tibble* (`tbl_df`), los cuales son más eficientes y amigables para el flujo de trabajo.

Las principales ventajas de trabajar con `tibble` son:

- Impresión en consola más legible y controlada (muestran un número limitado de filas y columnas).
- No cambian automáticamente el tipo de datos.
- Permiten nombres de columnas con espacios o caracteres especiales si se encierran entre comillas invertidas ` (aunque **no se recomienda**).

Para crear un objeto *tibble* manualmente usamos el siguiente código:

```
datos <- tibble(
  nombre = c("Ana", "Luis", "María"),
  edad = c(34, 28, 45),
  altura = c(1.65, 1.80, 1.70)
)
```

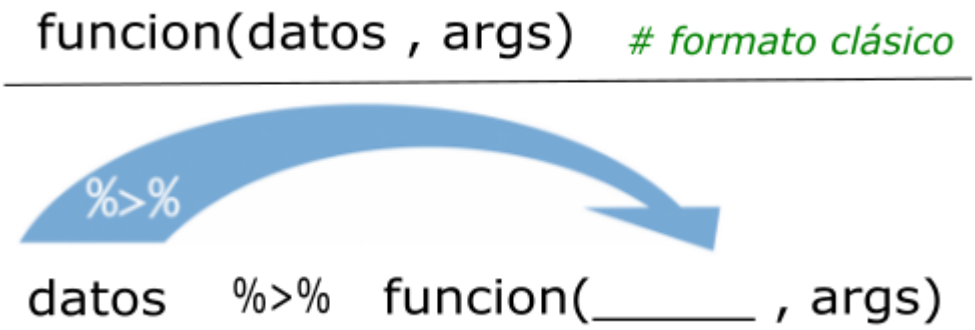


3.3. Tuberías con `magrittr`

Una de las incorporaciones más útiles y transversales del ecosistema *tidyverse* es el uso de "*tuberías*" o *pipe operators*. Una tubería conecta un bloque de código con otro, permitiendo encadenar operaciones de manera legible. El operador %> %, proveniente del paquete `magrittr` (Bache y Wickham 2026), transforma llamadas de funciones anidadas (con múltiples paréntesis) en una secuencia de pasos más simple de leer y escribir.

A partir de la versión 4.1.0 de R, también se incorporó una **tubería nativa** (|>), con un comportamiento muy similar. Ambas opciones son válidas y su uso es prácticamente equivalente.

Este enfoque refleja el principio de que *cada función representa un paso en una secuencia lógica de transformación de datos*. La forma de trabajar se puede ver en el siguiente esquema general:



A continuación, mostramos un ejemplo comparativo de cómo cambia la sintaxis usando el dataset incorporado en R `mtcars`, que contiene datos sobre autos:

```
head(sqrt(mtcars))
```

```
                      mpg      cyl     disp        hp     drat       wt
Mazda RX4         4.582576 2.449490 12.64911 10.488088 1.974842 1.618641
Mazda RX4 Wag     4.582576 2.449490 12.64911 10.488088 1.974842 1.695582
Datsun 710        4.774935 2.000000 10.39230  9.643651 1.962142 1.523155
Hornet 4 Drive    4.626013 2.449490 16.06238 10.488088 1.754993 1.793042
Hornet Sportabout 4.324350 2.828427 18.97367 13.228757 1.774824 1.854724
Valiant           4.254409 2.449490 15.00000 10.246951 1.661325 1.860108
                      qsec vs am     gear     carb
Mazda RX4         4.057093  0  1 2.000000 2.000000
Mazda RX4 Wag     4.125530  0  1 2.000000 2.000000
Datsun 710        4.313931  1  1 2.000000 1.000000
Hornet 4 Drive    4.409082  1  0 1.732051 1.000000
Hornet Sportabout 4.125530  0  0 1.732051 1.414214
Valiant           4.496665  1  0 1.732051 1.000000
```

En la línea de código anterior estamos pidiendo mostrar la cabecera (6 primeras observaciones de la tabla de datos) de la raíz cuadrada de los valores de la tabla `mtcars`, en formato del lenguaje clásico (anidado).

Ahora activamos `magrittr` y ejecutamos la línea anterior en formato tubería:

```
# Activa paquete
library(magrittr)

# Formato tubería
mtcars %>%
  sqrt() %>%
  head()
```

```
                      mpg      cyl     disp        hp     drat       wt
Mazda RX4         4.582576 2.449490 12.64911 10.488088 1.974842 1.618641
Mazda RX4 Wag     4.582576 2.449490 12.64911 10.488088 1.974842 1.695582
Datsun 710        4.774935 2.000000 10.39230  9.643651 1.962142 1.523155
Hornet 4 Drive    4.626013 2.449490 16.06238 10.488088 1.754993 1.793042
Hornet Sportabout 4.324350 2.828427 18.97367 13.228757 1.774824 1.854724
Valiant           4.254409 2.449490 15.00000 10.246951 1.661325 1.860108
                      qsec vs am     gear     carb
Mazda RX4         4.057093  0  1 2.000000 2.000000
Mazda RX4 Wag     4.125530  0  1 2.000000 2.000000
```

```
Datsun 710         4.313931  1  1 2.000000 1.000000
Hornet 4 Drive     4.409082  1  0 1.732051 1.000000
Hornet Sportabout 4.125530  0  0 1.732051 1.414214
Valiant            4.496665  1  0 1.732051 1.000000
```

Podemos hacer lo mismo con la tubería nativa de R sin activar ningún paquete (revisar que esté activada desde `Tools > Global Options`):

```r
# Tubería nativa
mtcars |>
  sqrt() |>
  head()
```

```
                      mpg      cyl     disp        hp     drat       wt
Mazda RX4         4.582576 2.449490 12.64911 10.488088 1.974842 1.618641
Mazda RX4 Wag     4.582576 2.449490 12.64911 10.488088 1.974842 1.695582
Datsun 710        4.774935 2.000000 10.39230  9.643651 1.962142 1.523155
Hornet 4 Drive    4.626013 2.449490 16.06238 10.488088 1.754993 1.793042
Hornet Sportabout 4.324350 2.828427 18.97367 13.228757 1.774824 1.854724
Valiant           4.254409 2.449490 15.00000 10.246951 1.661325 1.860108
                      qsec vs am     gear     carb
Mazda RX4         4.057093  0  1 2.000000 2.000000
Mazda RX4 Wag     4.125530  0  1 2.000000 2.000000
Datsun 710        4.313931  1  1 2.000000 1.000000
Hornet 4 Drive    4.409082  1  0 1.732051 1.000000
Hornet Sportabout 4.125530  0  0 1.732051 1.414214
Valiant           4.496665  1  0 1.732051 1.000000
```

Las tuberías le dan mucha mas claridad al código separándolo en partes, como si fuesen oraciones de un párrafo.



3.4. Archivos de texto plano con `readr`

El paquete `readr` (Wickham, Hester, et al. 2026) contiene funciones similares a las de la familia `read.table()` de R base, pero desarrollados bajo el ecosistema `tidyverse`.

Los archivos de texto plano (ASCII u otras codificaciones) son universalmente utilizados por la mayoría de los gestores de bases de datos y planillas de cálculo. Generalmente se encuentran con extensiones `.txt` o `.csv` (por *comma-separated values*) y son el tipo de archivo de datos más habitual en R.

Estos datos planos tienen dos características principales:

- La cabecera (en inglés *header*).
- El carácter o símbolo separador que indica la separación de columnas: pueden estar separadas por comas, punto y coma, tabulación, etc.

La presencia o no de una cabecera se maneja con los argumentos `col_names` y `skip`:

- `col_names = TRUE` indica que la primera fila contiene los nombres de las columnas (cabecera).
- `col_names = FALSE` indica que no hay cabecera y las columnas se nombran automáticamente (`X1`, `X2`, etc).
- `skip = 0` (valor por defecto) lee los datos desde la primera fila, pero si hay encabezados complejos (por ejemplo, títulos y subtítulos), se puede indicar cuántas filas deben omitirse. Ejemplo: `skip = 5` omite las primeras 5 filas del archivo.

Otro aspecto a considerar es el carácter **separador** utilizado para indicar la separación entre columnas. Los separadores más comunes son:

- coma (`,`)
- punto y coma (`;`)
- tabulación (`TAB`)
- espacio (`" "`)
- barra vertical o *pipe* (`|`)

Funciones de lectura

Algunas de las funciones del paquete asumen un separador particular. Por caso `read_csv()` lee separados por coma y `read_tsv()` separado por tabulaciones, pero la función `read_delim()` permite que definamos el separador a través del argumento `delim`.

En forma detallada el paquete `readr` soporta siete formatos de archivo a partir de siete funciones:

- `read_csv()`: archivos separados por comas (CSV).
- `read_tsv()`: archivos separados por tabulaciones.
- `read_delim()`: archivos separados con delimitadores generales.
- `read_fwf()`: archivos con columnas de ancho fijo.
- `read_table()`: archivos formato tabla con columnas separadas por espacios.
- `read_log()`: archivos log web.

En comparación con las funciones R `base`, las funciones de `readr`:

- Usan un esquema de nombres consistente de parámetros.
- Son más rápidas.
- Analizan eficientemente los formatos de datos comunes (especialmente fechas y horas).
- Muestra una barra de progreso para archivos grandes.
- Vienen incluidas dentro de `tidyverse` pero también pueden usarse de forma independiente:

```
library(readr)
```

A modo de ejemplo, leeremos un archivo sin cabecera separado por comas bajo el nombre `datos`:

```
datos <- read_csv("datos/ejemplo-datos.csv", col_names = F)
datos
```

```
# A tibble: 4 x 5
     X1 X2       X3       X4    X5
  <dbl> <chr>    <chr>    <chr> <date>
1     9 Leone    Fernando M     1958-12-24
2    26 Garcia   Esteban  M     1954-01-21
3    35 Salamone Nicolas  M     1993-06-27
4    48 Gonzalez Viviana  F     1965-06-21
```

Leemos el mismo archivo con cabecera y separado por punto y comas, bajo el nombre `info`:

```r
info <- read_csv2("datos/ejemplo-datos-header.csv", col_names = T)
info
```

```
# A tibble: 4 x 5
   Iden Apellido Nombre   Sexo  FNac
  <dbl> <chr>    <chr>    <chr> <date>
1     9 Leone    Fernando M     1958-12-24
2    26 Garcia   Laura    M     1954-01-21
3    35 Salamone Nicolas  M     1993-06-27
4    48 Gonzalez Viviana  F     1965-06-21
```

En estos ejemplos:

- `read_csv()` espera comas como separador
- `read_csv2()` espera punto y coma como separador

Al leer un archivo, `readr` intenta adivinar automáticamente el tipo de dato de cada columna (*parse*). Si no hay cabecera, los nombres de columna serán `X1`, `X2`, etc.

Podemos inspeccionar la estructura de un *dataframe* con `glimpse()`:

```r
glimpse(info)
```

```
Rows: 4
Columns: 5
$ Iden     <dbl> 9, 26, 35, 48
$ Apellido <chr> "Leone", "Garcia", "Salamone", "Gonzalez"
$ Nombre   <chr> "Fernando", "Laura", "Nicolas", "Viviana"
$ Sexo     <chr> "M", "M", "M", "F"
$ FNac     <date> 1958-12-24, 1954-01-21, 1993-06-27, 1965-06-21
```

Los tipos de datos posibles son:

- `character` (`<chr>`)
- `integer`, `double` o `numeric` (`<int>`, `<dbl>`)
- `logical` (`<lgl>`)
- `date`, `datetime`, etc.

Por ejemplo, columnas con enteros pueden aparecer como <dbl> si se interpretan como `double`, y las fechas como <date>.

Agregamos unos argumentos más y ejemplificamos la sintaxis con `read_delim()` para leer un archivo con cabecera compleja (la tabla comienza en la fila 9) separado por caracteres | (*pipes*).

```
read_delim(
  "ejemplo-datos-header-skip.txt",
  col_names = T,
  skip = 8,
  delim = "|"
)
```

> **Importante:** No olvides asignar la lectura a un nombre para guardar el *dataframe* dentro del entorno de trabajo (por ejemplo: `datos <-`).

Funciones de escritura

El paquete también incluye funciones para escribir archivos de texto plano, con formatos espejo de las funciones de lectura más comunes:

- `write_csv()`: escribe archivos separados por comas
- `write_csv2()`: escribe archivos separados por punto y comas
- `write_tsv()`: escribe archivos separados por tabulaciones
- `write_delim()`: escribe archivos separados con delimitadores definidos por el usuario

Los argumentos son generales y para el caso del último más extensos, dado que hay que definir cual es el separador que deseamos en el archivo. Podemos consultarlos con el siguiente código:

```
args(write_delim)
```

```
function (x, file, delim = " ", na = "NA", append = FALSE, col_names = !append,
    quote = c("needed", "all", "none"), escape = c("double",
        "backslash", "none"), eol = "\n", num_threads = readr_threads(),
    progress = show_progress())
NULL
```

Por ejemplo para exportar un conjunto de datos en texto plano al que denominaremos "`ejemplo.csv`" con separador **punto y coma** y **cabecera incluida** podemos hacer:

```
write_delim(x = datos, file = "ejemplo.csv", delim = ";")
```

o más sencillo, usando la función específica `write_csv2()`:

```
write_csv2(datos, "ejemplo.csv") # define cabecera y separador ;
```



3.5. Lectura de hojas de cálculo con `readxl`

Uno de los formatos más comunes para almacenar datos son las hojas de cálculo, en particular las creadas con Microsoft Excel. El paquete `readxl` (Wickham y Bryan 2026), parte del ecosistema `tidyverse`, permite leer este tipo de archivos.

`readxl` es compatible con hojas de cálculo de Excel 97-2003, con extensión `.xls`, y con versiones más recientes, con extensión `.xlsx`.

Una primera función útil es `excel_sheets()`, que permite conocer y listar los nombres de las hojas contenidas en un archivo Excel (también llamado libro o *workbook*).

Por ejemplo, supongamos que tenemos un archivo denominado "`datos.xlsx`" y queremos saber por cuantas hojas está compuesto y que nombre tienen:

```
library(readxl) # hay que activarlo independientemente de tidyverse

excel_sheets("datos/datos.xlsx")
```

```
[1] "diabetes"   "vigilancia" "mortalidad"
```

Esto devuelve, por ejemplo, tres hojas: "`diabetes`", "`vigilancia`" y "`mortalidad`".

Para leer una de estas hojas utilizamos la función `read_excel()`, cuyos argumentos principales son:

```
args(read_excel)
```

```
function (path, sheet = NULL, range = NULL, col_names = TRUE,
    col_types = NULL, na = "", trim_ws = TRUE, skip = 0, n_max = Inf,
    guess_max = min(1000, n_max), progress = readxl_progress(),
    .name_repair = "unique")
NULL
```

Entre los más relevantes encontramos:

- `path`: nombre del archivo y su ubicación (entre comillas)
- `sheet`: nombre de la hoja o su número de orden
- `col_names`: si es `TRUE`, toma la primera fila como nombres de las columnas
- `skip`: permite saltear un número determinado de filas antes de comenzar la lectura

Al ejecutar `read_excel()`, internamente se utiliza la función `excel_format()` para detectar si el archivo es `.xls` o `.xlsx`, y luego se aplica la función específica para cada caso: `read_xls()` o `read_xlsx()`. Estas funciones también pueden usarse directamente si se desea.

Supongamos ahora que queremos leer la hoja llamada `"diabetes"`:

```
diabetes <- read_excel(
  path = "datos/datos.xlsx",
  sheet = "diabetes",
  col_names = T
)

# mostramos las 6 primeras observaciones
head(diabetes)
```

```
# A tibble: 6 x 8
    A1C  hba1 GLUCB   SOG Tol_Glucosa    DM    SM  HOMA
  <dbl> <dbl> <dbl> <dbl> <chr>       <dbl> <dbl> <dbl>
1  6.17   7.9   101   122 IFG             0     1  4.04
2  5.58   7.2   103   100 IFG             0     0  5.03
3  5.38   7.1   103    90 IFG             0     1  2.92
4  5.38   6.6   109    96 IFG             0     1  4.79
5  5.19   6.3   107    69 IFG             0     1  3.06
6  4.89   6      NA   117 IFG             0     0  5.77
```

Observemos que en los argumentos escribimos el nombre del archivo que se encuentra en nuestro proyecto y por lo tanto en nuestra carpeta activa, el nombre de la hoja y nos aseguramos que la primer fila representa a la cabecera de la tabla (sus nombres de variables).

Como `readxl` forma parte del ecosistema `tidyverse` el formato de salida es un *tibble*. En este caso de 23 observaciones por 8 variables.

Ahora leamos la segunda hoja de nombre `"vigilancia"`:

```
vigilancia <- read_excel(path = "datos/datos.xlsx", sheet = 2, col_names = F)

# mostramos las 6 primeras observaciones
head(vigilancia)
```

```
# A tibble: 6 x 9
   ...1 ...2        ...3      ...4  ...5  ...6  ...7 ...8  ...9
  <dbl> <chr>      <dbl>     <dbl> <dbl> <dbl> <dbl> <chr> <chr>
1   875 09/28/2015  2015 544080000     1    31     1 F     VIGILANCIA EN SALUD ~
2   875 42317       2015 544080000     1    35     1 F     VIGILANCIA EN SALUD ~
3   875 42317       2015 544080000     1    47     1 F     VIGILANCIA EN SALUD ~
4   307 09/26/2015  2015 544005273     1    23     1 M     VIGILANCIA INTEGRADA~
5   307 09/24/2015  2015 544005273     1    19     1 M     VIGILANCIA INTEGRADA~
6   875 09/28/2015  2015 544080000     1    63     1 F     VIGILANCIA EN SALUD ~
```

En este caso, en lugar del nombre de la hoja usamos un 2 que es su ubicación y especificamos `col_names = FALSE` porque el conjunto de datos no tiene cabecera. `readxl` asignará nombres genéricos como `...1`, `...2`, etc.

Finalmente leamos la última hoja disponible del archivo:

```
mortalidad <- read_excel(
  path = "datos/datos.xlsx",
  sheet = "mortalidad",
  col_names = T,
  skip = 1
)

# mostramos las 6 primeras observaciones
head(mortalidad)
```

```
# A tibble: 5 x 10
  grupo_edad grupo.I.1.1 grupo.II.1.1 grupo.III.1.1 grupo.I.2.1 grupo.II.2.1
  <chr>            <dbl>        <dbl>         <dbl>       <dbl>        <dbl>
```

```
1 30-44                41          202          222          539          1438
2 45-59                99         1071          181          759          6210
3 60-69               114         1782          119          985          9238
4 70-79               221         2336          119         1571         12369
5 80+                 362         2492           81         2523         14642
# i 4 more variables: grupo.III.2.1 <dbl>, grupo.I.3.1 <dbl>,
#   grupo.II.3.1 <dbl>, grupo.III.3.1 <dbl>
```

Lo novedoso de esta lectura es el argumento `skip = 1` que debimos incorporar dado que, en este caso, la hoja de Excel comienza con una línea de título que no pertenece al conjunto de datos. También que el argumento `sheet` permite el nombre de la hoja elegida entre comillas.

Además de los argumentos generales de `read_xl()`, podemos mencionar estos otros:

- `n_max`: número máximo de filas a leer.
- `range`: rango de celdas a importar (como en Excel, por ejemplo `"B3:D87"`).
- `col_types`: define el tipo de datos de cada columna. Valores posibles: `"numeric"`, `"logical"`, `"text"`, `"date"`, `"skip"` (no leer la columna), `"guess"` (modo predeterminado: la función decide automáticamente el tipo).
- `na`: carácter o vector de caracteres que se deben interpretar como valores perdidos (`NA`). Por defecto, las celdas vacías se interpretan así.

⚠ En los siguientes capítulos se abordarán dos componentes fundamentales del ecosistema `tidyverse`: `dplyr`, para la manipulación de datos y la construcción de tablas de resumen, y `ggplot2`, para la elaboración de gráficos estadísticos.

3.6. Referencias

4 Gestión de datos con `dplyr`



4.1. Introducción

El paquete `dplyr` (Wickham, François, et al. 2026) fue desarrollado por *Hadley Wickham* como una versión optimizada del paquete `plyr` (Wickham 2011).

Su principal contribución es ofrecer una *gramática* para la manipulación de datos, basada en funciones que actúan como verbos, lo que facilita la lectura y comprensión del código.

Las funciones clave del paquete permiten realizar las siguientes acciones (verbos):

- `select()`: selecciona un conjunto de columnas (variables)
- `rename()`: renombra variables en un conjunto de datos
- `filter()`: selecciona un conjunto de filas (observaciones) según una o varias condiciones lógicas
- `arrange()`: reordena las filas de un conjunto de datos
- `mutate()`: añade nuevas variables/columnas o transforma variables existentes
- `summarise()`/ `summarize()`: genera resúmenes estadísticos de diferentes variables en el conjunto de datos
- `group_by()`: agrupa las observaciones en función de una o más variables, lo que permite realizar operaciones por grupo
- `count()`: contabiliza valores que se repiten, generando una tabla de frecuencias

Además, al ser parte del ecosistema *tidyverse*, `dplyr` integra al operador *pipe* (%>%) formando una única secuencia de procesamiento o *pipeline*.

4.1.1. Argumentos comunes

Todas las funciones, básicamente, tienen en común una serie de argumentos.

1. El primer argumento es el nombre del conjunto de datos (objeto donde esta nuestra tabla de datos).
2. Los otros argumentos describen que hacer con el conjunto de datos especificado en el primer argumento, podemos referirnos a las columnas en el objeto directamente sin utilizar el operador $, es decir sólo con el nombre de la columna/variable.
3. El valor de retorno es un nuevo conjunto de datos.
4. Los conjuntos de datos deben estar bien organizados/estructurados, es decir debe existir una observación por columna y, cada columna representar una variable, medida o característica de esa observación. Es decir, debe cumplir con *tidy data*.

4.1.2. Activación del paquete

`dplyr` está incluído en el núcleo base de `tidyverse`, por lo que se encuentra disponible si tenemos activado a este último.

También se puede activar en forma independiente:

```
library(dplyr)
```

4.1.3. Conjunto de datos para ejemplo

Para visualizar y comprender el funcionamiento de estos "verbos" de manipulación, resulta muy útil contar con ejemplos concretos. Por eso, en esta unidad trabajaremos con un conjunto de datos que nos permitirá practicar el uso de las funciones del paquete.

> Recuerden que pueden **descargar los datos** utilizados en los ejemplos del curso y descomprimirlos en la carpeta donde tengan guardado su proyecto de RStudio.

Uno de los archivos incluidos, "`noti-vih.csv`", contiene registros de notificaciones de VIH por jurisdicción en Argentina correspondientes a los años 2015 y 2016.

```
# asignamos la lectura a datos
datos <- read_csv("datos/noti-vih.csv")

# mostramos las 6 primeras observaciones
head(datos)
```

```
# A tibble: 6 x 4
  jurisdiccion   año casos      pob
  <chr>        <dbl> <dbl>    <dbl>
1 Buenos Aires  2015  1513 16626374
2 Buenos Aires  2016   957 16789474
3 CABA          2015   901  3054237
4 CABA          2016   427  3050000
5 Catamarca     2015    69   396552
6 Catamarca     2016    51   401575
```

4.2. Función `select()`

La función `select()` permite elegir columnas específicas de un conjunto de datos, devolviendo una versión "recortada por columnas" del mismo.

A continuación, exploramos algunas formas útiles de seleccionar variables:

Seleccionar todas las variables excepto `pob`:

```
datos |>
  select(-pob)
```

```
# A tibble: 48 x 3
   jurisdiccion   año casos
   <chr>        <dbl> <dbl>
 1 Buenos Aires  2015  1513
 2 Buenos Aires  2016   957
 3 CABA          2015   901
 4 CABA          2016   427
 5 Catamarca     2015    69
 6 Catamarca     2016    51
 7 Chaco         2015    15
 8 Chaco         2016     9
 9 Chubut        2015   110
10 Chubut        2016    89
# i 38 more rows
```

Otra forma para el mismo resultado anterior (mediante el operador rango `:`):

```
datos |>
  select(jurisdiccion:casos)
```

```
# A tibble: 48 x 3
   jurisdiccion   año casos
   <chr>        <dbl> <dbl>
 1 Buenos Aires  2015  1513
 2 Buenos Aires  2016   957
 3 CABA          2015   901
 4 CABA          2016   427
 5 Catamarca     2015    69
 6 Catamarca     2016    51
 7 Chaco         2015    15
 8 Chaco         2016     9
 9 Chubut        2015   110
10 Chubut        2016    89
# i 38 more rows
```

Seleccionar solamente las variables `jurisdiccion` y `casos`:

```
datos |>
  select(jurisdiccion, casos)
```

```
# A tibble: 48 x 2
   jurisdiccion casos
   <chr>        <dbl>
 1 Buenos Aires  1513
 2 Buenos Aires   957
 3 CABA           901
 4 CABA           427
 5 Catamarca       69
 6 Catamarca       51
 7 Chaco           15
 8 Chaco            9
 9 Chubut         110
10 Chubut          89
# i 38 more rows
```

Lo mismo que el ejemplo anterior, pero usando la posición de las columnas:

```
datos |>
  select(1, 3)
```

```
# A tibble: 48 x 2
   jurisdiccion casos
   <chr>        <dbl>
 1 Buenos Aires  1513
 2 Buenos Aires   957
 3 CABA           901
 4 CABA           427
 5 Catamarca       69
 6 Catamarca       51
 7 Chaco           15
 8 Chaco            9
 9 Chubut         110
10 Chubut          89
# i 38 more rows
```

Mover la variable año al inicio y mantener todas las demás:

```
datos |>
  select("año", everything())
```

```
# A tibble: 48 x 4
     año jurisdiccion casos      pob
   <dbl> <chr>        <dbl>    <dbl>
 1  2015 Buenos Aires  1513 16626374
 2  2016 Buenos Aires   957 16789474
 3  2015 CABA           901  3054237
 4  2016 CABA           427  3050000
 5  2015 Catamarca       69   396552
 6  2016 Catamarca       51   401575
 7  2015 Chaco           15  1153846
 8  2016 Chaco            9  1125000
 9  2015 Chubut         110   567010
10  2016 Chubut          89   577922
# i 38 more rows
```

Cambiar el nombre de una de la variable año y mover al inicio:

```
datos |>
  select(anio = año, everything())
```

```
# A tibble: 48 x 4
    anio jurisdiccion casos      pob
   <dbl> <chr>        <dbl>    <dbl>
 1  2015 Buenos Aires  1513 16626374
 2  2016 Buenos Aires   957 16789474
 3  2015 CABA           901  3054237
 4  2016 CABA           427  3050000
 5  2015 Catamarca       69   396552
 6  2016 Catamarca       51   401575
 7  2015 Chaco           15  1153846
 8  2016 Chaco            9  1125000
 9  2015 Chubut         110   567010
10  2016 Chubut          89   577922
# i 38 more rows
```

Otros posibles argumentos son:

- `starts_with()`: selecciona todas las columnas que comiencen con el patrón indicado.
- `ends_with()`: selecciona todas las columnas que terminen con el patrón indicado.
- `contains()`: selecciona las columnas que posean el patrón indicado.
- `matches()`: similar a `contains()`, pero permite poner una expresión regular.
- `all_of()`: selecciona las variables pasadas en un vector (todos los nombres deben estar presentes o devuelve un error).
- `any_of()`: idem anterior excepto que no se genera ningún error para los nombres que no existen.
- `num_range()`: selecciona variables con nombre combinados con caracteres y números (ejemplo: `num_range("x", 1:3)` selecciona las variables `x1`, `x2` y `x3`.
- `where()`: aplica una función a todas las variables y selecciona aquellas para las cuales la función regresa TRUE (por ejemplo: `is.numeric()` para seleccionar todas las variables numéricas).

4.3. Función `rename()`

La función `rename()` puede considerarse una extensión de `select()`. Si bien `select()` también permite renombrar variables, no resulta muy útil para este fin, ya que descarta todas las variables que no se mencionan explícitamente.

En cambio, `rename()` permite cambiar el nombre de una o más variables sin eliminar las demás. Solo se modifican los nombres indicados, y el resto del conjunto de datos permanece sin cambios.

Ejemplo: renombrar la variable `pob` como `población`:

```
datos |>
  rename("población" = pob)
```

```
# A tibble: 48 x 4
   jurisdiccion   año casos población
   <chr>        <dbl> <dbl>     <dbl>
 1 Buenos Aires  2015  1513  16626374
 2 Buenos Aires  2016   957  16789474
 3 CABA          2015   901   3054237
 4 CABA          2016   427   3050000
 5 Catamarca     2015    69    396552
 6 Catamarca     2016    51    401575
 7 Chaco         2015    15   1153846
 8 Chaco         2016     9   1125000
 9 Chubut        2015   110    567010
10 Chubut        2016    89    577922
# i 38 more rows
```

Se puede renombrar una serie de columnas usando `rename_with()`, por ejemplo para agregar el prefijo `"n_"` a las variables de conteo:

```
datos |>
  rename_with(
    .cols = c(casos, pob), # Columnas a renombrar
    .fn = ~ paste0("n_", .x) # Función o vector para renombrar columnas
  )
```

```
# A tibble: 48 x 4
   jurisdiccion   año n_casos    n_pob
   <chr>        <dbl>   <dbl>    <dbl>
 1 Buenos Aires  2015    1513 16626374
 2 Buenos Aires  2016     957 16789474
 3 CABA          2015     901  3054237
 4 CABA          2016     427  3050000
 5 Catamarca     2015      69   396552
 6 Catamarca     2016      51   401575
 7 Chaco         2015      15  1153846
 8 Chaco         2016       9  1125000
 9 Chubut        2015     110   567010
10 Chubut        2016      89   577922
# i 38 more rows
```

4.4. Función `filter()`

La función `filter()` permite seleccionar filas de un conjunto de datos, produciendo un subconjunto de observaciones.

Veamos un ejemplo sencillo con nuestros datos:

```
datos |>
  filter(jurisdiccion == "Tucuman")
```

```
# A tibble: 2 x 4
  jurisdiccion   año casos     pob
  <chr>        <dbl> <dbl>   <dbl>
1 Tucuman       2015   258 1592593
2 Tucuman       2016   246 1618421
```

Utiliza los mismos operadores de comparación propios del lenguaje R:

Operador	Descripción
<	Menor que
>	Menor que
<=	Menor o igual que
>=	Mayor o igual que
==	Igual que
!=	No igual que
%in %	Es parte de
is.na()	Es un valor ausente
!is.na()	No es un valor ausente

Lo mismo con los operadores lógicos que se utilizan como conectores entre las expresiones:

Operador	Descripción
&	AND booleano
|	OR booleano
xor()	OR exclusivo
!	NOT
any()	cualquier TRUE
all()	todos TRUE

Cuando usamos múltiples argumentos separados por coma dentro de `filter()`, estas se combinan con un operador **AND** implícito, es decir, cada expresión debe ser verdadera para que la fila sea incluida en la salida.

Por ejemplo, filtramos las observaciones que cumplan que `casos` sea mayor a 100 **y** que `pob` sea menor a 1.000.000:

```
datos |>
  filter(casos > 100, pob < 1000000)
```

```
# A tibble: 7 x 4
  jurisdiccion   año casos    pob
  <chr>        <dbl> <dbl>  <dbl>
1 Chubut        2015   110 567010
2 Jujuy         2015   160 727273
3 Jujuy         2016   133 734807
4 Neuquen       2015   109 619318
5 Neuquen       2016   101 627329
6 Rio Negro     2015   112 700000
7 Rio Negro     2016   105 709459
```

Para combinar condiciones dentro de una misma variable usamos el operador OR (`|`) o, de forma más práctica, `%in%`:

```
# Con OR
datos |>
  filter(jurisdiccion == "Buenos Aires" | jurisdiccion == "La Pampa")
```

```
# A tibble: 4 x 4
  jurisdiccion   año casos      pob
  <chr>        <dbl> <dbl>    <dbl>
1 Buenos Aires  2015  1513 16626374
2 Buenos Aires  2016   957 16789474
3 La Pampa      2015    57   343373
4 La Pampa      2016    67   345361
```

```
# Con %in%
datos |>
  filter(jurisdiccion %in% c("Buenos Aires", "La Pampa"))
```

```
# A tibble: 4 x 4
  jurisdiccion   año casos      pob
  <chr>        <dbl> <dbl>    <dbl>
1 Buenos Aires  2015  1513 16626374
2 Buenos Aires  2016   957 16789474
3 La Pampa      2015    57   343373
4 La Pampa      2016    67   345361
```

En el siguiente ejemplo, filtramos observaciones del año 2016 con más de 200 casos. El uso de `&` es equivalente al uso de coma:

```
datos |>
  filter(año == "2016" & casos > 200)
```

```
# A tibble: 6 x 4
  jurisdiccion   año casos      pob
  <chr>        <dbl> <dbl>    <dbl>
1 Buenos Aires  2016   957 16789474
2 CABA          2016   427  3050000
3 Cordoba       2016   368  3607843
4 Mendoza       2016   254  1909774
5 Salta         2016   230  1352941
6 Tucuman       2016   246  1618421
```

Por último, podemos usar `xor()` para seleccionar observaciones que cumplan **solo una** de las condiciones, pero no ambas. Por ejemplo, el siguiente filtro selecciona registros donde el año sea 2016 ó los casos sean mayores a 200, pero no ambos al mismo tiempo (es decir que no se den ambos en `TRUE`):

```
datos |>
  filter(xor(año == "2016", casos > 200))
```

```
# A tibble: 25 x 4
   jurisdiccion   año casos      pob
   <chr>        <dbl> <dbl>    <dbl>
 1 Buenos Aires  2015  1513 16626374
 2 CABA          2015   901  3054237
 3 Catamarca     2016    51   401575
 4 Chaco         2016     9  1125000
 5 Chubut        2016    89   577922
 6 Cordoba       2015   468  3572519
 7 Corrientes    2016    99  1076087
```

```
 8 Entre Rios    2016   109  1329268
 9 Formosa       2016    60   582524
10 Jujuy         2016   133   734807
# i 15 more rows
```

En las últimas versiones de `dplyr` se añadió la función `filter_out()`, que equivale a usar el operador `!` para negar una condición. Por ejemplo, si quisiéramos excluir todos los datos de la provincia de Mendoza:

```
datos |>
  filter_out(jurisdiccion == "Mendoza")
```

```
# A tibble: 46 x 4
   jurisdiccion   año casos      pob
   <chr>        <dbl> <dbl>    <dbl>
 1 Buenos Aires  2015  1513 16626374
 2 Buenos Aires  2016   957 16789474
 3 CABA          2015   901  3054237
 4 CABA          2016   427  3050000
 5 Catamarca     2015    69   396552
 6 Catamarca     2016    51   401575
 7 Chaco         2015    15  1153846
 8 Chaco         2016     9  1125000
 9 Chubut        2015   110   567010
10 Chubut        2016    89   577922
# i 36 more rows
```

4.5. Función `arrange()`

La función `arrange()` se utiliza para ordenar las filas de un conjunto de datos de acuerdo a una o varias columnas/variables. Por defecto, el ordenamiento es ascendente alfanumérico.

Ordenamos la tabla por la variable `pob` (forma ascendente predeterminada):

```
datos |>
  arrange(pob)
```

```
# A tibble: 48 x 4
   jurisdiccion       año casos    pob
   <chr>            <dbl> <dbl>  <dbl>
```

```
 1 Tierra del Fuego  2015    36 152542
 2 Tierra del Fuego  2016    34 156682
 3 Santa Cruz        2015    65 320197
 4 Santa Cruz        2016    59 329609
 5 La Pampa          2015    57 343373
 6 La Pampa          2016    67 345361
 7 La Rioja          2015    41 369369
 8 La Rioja          2016     6 375000
 9 Catamarca         2015    69 396552
10 Catamarca         2016    51 401575
# i 38 more rows
```

Para ordenar en forma descendente podemos utilizar `desc()` dentro de los argumentos de `arrange()`:

```
datos |>
  arrange(desc(pob))
```

```
# A tibble: 48 x 4
   jurisdiccion   año casos      pob
   <chr>        <dbl> <dbl>    <dbl>
 1 Buenos Aires  2016   957 16789474
 2 Buenos Aires  2015  1513 16626374
 3 Cordoba       2016   368  3607843
 4 Cordoba       2015   468  3572519
 5 Santa Fe      2016   170  3400000
 6 Santa Fe      2015   301  3382022
 7 CABA          2015   901  3054237
 8 CABA          2016   427  3050000
 9 Mendoza       2016   254  1909774
10 Mendoza       2015   316  1880952
# i 38 more rows
```

Que equivale a:

```
datos |>
  arrange(-pob)
```

```
# A tibble: 48 x 4
   jurisdiccion   año casos      pob
   <chr>        <dbl> <dbl>    <dbl>
```

```
 1 Buenos Aires  2016   957 16789474
 2 Buenos Aires  2015  1513 16626374
 3 Cordoba       2016   368  3607843
 4 Cordoba       2015   468  3572519
 5 Santa Fe      2016   170  3400000
 6 Santa Fe      2015   301  3382022
 7 CABA          2015   901  3054237
 8 CABA          2016   427  3050000
 9 Mendoza       2016   254  1909774
10 Mendoza       2015   316  1880952
# i 38 more rows
```

Podemos combinar ordenamientos. Por ejemplo, en forma alfabética ascendente para `jusrisdiccion` y luego numérica descendente para `casos`:

```
datos |>
  arrange(jurisdiccion, desc(casos))
```

```
# A tibble: 48 x 4
   jurisdiccion   año casos      pob
   <chr>        <dbl> <dbl>    <dbl>
 1 Buenos Aires  2015  1513 16626374
 2 Buenos Aires  2016   957 16789474
 3 CABA          2015   901  3054237
 4 CABA          2016   427  3050000
 5 Catamarca     2015    69   396552
 6 Catamarca     2016    51   401575
 7 Chaco         2015    15  1153846
 8 Chaco         2016     9  1125000
 9 Chubut        2015   110   567010
10 Chubut        2016    89   577922
# i 38 more rows
```

4.6. Función `mutate()`

Esta función nos permite transformar variables dentro de un conjunto de datos. A menudo tendremos la necesidad de modificar variables existentes o crear nuevas variables a partir de las ya disponibles. La función `mutate()` nos ofrece una forma clara y eficiente de realizar este tipo de operaciones.

Por ejemplo, podríamos querer calcular tasas crudas para cada jurisdicción y año, en función del número de casos y de la población total:

```
datos |>
  mutate(tasa = casos / pob * 100000)
```

```
# A tibble: 48 x 5
   jurisdiccion   año casos      pob  tasa
   <chr>        <dbl> <dbl>    <dbl> <dbl>
 1 Buenos Aires  2015  1513 16626374  9.10
 2 Buenos Aires  2016   957 16789474  5.70
 3 CABA          2015   901  3054237 29.5
 4 CABA          2016   427  3050000 14
 5 Catamarca     2015    69   396552 17.4
 6 Catamarca     2016    51   401575 12.7
 7 Chaco         2015    15  1153846  1.30
 8 Chaco         2016     9  1125000  0.8
 9 Chubut        2015   110   567010 19.4
10 Chubut        2016    89   577922 15.4
# i 38 more rows
```

En este caso, `mutate()` calcula la tasa cruda por 100.000 habitantes e incorpora una nueva variable (`tasa`) con los resultados correspondientes a cada observación.

También se pueden construir múltiples variables en la misma expresión, solamente separadas por comas:

```
datos |>
  mutate(
    tasaxcien_mil = casos / pob * 100000,
    tasaxdiez_mil = casos / pob * 10000
  )
```

```
# A tibble: 48 x 6
   jurisdiccion   año casos      pob tasaxcien_mil tasaxdiez_mil
   <chr>        <dbl> <dbl>    <dbl>         <dbl>         <dbl>
 1 Buenos Aires  2015  1513 16626374          9.10         0.910
 2 Buenos Aires  2016   957 16789474          5.70         0.570
 3 CABA          2015   901  3054237         29.5          2.95
 4 CABA          2016   427  3050000         14            1.4
 5 Catamarca     2015    69   396552         17.4          1.74
 6 Catamarca     2016    51   401575         12.7          1.27
 7 Chaco         2015    15  1153846          1.30         0.130
 8 Chaco         2016     9  1125000          0.8          0.08
```

```
 9 Chubut        2015   110   567010         19.4          1.94
10 Chubut        2016    89   577922         15.4          1.54
# i 38 more rows
```

Si deseamos que estas nuevas variables se incorporen de forma permanente al conjunto de datos (y no solo se muestren en la consola), debemos utilizar el operador de asignación `<-`:

```
datos <- datos |>
  mutate(
    tasaxcien_mil = casos / pob * 100000,
    tasaxdiez_mil = casos / pob * 10000
  )
```

Un aspecto fundamental es que las funciones utilizadas dentro de `mutate()` deben estar *vectorizadas*: deben aceptar un vector de entrada y devolver otro vector del mismo tamaño como salida.

Existen muchas funciones que se pueden utilizar dentro de `mutate()`. A continuación se presentan algunas útiles:

- **Operadores aritméticos**: `+`, `-`, `*`, `/`, `^`.
- **Aritmética modular**: `%/%` (división entera) y `%%` (resto), donde `x == y * (x %/% y) + (x %% y)`.
- **Funciones matemáticas**: `log()`, `log2()`, `log10()`, `exp()`, `sqrt()`, `abs()`, entre otras.
- **Valores acumulados**: `cumsum()`, `cumprod()`, `cummin()`, `cummax()` y `cummean()`.
- **Clasificación o *ranking***: funciones como `min_rank()` permiten asignar rangos (1º, 2º, etc.). Por defecto, los valores más pequeños reciben rangos más bajos. Si se desea invertir el orden, puede utilizarse `desc(x)`.

En versiones recientes de `dplyr`, `mutate()` incorpora argumentos que facilitan el trabajo con datos:

- `.by`: permite aplicar transformaciones por grupos de manera **temporal**, sin necesidad de usar `group_by()`
- `.before` / `.after`: permiten ubicar la nueva variable antes o después de otra columna del conjunto de datos

Finalmente, si se utiliza en `mutate()` el mismo nombre de una variable que ya existe en la tabla, dicha variable será sobrescrita (por ejemplo, al cambiarle el tipo de `character` a `factor`). Si se desea crear una variable nueva, se debe utilizar un nombre que no esté previamente en el conjunto de datos.

4.7. Funciones `group_by()` y `summarise()`

Estas dos funciones suelen utilizan en conjunto para calcular **resúmenes estadísticos por grupos**. En el caso de `summarise()` / `summarize()`, podemos reducir un conjunto de datos a uno o más valores resumen.

Por ejemplo:

```
datos |>
  summarise(
    promedio_casos = mean(casos),
    casos_totales = sum(casos)
  )
```

```
# A tibble: 1 x 2
  promedio_casos casos_totales
           <dbl>         <dbl>
1           192.          9211
```

Así, se obtiene una tabla con una sola fila que contiene el promedio y el total de casos en todo el conjunto de datos.

La función `group_by()` permite definir grupos dentro del conjunto de datos según una o más variables. A partir de ese momento, las operaciones siguientes se aplican **de forma independiente dentro de cada grupo**.

Por ejemplo, si queremos calcular los mismos indicadores por año:

```
datos |>
  group_by(año) |>
  summarise(
    promedio_casos = mean(casos),
    casos_totales = sum(casos)
  )
```

```
# A tibble: 2 x 3
    año promedio_casos casos_totales
  <dbl>          <dbl>         <dbl>
1  2015           224.          5369
2  2016           160.          3842
```

El resultado es una tabla con una fila por cada año, mostrando los valores correspondientes a cada grupo.

También es posible agrupar por más de una variable. Por ejemplo, calcular una tasa por jurisdicción y año:

```
datos |>
  group_by(jurisdiccion, año) |>
  summarise(
    tasa = casos / pob * 100000
  )
```

```
# A tibble: 48 x 3
# Groups:   jurisdiccion [24]
   jurisdiccion   año  tasa
   <chr>        <dbl> <dbl>
 1 Buenos Aires  2015  9.10
 2 Buenos Aires  2016  5.70
 3 CABA          2015 29.5
 4 CABA          2016 14
 5 Catamarca     2015 17.4
 6 Catamarca     2016 12.7
 7 Chaco         2015  1.30
 8 Chaco         2016  0.8
 9 Chubut        2015 19.4
10 Chubut        2016 15.4
# i 38 more rows
```

Después de usar `group_by()`, los datos permanecen agrupados. Si se desea continuar trabajando sin esa estructura, se puede utilizar la función `ungroup()` o el argumento `.groups = "drop"` dentro de `summarise()`:

```
# Descartar grupos con ungroup()
datos |>
  group_by(jurisdiccion, año) |>
  summarise(
    tasa = casos / pob * 100000
  ) |>
  ungroup()
```

```
# A tibble: 48 x 3
   jurisdiccion   año  tasa
```

```
   <chr>        <dbl> <dbl>
 1 Buenos Aires  2015  9.10
 2 Buenos Aires  2016  5.70
 3 CABA          2015 29.5
 4 CABA          2016 14
 5 Catamarca     2015 17.4
 6 Catamarca     2016 12.7
 7 Chaco         2015  1.30
 8 Chaco         2016  0.8
 9 Chubut        2015 19.4
10 Chubut        2016 15.4
# i 38 more rows
```

```r
# Descartar grupos dentro de summarise()
datos |>
  group_by(jurisdiccion, año) |>
  summarise(
    tasa = casos / pob * 100000,
    .groups = "drop"
  )
```

```
# A tibble: 48 x 3
   jurisdiccion   año  tasa
   <chr>        <dbl> <dbl>
 1 Buenos Aires  2015  9.10
 2 Buenos Aires  2016  5.70
 3 CABA          2015 29.5
 4 CABA          2016 14
 5 Catamarca     2015 17.4
 6 Catamarca     2016 12.7
 7 Chaco         2015  1.30
 8 Chaco         2016  0.8
 9 Chubut        2015 19.4
10 Chubut        2016 15.4
# i 38 more rows
```

4.7.1. Funciones de resumen más frecuentes

Algunas de las funciones del R `base` que se pueden utilizar dentro de los argumentos de esta función son:

- `min()`: mínimo

- `max()`: máximo
- `mean()`: media
- `median()`: mediana
- `var()`: varianza
- `sd()`: desvío
- `sum()`: sumatoria

Otras funciones que se pueden incorporar las provee el mismo paquete `dplyr`, por ejemplo:

- `first()`: primer valor en el vector.
- `last()`: último valor en el vector.
- `n()`: número de valores en el vector.
- `n_distinct()`: números de valores distintos en el vector.

4.7.2. Combinaciones

En los ejemplos anteriores vimos cómo se van integrando algunas de las funciones mediante el uso del operador de tubería %>% o |>. La idea detrás de esta "*gramática de los datos*" que propone el paquete `dplyr` es poder encadenar acciones de forma legible y lógica, construyendo oraciones más complejas paso a paso.

Veamos un ejemplo que integra muchas de las funciones vistas hasta ahora:

> Obtener una nueva tabla con las tasas crudas de casos notificados de VIH, por año y jurisdicción, **mayores a 20 por 100.000 habitantes**, ordenadas de **mayor a menor**.

```
datos |> # siempre partimos de los datos
  group_by(año, jurisdiccion) |> # agrupamos
  summarise(tasa = casos / pob * 100000) |> # resumimos
  filter(tasa > 20) |> # filtramos
  arrange(desc(tasa)) # ordenamos
```

```
# A tibble: 5 x 3
# Groups:   año [2]
    año jurisdiccion      tasa
  <dbl> <chr>            <dbl>
1  2015 CABA              29.5
2  2015 Tierra del Fuego  23.6
3  2015 Jujuy             22.0
4  2016 Tierra del Fuego  21.7
5  2015 Santa Cruz        20.3
```

Una buena práctica para construir este tipo de código es escribir cada paso de la operación **en una línea separada**. Esto no solo mejora la legibilidad, sino que facilita la identificación de errores o la modificación de pasos específicos.

Este ejemplo muestra claramente el poder y la claridad que se logran al combinar funciones como `group_by()`, `summarise()`, `filter()` y `arrange()` en una misma operación fluida y coherente.

4.8. Función `count()`

Esta función permite contar rápidamente los valores únicos de una o más variables en un conjunto de datos. Es especialmente útil para crear tablas de frecuencias absolutas, que posteriormente pueden ser usadas para calcular frecuencias relativas.

Tiene un par de argumentos opcionales:

- `name`: Define el nombre de la columna que contendrá el conteo. Por defecto, esta columna se llama `n`.
- `sort`: Ordena la tabla de frecuencias de mayor a menor (por defecto, no realiza ninguna ordenación).
- `wt`: Permite incorporar una variable que funcione como ponderación (o factor de expansión) para el cálculo de la frecuencia.

Un ejemplo básico de su uso es contar las observaciones por cada valor único de la variable `jurisdiccion` en el conjunto de datos:

```
datos |>
  count(jurisdiccion)
```

```
# A tibble: 24 x 2
   jurisdiccion     n
   <chr>        <int>
 1 Buenos Aires     2
 2 CABA             2
 3 Catamarca        2
 4 Chaco            2
 5 Chubut           2
 6 Cordoba          2
 7 Corrientes       2
 8 Entre Rios       2
 9 Formosa          2
10 Jujuy            2
# i 14 more rows
```

Usando el argumento `wt` obtenemos el conteo de casos por jurisdicción:

```
datos |>
  count(jurisdiccion, wt = casos)
```

```
# A tibble: 24 x 2
   jurisdiccion     n
   <chr>        <dbl>
 1 Buenos Aires  2470
 2 CABA          1328
 3 Catamarca      120
 4 Chaco           24
 5 Chubut         199
 6 Cordoba        836
 7 Corrientes     213
 8 Entre Rios     253
 9 Formosa        124
10 Jujuy          293
# i 14 more rows
```

Lo cual es equivalente a:

```
datos |>
  group_by(jurisdiccion) |>
  summarise(
    n = sum(casos, na.rm = TRUE),
    .groups = "drop"
  )
```

```
# A tibble: 24 x 2
   jurisdiccion     n
   <chr>        <dbl>
 1 Buenos Aires  2470
 2 CABA          1328
 3 Catamarca      120
 4 Chaco           24
 5 Chubut         199
 6 Cordoba        836
 7 Corrientes     213
 8 Entre Rios     253
 9 Formosa        124
10 Jujuy          293
# i 14 more rows
```

Referencias

5 Gráficos estadísticos con `ggplot2`

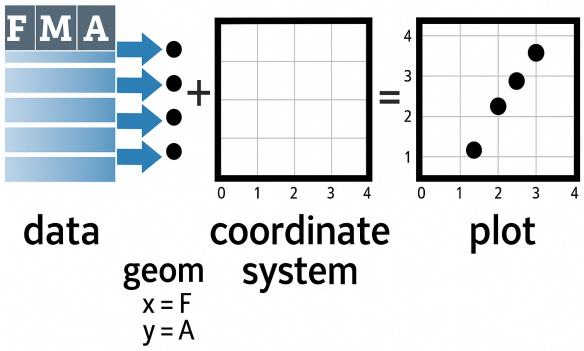


5.1. Introducción

El paquete `ggplot2` (Wickham 2016) es parte del ecosistema `tidyverse` y se autodefine como una librería para "*crear elegantes visualizaciones de datos utilizando una gramática de gráficos*". Proporciona una forma intuitiva de construir gráficos basada en **The Grammar of Graphics** a través de un sistema basado en tres componentes básicos:

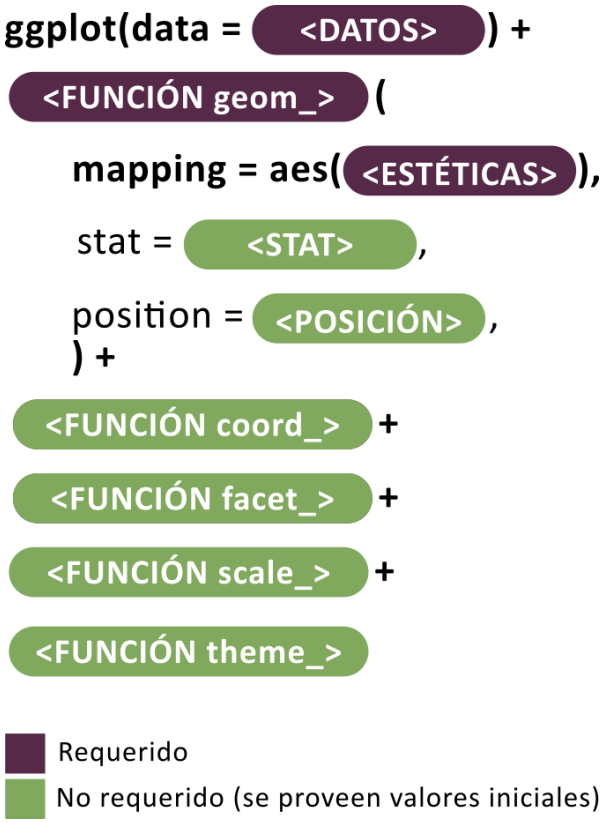
- datos
- coordenadas
- objetos geométricos

La estructura para construir un gráfico es la siguiente:



5.2. Anatomía de gráficos con `ggplot2`

La construcción de gráficos en `ggplot2` se organiza a partir de una gramática que puede entenderse a través de sus componentes fundamentales:



- `data`: el conjunto de datos que vamos a graficar, debe contener toda la información necesaria.
- `aes()`: el mapeo estético (*aesthetic mapping*) es donde se declaran las variables que se representarán en el gráfico (por ejemplo, los ejes X e Y o cómo se asignarán los colores).
- `geoms`: representaciones gráficas de los datos (puntos, líneas, barras, cajas, etc.). Son las "geometrías" que dibujan el gráfico.
- `stats`: transformaciones estadísticas sobre los datos (medias, regresiones, suavizados, etc.), que permiten resumir la información.
- `scales`: controlan cómo se representan los datos (ejes, colores, leyendas).
- `coordinate systems`: sistema de coordenadas utilizado para representar el gráfico en un plano bidimensional.
- `facets`:permiten dividir los datos en subconjuntos y generar gráficos en paneles (viñetas).
- `theme`: controla la apariencia general de los elementos no asociados a los datos (fondos, tipografías, bordes, etc.).

5.2.1. Ejemplo con datos

Antes de mostrar como usar estos componentes, cargaremos la base de datos "`facultad.csv`", que contiene datos ficticios sobre ingresantes a una facultad (por ejemplo, sexo, edad, talla y peso). Este conjunto se utilizará en los ejemplos:

```
# Cargar datos
facultad <- read_csv("datos/facultad.csv")

# Mostramos las 6 primeras observaciones
head(facultad)
```

```
# A tibble: 6 x 18
     hc sexo   edad ant_diabetes ant_tbc ant_cancer ant_obesidad ant_ecv ant_ht
  <dbl> <chr> <dbl> <chr>        <chr>   <chr>      <chr>        <chr>   <chr>
1 26880 M        17 NO           NO      NO         SI           NO      SI
2 26775 M        18 SI           NO      NO         NO           NO      NO
3 26877 M        18 SI           NO      SI         NO           NO      SI
4 26776 M        18 NO           NO      NO         SI           SI      NO
5 26718 M        18 NO           NO      NO         NO           NO      SI
6 26738 M        18 NO           NO      NO         NO           NO      SI
# i 9 more variables: ant_col <chr>, fuma <chr>, edadini <dbl>, cantidad <dbl>,
#   col <dbl>, peso <dbl>, talla <dbl>, sist <dbl>, diast <dbl>
```

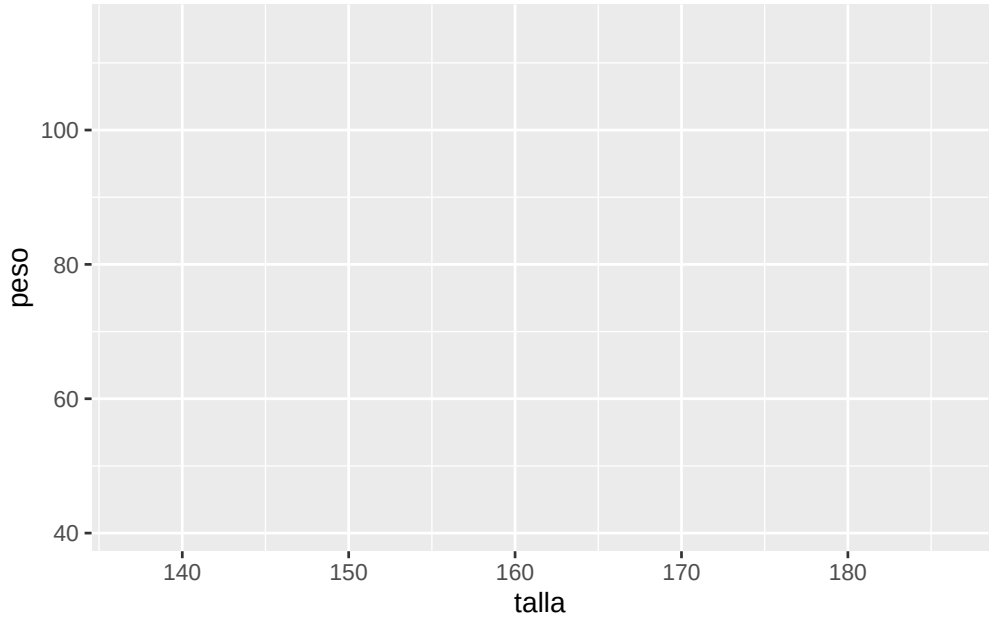
5.2.2. Mapeo estético

La función básica para construir un gráfico en `ggplot2` es `ggplot()`, que crea una estructura inicial (vacía) a la que le irán agregando capas. Anteriormente dijimos que la función `aes()` define el contenido estético del gráfico, es decir, indica cómo se representan visualmente las variables (posición, color, tamaño, forma, etc.).

Es importante notar que: - Si aes() se incluye dentro de ggplot(), el mapeo se aplica a todas las capas. - Si se incluye dentro de una geometría, se aplica solo a esa capa. - aes() genera automáticamente leyendas cuando corresponde.

Veamos cómo funciona y cuáles son sus implicancias:

```
facultad |>
  # solo la capa estética aes()
  ggplot(aes(x = talla, y = peso))
```



Este código genera un gráfico vacío que contiene únicamente los ejes definidos (`peso` y `talla`), pero aún no muestra los datos.

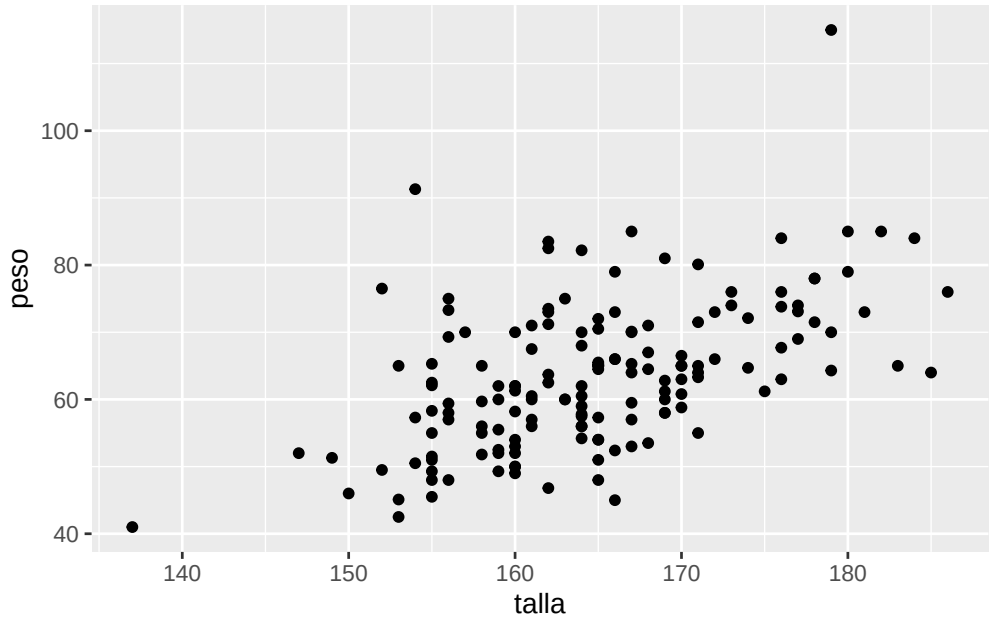
5.2.3. Capas geométricas

Para visualizar los datos, es necesario agregar capas geométricas mediante funciones `geom_`. Estas representan los objetos gráficos que se dibujan en el gráfico.

Las capas se agregan utilizando el operador +:

```
facultad |>
  # mapeo estético
  ggplot(aes(x = talla, y = peso)) +

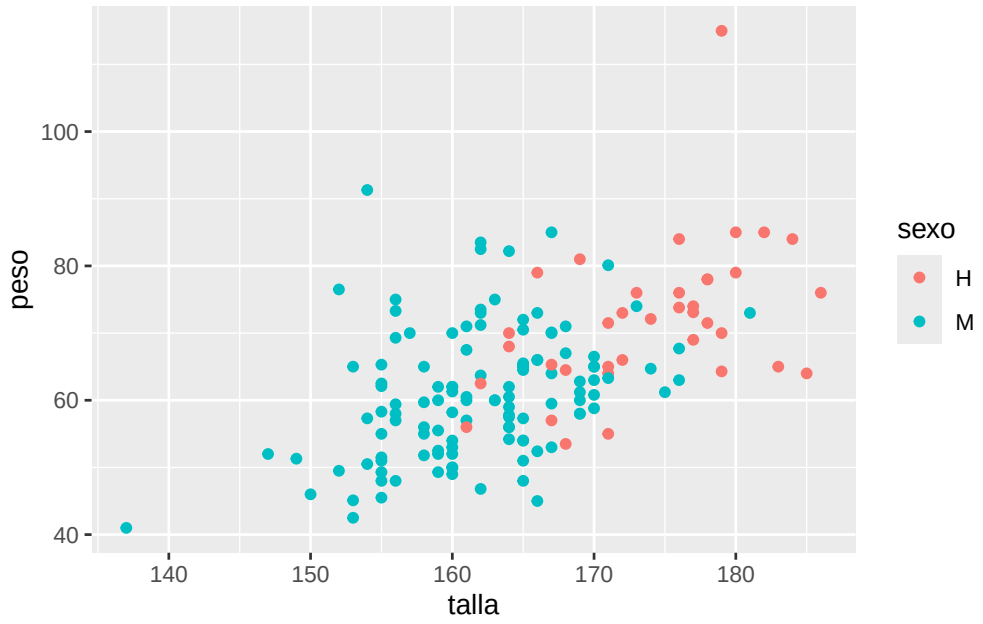
  # agregamos la capa geométrica de puntos
  geom_point()
```



Podemos diferenciar los puntos según `sexo` incorporando esa variable al mapeo estético:

```
facultad |>
  # mapeo estético
  ggplot(aes(x = talla, y = peso, color = sexo)) +

  # agregamos la capa geométrica de puntos
  geom_point()
```

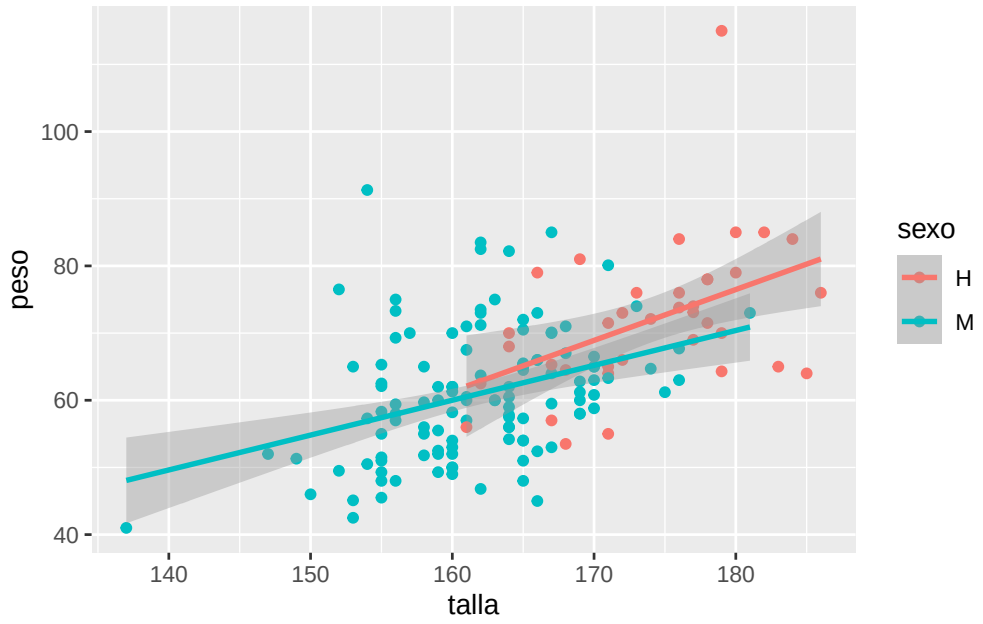


También es posible superponer capas. Por ejemplo, agregar rectas de regresión:

```
facultad |>
  # mapeo estético
  ggplot(aes(x = talla, y = peso, color = sexo)) +

  # agregamos la capa geométrica de puntos
  geom_point() +

  # agregamos una segunda capa geométrica para la recta de regresión
  geom_smooth(method = "lm")
```



La función `geom_smooth()` permite aplicar distintos métodos de ajuste. En este caso, `"lm"` indica una regresión lineal, incluyendo su intervalo de confianza.

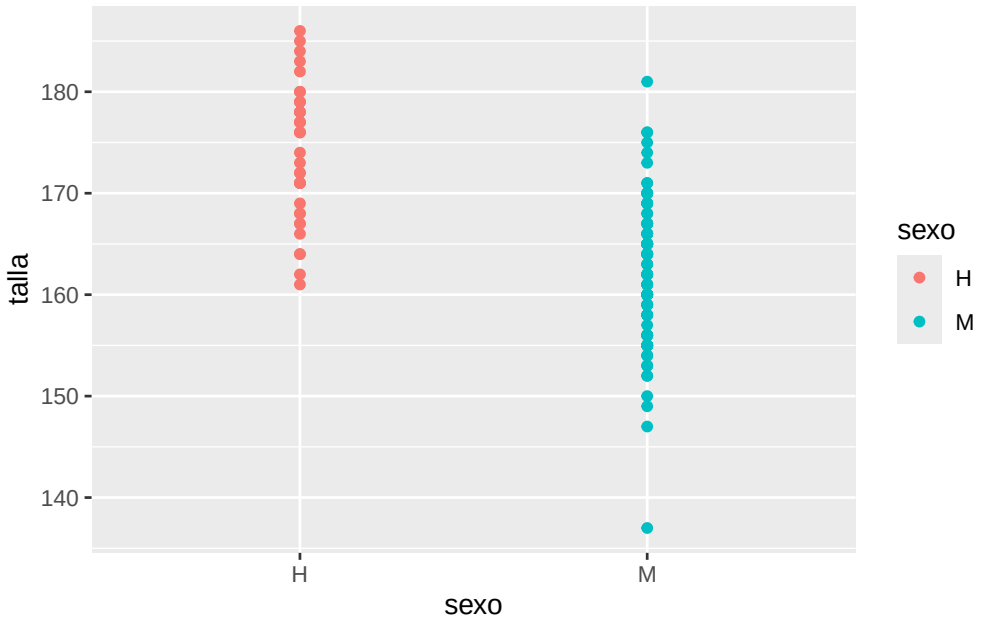
Algunas de las capas geométricas más utilizadas son:

- Histogramas: `geom_histogram()`
- Gráfico de caja y bigotes (*boxplot*): `geom_boxplot()`
- Gráfico de barras: `geom_bar()`
- Gráfico de columnas: `geom_col()`
- Gráfico de puntos (*scatter plot*): `geom_point()` o `geom_jitter()`
- Gráfico de líneas: `geom_line()` o `geom_path()`
- Gráfico de violín: `geom_violin()`
- Tendencias: `geom_smooth()`
- Curvas de densidad: `geom_density()`

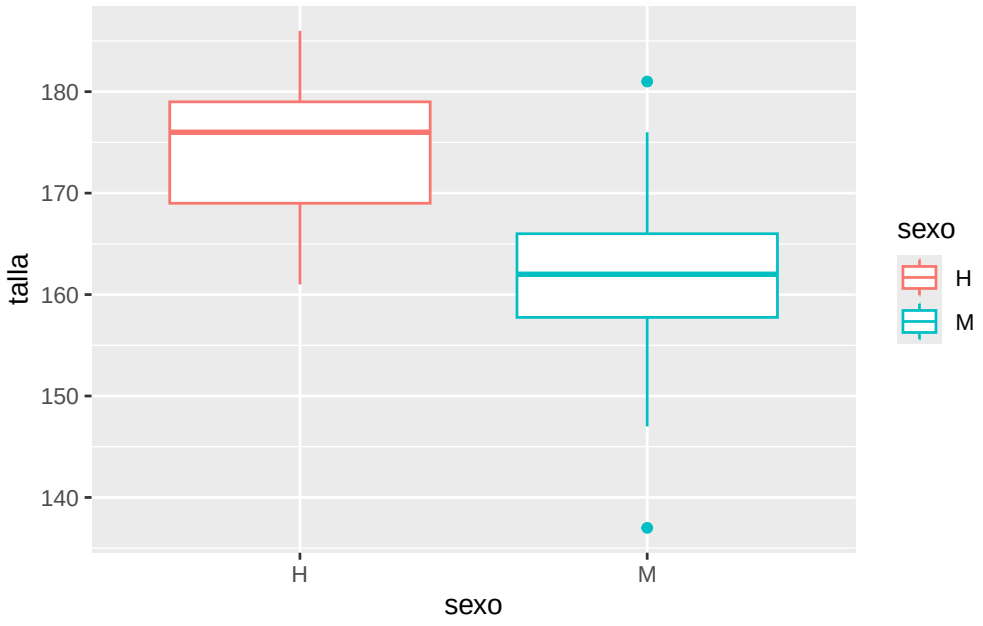
Todas estas funciones pueden aplicarse sobre los mismos datos, y su uso depende del objetivo del análisis. Las capas se grafican secuencialmente, de modo que la última geometría en añadirse aparece adelante.

A continuación, algunos ejemplos para visualizar la relación entre `sexo` y `talla` utilizando distintas capas geométricas:

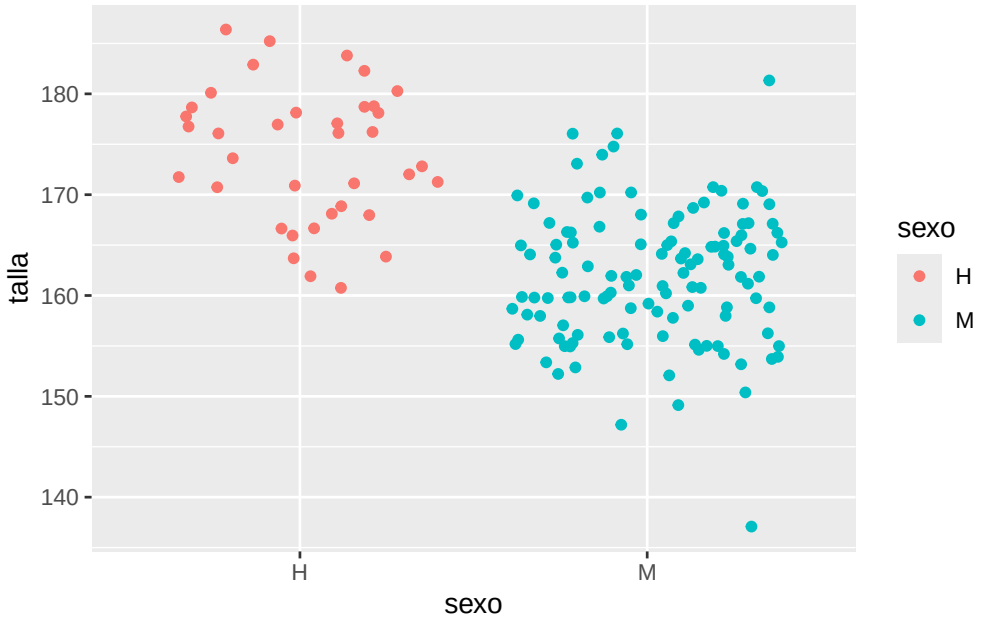
```r
# Gráfico de puntos
facultad |>
  ggplot(aes(x = sexo, y = talla, color = sexo)) +
  # capa geométrica de puntos
  geom_point()
```



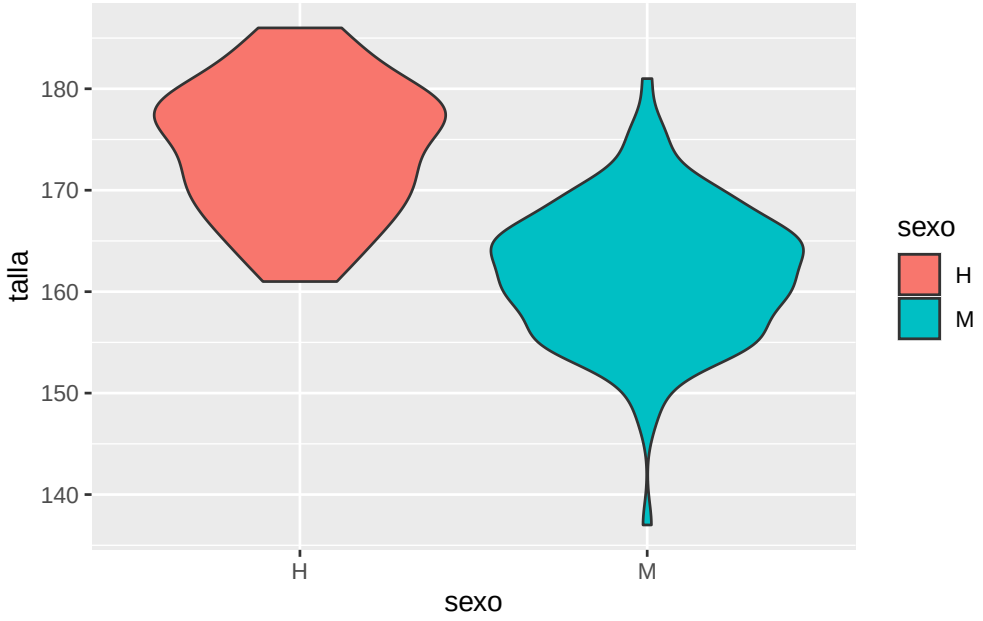

```r
# Boxplot
facultad |>
  ggplot(aes(x = sexo, y = talla, color = sexo)) +
  # capa geométrica de boxplot
  geom_boxplot()
```




```
# Entramado de puntos
facultad |>
  ggplot(aes(x = sexo, y = talla, color = sexo)) +
  # capa geométrica jitter (entramado de puntos)
  geom_jitter()
```



```r
# Gráfico de violín
facultad |>
  ggplot(aes(x = sexo, y = talla, fill = sexo)) +
  # capa geométrica de violin
  geom_violin()
```



En el último caso se utiliza `fill` en lugar de `color`. Mientras `color` controla el contorno, `fill` define el relleno de las geometrías.

5.2.4. Estéticas estáticas y dinámicas

En `ggplot2`, las estéticas son propiedades visuales asociadas a los datos en las capas geométricas. No incluyen elementos como títulos o etiquetas.

Algunas estéticas frecuentes:

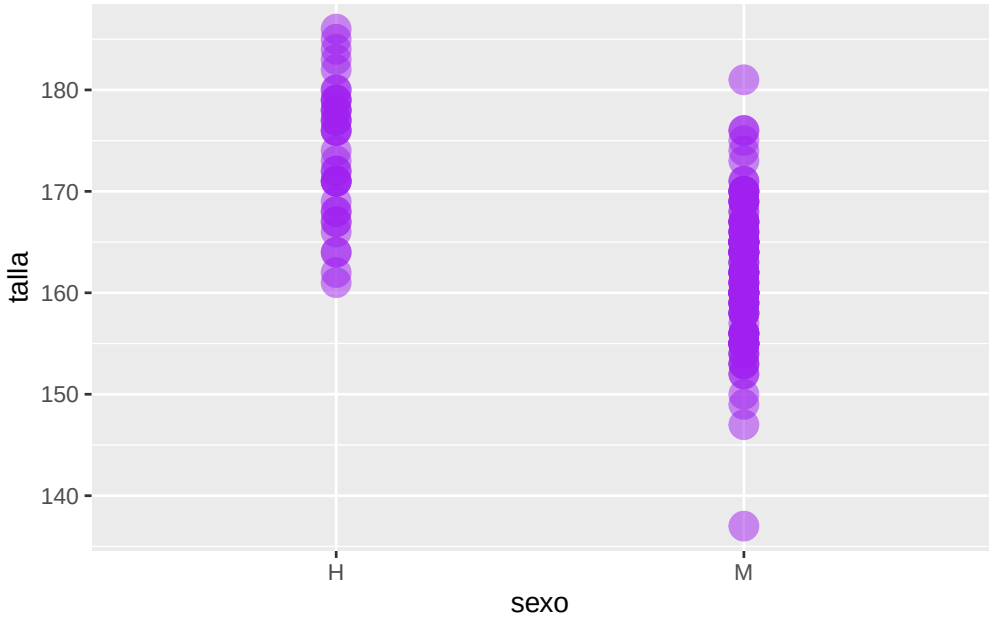
- `color` / `colour`
- `fill`
- `alpha`
- `size`

- `linewidth` / `lwd`
- `linetype`/ `lty`

Valores estáticos

Si queremos que todas las observaciones tengan una misma estética, colocamos los argumentos por fuera de `aes()`:

```
facultad |>
  ggplot(aes(x = sexo, y = talla)) +
  # capa geométrica de puntos
  geom_point(
    color = "purple", # Color de los puntos
    size = 5, # Tamaño de los puntos
    alpha = .5 # Transparencia de los puntos
  )
```



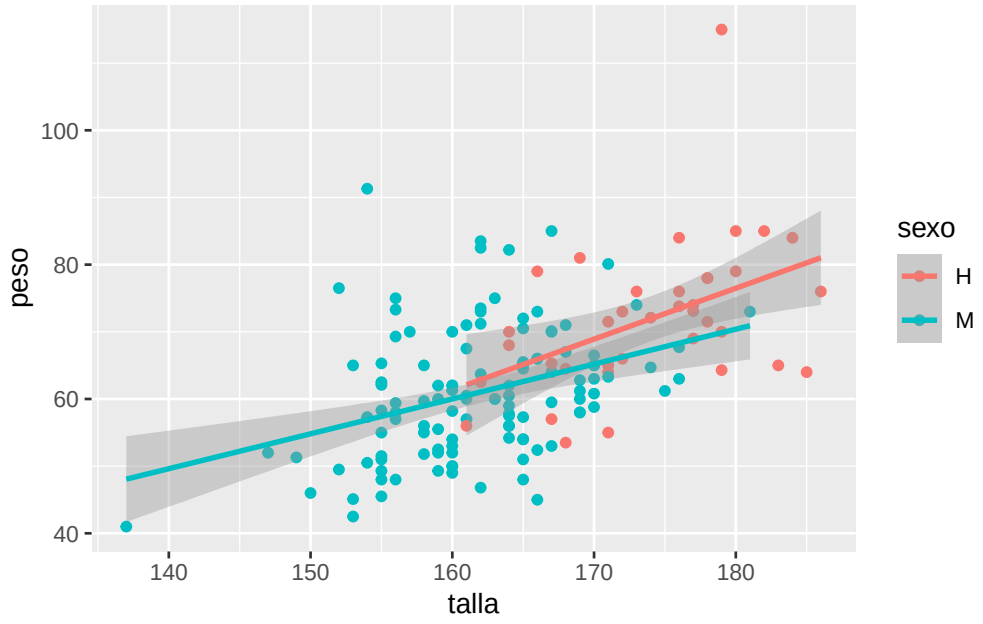
Valores dinámicos

Se definen dentro de `aes()` y aplican a todas las capas:

```r
facultad |>
  # Mapeo estético
  ggplot(aes(x = talla, y = peso, color = sexo)) +

  # agregamos la capa geométrica de puntos
  geom_point() +

  # agregamos una segunda capa geométrica para la recta de regresión
  geom_smooth(method = "lm")
```

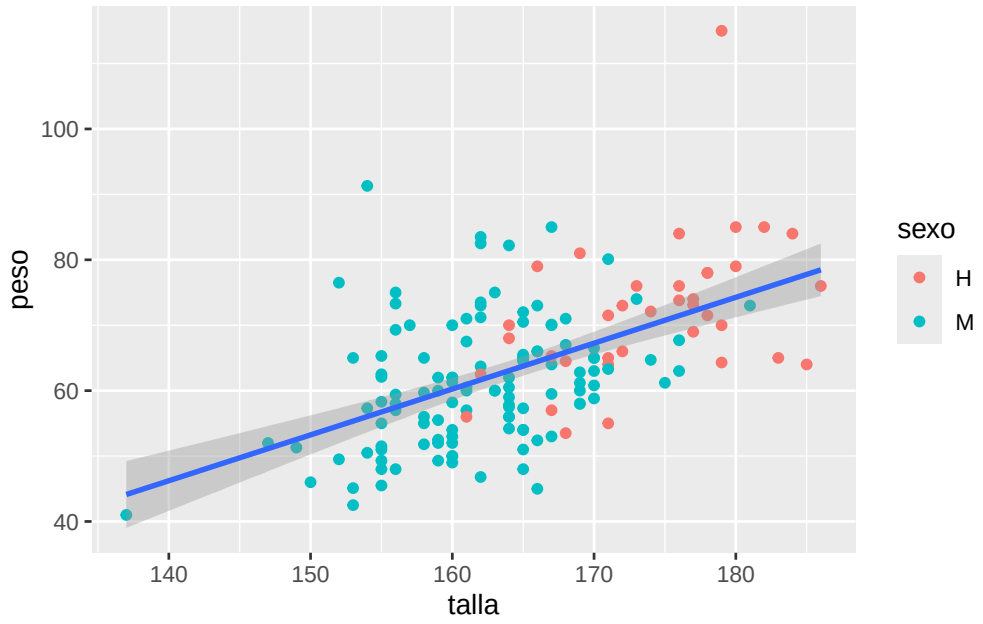


En este otro ejemplo, el color por sexo solo se aplica a los puntos de `geom_point()`, mientras que la regresión se calcula sobre el conjunto completo:

```r
facultad |>
  # mapeo estético
  ggplot(aes(x = talla, y = peso)) +

  # agregamos la capa geométrica de puntos coloreada por sexo
  geom_point(aes(color = sexo)) +

  # agregamos una segunda capa geométrica para la recta de regresión
  geom_smooth(method = "lm")
```



Este comportamiento permite una gran flexibilidad al construir gráficos, definiendo qué capas responden a cada mapeo estético.

5.3. Personalización de escalas

El sistema de escalas en `ggplot2` permite ajustar distintos aspectos visuales de un gráfico, como colores de contorno y relleno, ejes, tamaños y tipos de línea, entre otros.

Todas las funciones asociadas a escalas comienzan con el prefijo `scale_`, seguido del atributo que se desea modificar (por ejemplo, `scale_fill_` para el color de relleno).

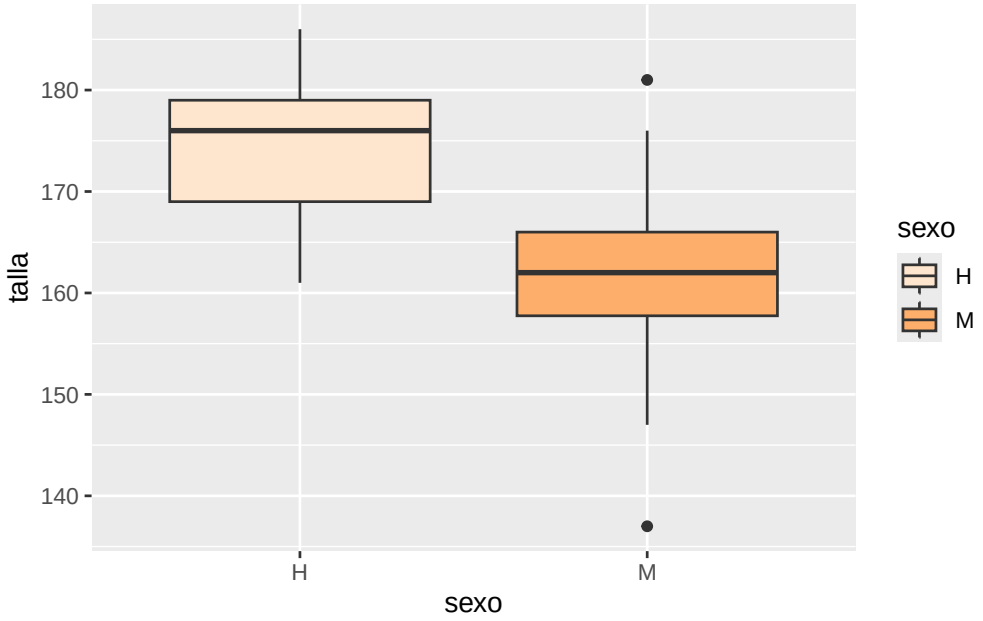
5.3.1. Escalas de colores

A continuación, se presentan algunos ejemplos utilizando el conjunto de datos `facultad`:

```
facultad |>
  ggplot(aes(x = sexo, y = talla, fill = sexo)) +

  # capa geométrica boxplot
  geom_boxplot() +
```

```
  # paleta de naranjas
  scale_fill_brewer(palette = "Oranges")
```



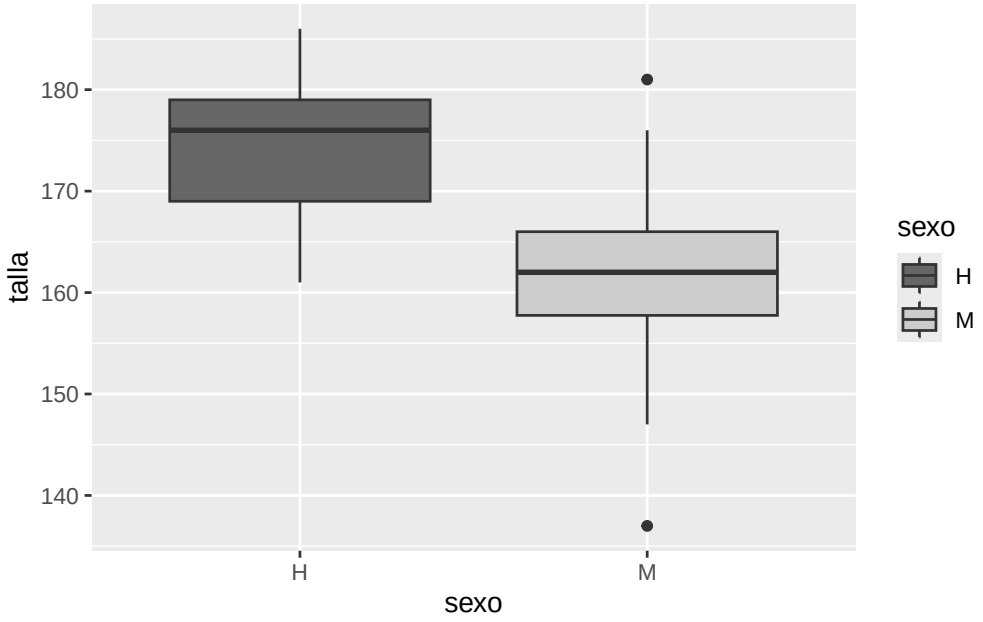
En este ejemplo, se aplica la función `scale_fill_brewer()`, que utiliza una paleta de colores (en este caso, `"Oranges"`) y se vincula con el argumento `fill` definido en `aes()`, determinando el color de relleno del *boxplot*.

Otra alternativa es utilizar una escala de grises mediante `scale_fill_grey()`:

```
facultad |>
  ggplot(aes(x = sexo, y = talla, fill = sexo)) +

  # capa geométrica boxplot
  geom_boxplot() +

  # paleta en escala de grises
  scale_fill_grey(start = 0.4, end = 0.8)
```



Se recomienda trabajar con paletas de colores accesibles para personas con daltonismo y otras dificultades en la percepción visual, como las incluidas en las dependencias `RColorBrewer` y `viridisLite`, así como en paquetes específicos con paletas *colorblind-friendly*, como `scico` (Pedersen y Crameri 2023) y `cols4all` (Tennekes y Puts 2023).

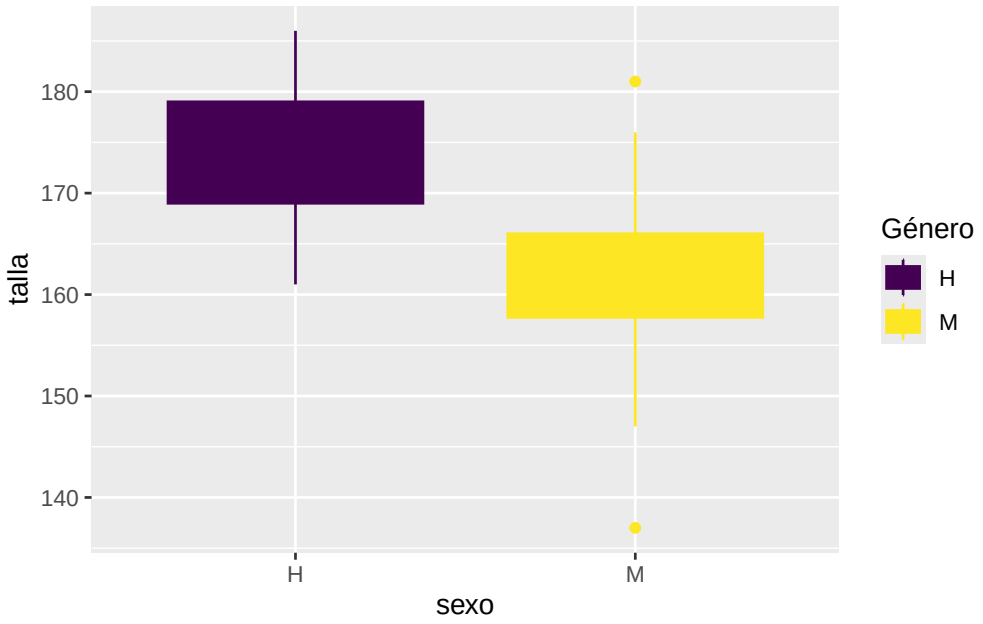
En versiones recientes de `ggplot2`, varias funciones de escala permiten utilizar el argumento `aesthetics`, que facilita aplicar una misma escala a múltiples estéticas simultáneamente. Esto resulta especialmente útil cuando queremos usar una misma paleta de colores tanto para el contorno (`color`) como para el relleno (`fill`).

Retomando el ejemplo anterior:

```
facultad |>
  ggplot(aes(x = sexo, y = talla, fill = sexo, color = sexo)) +

  # capa geométrica boxplot
  geom_boxplot() +

  # paleta en escala viridis
  scale_fill_viridis_d(
    name = "Género",
    aesthetics = c("fill", "color")
  )
```



En este caso, la paleta `"viridis"` se aplica simultáneamente al relleno y al contorno de las cajas, evitando inconsistencias visuales entre ambas estéticas.

Sin `aesthetics` los colores de relleno y de línea deberían especificarse como:

```
# paleta en escala viridis
scale_fill_viridis_d(name = "Género") +
  scale_color_viridis_d()
```

Lo que duplica código y puede desincronizar los cambios en una paleta.

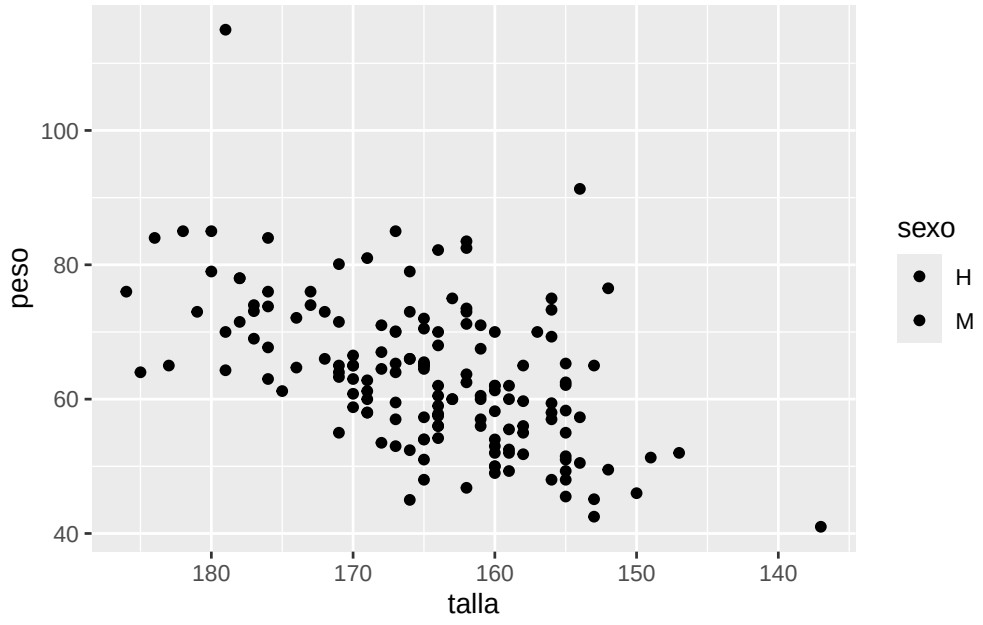
5.3.2. Escalas de ejes

También es posible modificar las escalas de los ejes. Por ejemplo, invertir el eje X con `scale_x_reverse()`:

```
facultad |>
  ggplot(aes(x = talla, y = peso, fill = sexo)) +

  # capa geométrica de puntos
  geom_point() +
```

```
  # invierte el eje x
  scale_x_reverse()
```



La inclusión de `scale_x_reverse()` provoca que la variable `talla` se muestre en orden descendente (de mayor a menor).

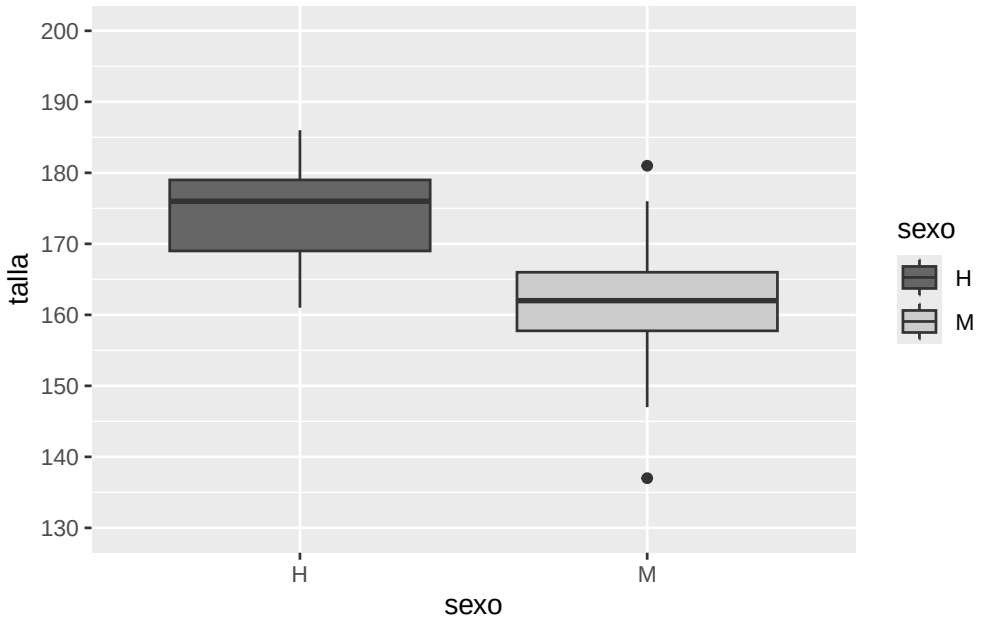
Por último, se pueden personalizar los cortes del eje Y mediante `scale_y_continuous()`. En el siguiente ejemplo, se define que el eje Y vaya de 130 a 200 cm, con marcas cada 10 unidades:

```
facultad |>
  ggplot(aes(x = sexo, y = talla, fill = sexo)) +

  # capa geométrica boxplot
  geom_boxplot() +

  # paleta en escala de grises
  scale_fill_grey(start = 0.4, end = 0.8) +

  # puntos de corte del eje Y
  scale_y_continuous(
    limits = c(130, 200), # límites
    breaks = seq(130, 200, 10)
  ) # nro de etiquetas
```



5.4. Transformaciones estadísticas

En ggplot2, algunos gráficos no requieren transformaciones estadísticas explícitas, como los gráficos de dispersión. Sin embargo, otros —como histogramas, *boxplots* o curvas de tendencia— incorporan **transformaciones estadísticas implícitas** que pueden modificarse o personalizarse.

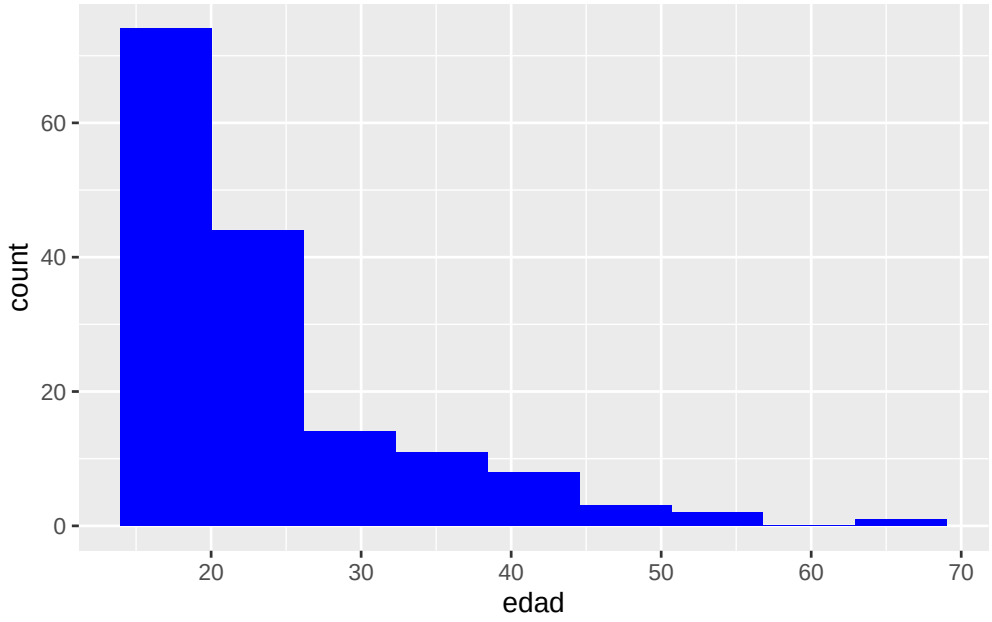
Estas transformaciones pueden:

- estar integradas dentro de las funciones geométricas (`geom_`)
- agregarse como capas independientes mediante funciones `stat_`

Por ejemplo, en un histograma la transformación estadística consiste en agrupar los datos en intervalos (clases) y contar las observaciones en cada uno. Esta transformación está incluida en `geom_histogram()`, pero puede parametrizarse:

```
facultad |>
  ggplot(aes(edad)) +

  # capa geométrica histograma
  geom_histogram(bins = nclass.Sturges(facultad$edad), fill = "Blue")
```



En este caso, se utiliza la regla de Sturges —a través de `nclass.Sturges()`— para determinar automáticamente la cantidad de clases de la variable `edad`.

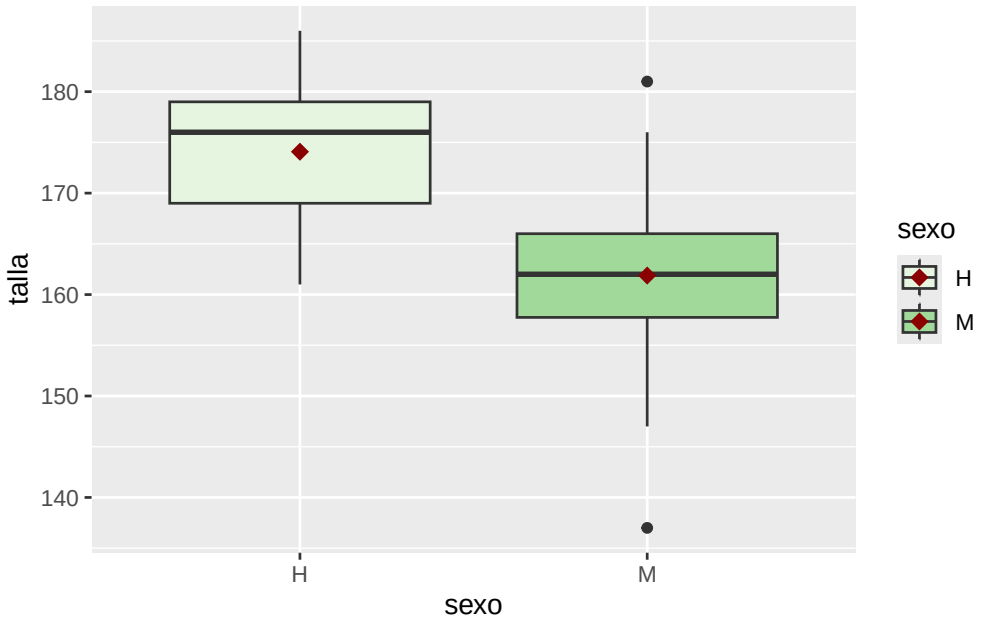
También es posible agregar transformaciones estadísticas como capas adicionales. Por ejemplo, si queremos mostrar la media de `talla` para cada grupo de `sexo` sobre un *boxplot*, podemos utilizar `stat_summary()`:

```
facultad |>
  ggplot(aes(x = sexo, y = talla, fill = sexo)) +

  # capa geométrica boxplot
  geom_boxplot() +

  # paleta de tonos verdes
  scale_fill_brewer(palette = "Greens") +

  # añade capa summary
  stat_summary(
    fun = mean,
    color = "darkred",
    geom = "point",
    shape = 18,
    size = 3
  )
```



La función `stat_summary()` permite calcular estadísticas (como `mean`, `median`, `sd`, entre otras) y representarlas mediante una geometría. En este caso, la media de `talla` se muestra como un punto rojo oscuro (`color = "darkred"`), con forma romboidal (`shape = 18`) y tamaño destacado (`size = 3`).

5.5. Facetado

El **facetado** permite dividir un gráfico en múltiples paneles (o viñetas) según los niveles de una o más variables categóricas. Esto resulta especialmente útil para comparar patrones entre grupos sin sobrecargar un único gráfico.

En `ggplot2` existen dos funciones principales para implementar esta técnica:

- `facet_wrap()`: divide los datos según una única variable categórica, generando paneles dispuestos automáticamente en filas y columnas.
- `facet_grid()`: permite crear una matriz de paneles a partir de dos variables categóricas, una definida en las filas y otra en las columnas.

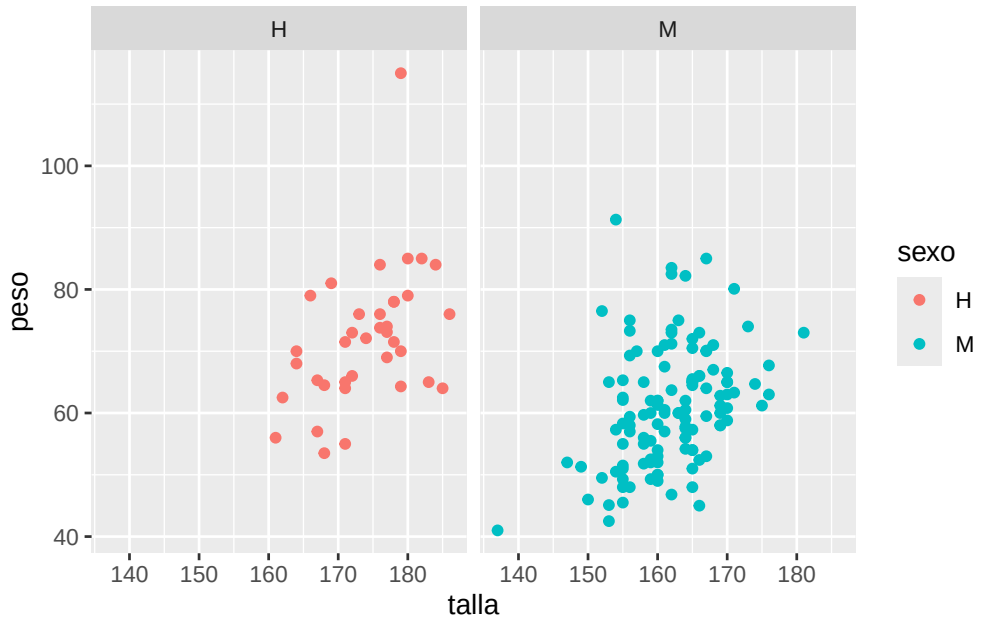
Retomemos el gráfico de dispersión entre `talla` y `peso`, y generemos paneles separados para cada nivel de la variable `sexo` usando `facet_wrap()`:

```r
facultad |>

  ggplot(aes(x = talla, y = peso, color = sexo)) +

  # capa geométrica de puntos
  geom_point() +

  # separa en paneles por sexo
  facet_wrap(~sexo)
```



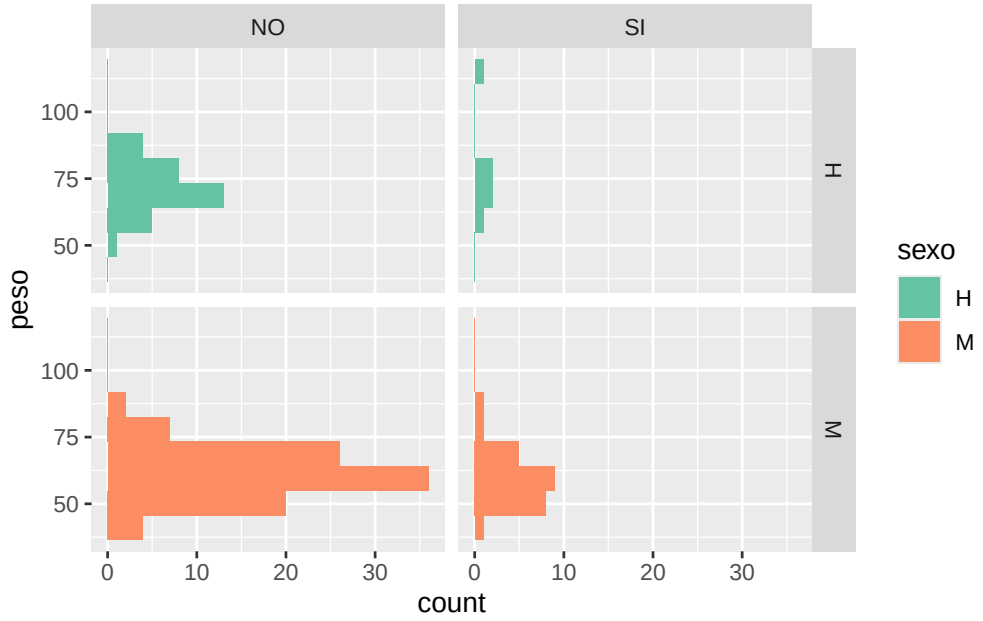
Ahora utilizamos `facet_grid()` para crear una matriz de histogramas a partir del cruce de las variables `sexo` y `fuma`. En cada panel se representa la distribución de `peso`:

```r
facultad |>
  ggplot(aes(y = peso, fill = sexo)) +

  # capa geométrica histograma
  geom_histogram(bins = nclass.Sturges(facultad$peso)) +

  # paleta de colores
  scale_fill_brewer(palette = "Set2") +
```

```
# separa en paneles por sexo y fuma
facet_grid(sexo ~ fuma)
```



Como se observa en estos ejemplos, el facetado se integra naturalmente con otros componentes de la gramática de gráficos: mapeo estético (`aes()`), capas geométricas (`geom_`) y escalas (`scale_`).

El número de combinaciones posibles es muy amplio, dada la flexibilidad de `ggplot2`. Sin embargo, el objetivo de este material es comprender los principios fundamentales sobre los que se construye esta *gramática de los gráficos*.

5.6. Sistema de coordenadas

En algunos casos, puede ser útil modificar el sistema de coordenadas predeterminado de un gráfico. Mientras que las funciones `scale_` controlan **cómo se representan los datos**, las funciones de coordenadas (`coord_`) modifican **cómo se visualiza el espacio del gráfico**.

En `ggplot2` una de las transformaciones más comunes es invertir la orientación de los ejes mediante `coord_flip()`, lo que resulta útil, por ejemplo, para presentar gráficos de barras en forma horizontal:

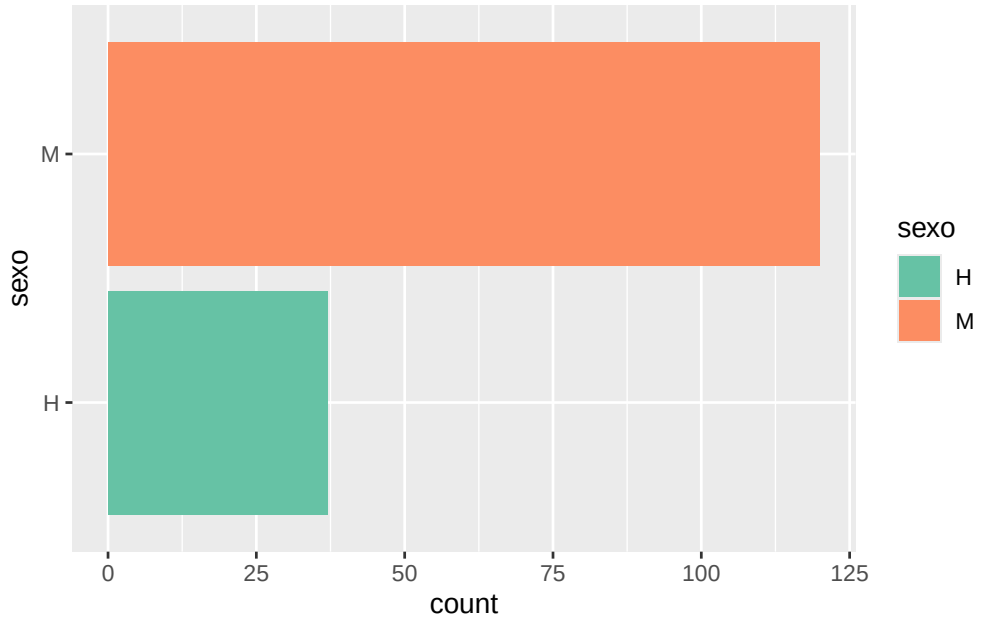
```r
facultad |>

  ggplot(aes(sexo, fill = sexo)) +

  # capa geométrica barras
  geom_bar() +

  # paleta de colores
  scale_fill_brewer(palette = "Set2") +

  # invierte disposición de ejes
  coord_flip()
```



5.7. Temas

ggplot2 incluye un conjunto de temas gráficos predefinidos que permiten modificar el aspecto general del gráfico. El tema por defecto es `theme_gray()`, pero puede cambiarse agregando una capa `theme_*()` dentro del gráfico.

Repetimos el gráfico anterior, esta vez utilizando el tema blanco y negro (`theme_bw()`):

```r
facultad |>

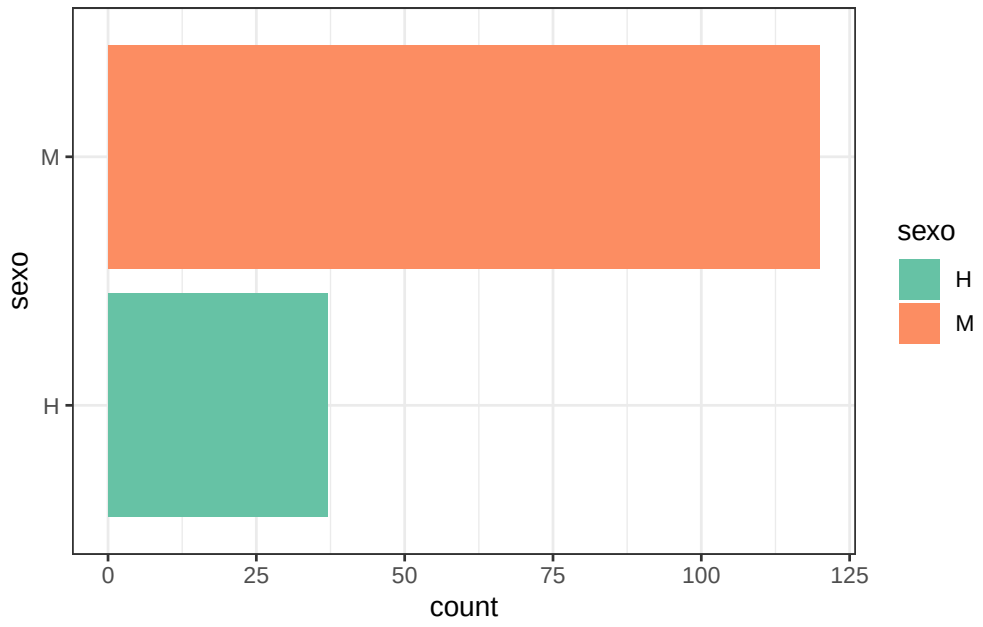
  ggplot(aes(sexo, fill = sexo)) +

  # capa geométrica barras
  geom_bar() +

  # paleta de colores
  scale_fill_brewer(palette = "Set2") +

  # invierte disposición de ejes
  coord_flip() +

  # tema en blanco y negro
  theme_bw()
```



También podemos aplicar un tema de fondo oscuro con `theme_dark()`:

```r
facultad |>

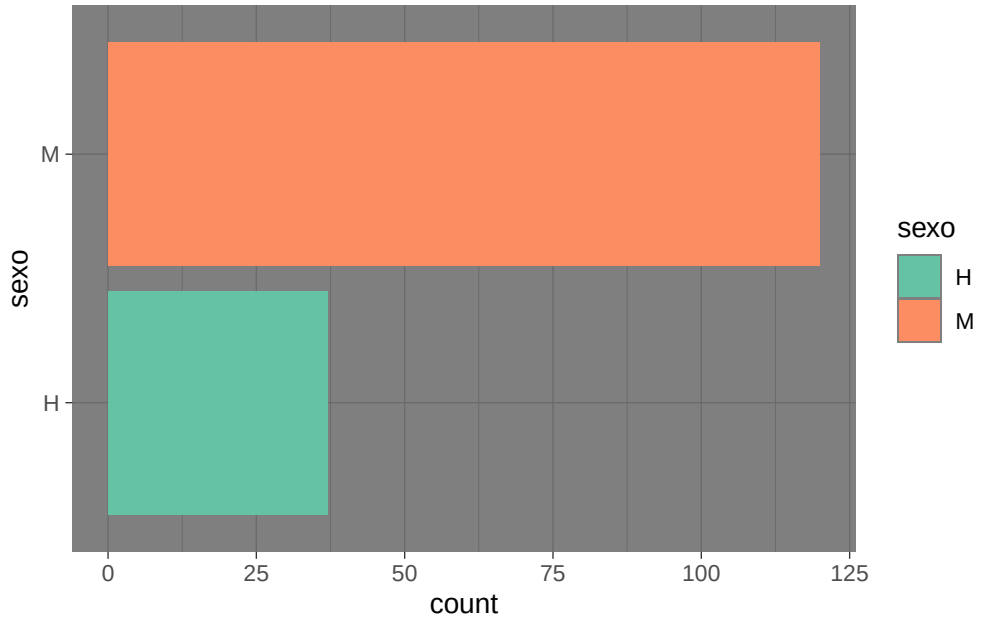
  ggplot(aes(sexo, fill = sexo)) +
```

```r
# capa geométrica barras
geom_bar() +

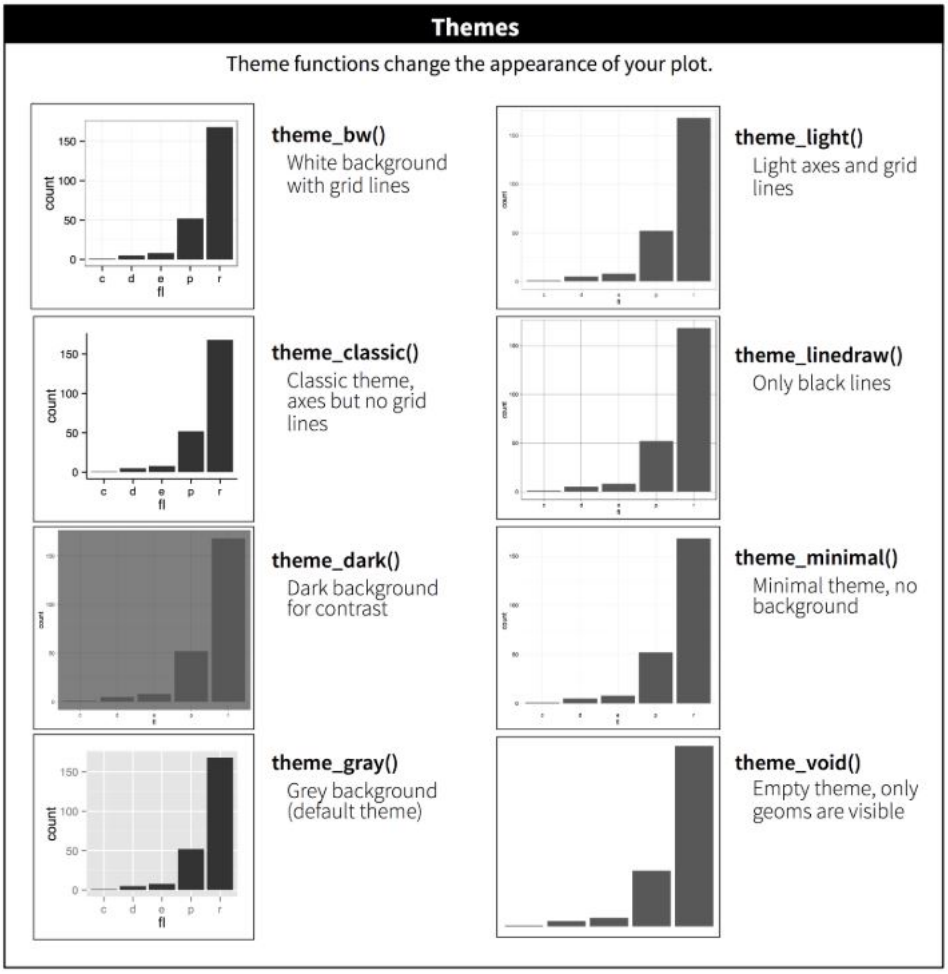
# paleta de colores
scale_fill_brewer(palette = "Set2") +

# invierte disposición de ejes
coord_flip() +

# tema con fondo oscuro
theme_dark()
```



A continuación se muestra un cuadro con los principales temas disponibles en `ggplot2` y sus características visuales:

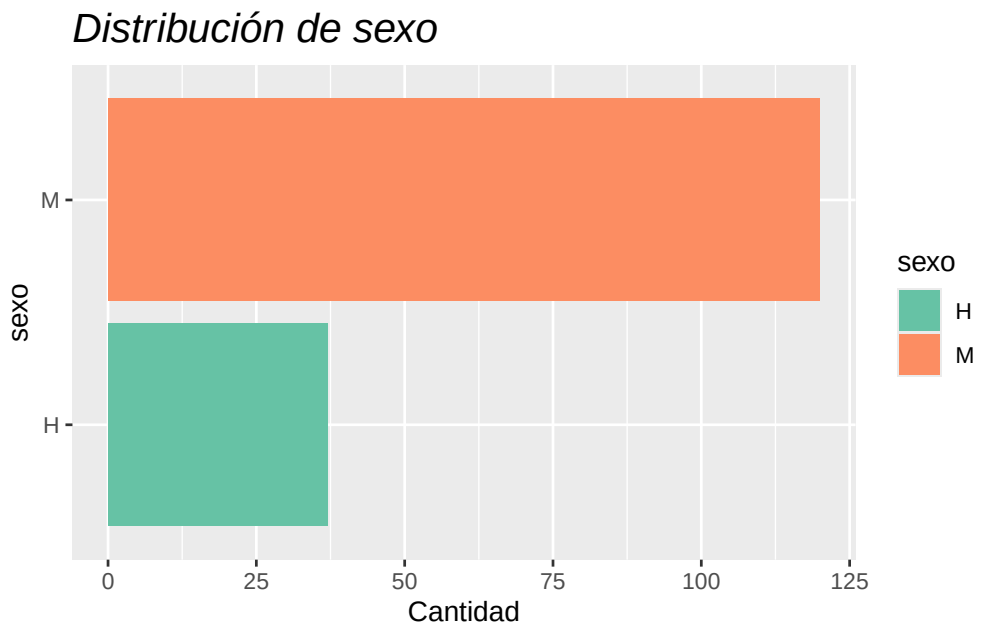


Además del aspecto general del gráfico, es posible agregar títulos, subtítulos y etiquetas de ejes con la función `labs()`. Por ejemplo:

```
labs(
  x = "Etiqueta X",
  y = "Etiqueta Y",
  title = "Título del gráfico",
  subtitle = "Subtítulo del gráfico"
)
```

También se pueden ajustar detalles del texto, como tipo de fuente o tamaño, mediante la función `theme()`:

```r
facultad |>
  ggplot(aes(sexo, fill = sexo)) +
  # capa geométrica barras
  geom_bar() +
  # paleta de colores
  scale_fill_brewer(palette = "Set2") +
  # invierte disposición de ejes
  coord_flip() +
  # cambia etiquetas de los ejes
  labs(y = "Cantidad", title = "Distribución de sexo") +
  # modifica el estilo de fuente del título
  theme(plot.title = element_text(face = "italic", size = 16))
```



5.8. Paquete `esquisse`



El paquete`esquisse` (Meyer y Perrier 2025) es una extensión de `ggplot2` que permite crear gráficos de manera interactiva, mediante una interfaz gráfica intuitiva basada en el sistema de arrastrar y soltar (*drag & drop*).

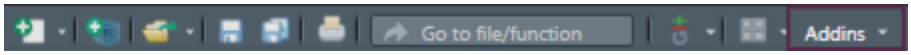
Con `esquisse` es posible:

- Explorar visualmente los datos según su tipo.
- Asignar variables a diferentes estéticas del gráfico (ejes, color, tamaño, etc.).
- Exportar los resultados en distintos formatos.
- Recuperar el código R que genera el gráfico, facilitando su reproducción y modificación.

El paquete se instala mediante el menú **Packages** de RStudio o ejecutando:

```
install.packages("esquisse")
```

Luego se puede acceder a la aplicación por medio del acceso Addins:



o ejecutando en consola:

```
esquisser()
```

También se puede agregar el nombre de la tabla de datos dentro de los paréntesis

```
esquisser(datos)
```

Para más detalles, se puede consultar la viñeta del paquete disponible en CRAN o en su repositorio de GitHub.

5.9. Extensiones de `ggplot2`

Una de las grandes fortalezas de `ggplot2` es su ecosistema de extensiones. Hasta el momento, existen más de 158 **paquetes** desarrollados para ampliar sus funcionalidades, muchos de ellos diseñados para facilitar tareas específicas o agregar nuevas capas, geometrías, temas, escalas o herramientas interactivas.

Algunas de las extensiones que utilizaremos durante el curso son:

- `patchwork` (Pedersen 2025): permite combinar múltiples gráficos de `ggplot2` en una misma figura de forma sencilla y controlada.
- `GGally` (Schloerke et al. 2025): extiende `ggplot2` con funciones para crear matrices de gráficos (como `ggpairs()`), útiles para análisis exploratorio multivariado.
- `ggpubr` (Kassambara 2026): facilita la creación de gráficos listos para publicar, con funciones simplificadas para agregar títulos, etiquetas, comparaciones estadísticas y anotaciones.
- `see` (Lüdecke et al. 2021): ofrece temas, paletas de colores y escalas compatibles con personas con daltonismo, además de geometrías personalizadas.
- `survminer`(Kassambara et al. 2026): diseñado para la visualización de análisis de supervivencia (como curvas de Kaplan-Meier) utilizando `ggplot2`.
- `ggfortify`(Horikoshi y Tang 2018): permite graficar objetos estadísticos como modelos lineales, PCA o series temporales sin necesidad de escribir código `ggplot2` desde cero.

Estas extensiones permiten ampliar la flexibilidad y expresividad gráfica de `ggplot2` sin perder la estructura gramatical que lo caracteriza.

Referencias

6 Análisis exploratorio de datos



6.1. Introducción

El análisis exploratorio de datos (conocido como **EDA**, su sigla en inglés) es un enfoque de análisis fundamental para resumir y visualizar las características importantes de un conjunto de datos.

John Tukey, estadístico estadounidense, fue uno de los principales impulsores de este enfoque. En 1977 publicó el libro *Exploratory Data Analysis*, donde, entre otras contribuciones, introdujo el gráfico *boxplot* (diagrama de caja y bigotes).

En términos sencillos, antes de avanzar hacia el análisis formal o la construcción de modelos estadísticos, resulta esencial explorar, conocer y describir las variables presentes en nuestra tabla de datos.

Entre los principales objetivos perseguidos por EDA se encuentran:

- Conocer la estructura de la tabla de datos y sus tipos de variable.
- Detectar observaciones incompletas (valores *missing* o `NA`).
- Explorar la distribución de las variables de interés a partir de:
    - Estadísticos descriptivos
    - Representaciones gráficas
- Detectar valores atípicos (*outliers*).

Aclaración

En este documento utilizaremos funciones del lenguaje R basadas en la filosofía *tidyverse*, junto con otros paquetes diseñados para tareas específicas. Esto no implica que no se puedan emplear funciones del R base; sin embargo, el ecosistema `tidyverse` facilita la comprensión y legibilidad del código.

Presentaremos estas diferentes funciones de distintos paquetes que pueden servir en cada etapa de un EDA. Los paquetes con los que trabajaremos son:

- `tidyverse`
- `skimr`
- `dlookr`
- `janitor`

> ⚠ **Nota:** Algunos paquetes, como `dlookr`, pueden generar falsos positivos en la detección del antivirus durante el proceso de instalación. Sugerimos desactivar momentáneamente el antivirus para evitar inconvenientes.

Una vez instalados, podemos activar los paquetes con el siguiente código:

```
# Carga de paquetes
library(skimr)
library(janitor)
library(dlookr)
library(tidyverse)
```

Se recomienda cargar `tidyverse` al final de la lista para evitar conflictos con funciones que puedan solaparse entre paquetes.

> Es importante destacar que no existe un único camino y/o función para realizar un análisis exploratorio. Esta selección de herramientas puede adaptarse según las preferencias y necesidades de cada usuario. Por lo tanto, quienes ya tengan familiaridad con otras funciones o paquetes pueden continuar utilizándolos sin inconvenientes.

Para ilustrar los pasos del análisis exploratorio, utilizaremos un archivo con datos ficticios llamado "`datos2.txt`", que contiene variables de distintos tipos.

6.2. Conocer la estructura de la tabla de datos y sus tipos de variable

El primer paso en la exploración de un conjunto de datos es conocer su estructura y tamaño:

- El **tamaño** se refiere a la cantidad de observaciones (filas) y de variables (columnas).
- La **estructura** incluye cómo están organizadas las variables, qué tipo de datos contiene cada una y qué categorías o valores pueden tomar.

Comenzaremos por cargar los datos de ejemplo con la función `read_csv2()` de `tidyverse`:

```
datos <- read_csv2("datos/datos2.txt")
```

Una vez cargados los datos, la función `glimpse()` permite obtener una visión general de la tabla:

```
glimpse(datos)
```

```
Rows: 74
Columns: 7
$ id      <dbl> 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18,~
$ sexo    <chr> "M", "M", "M", "M", "M", "M", "M", "M", NA, "F", "F", "M", "F"~
$ edad    <dbl> 76, 68, 50, 49, 51, 68, 70, 64, 60, 57, 83, 76, 27, 34, 17, 45~
$ peso    <dbl> 71.0, 71.0, 79.0, 71.0, 87.0, 75.0, 80.0, 83.0, 69.0, 73.0, 60~
$ talla   <dbl> 167.0, 164.0, 164.0, 164.0, 167.5, 170.0, 166.0, 160.0, 160.0,~
$ trabaja <lgl> FALSE, FALSE, FALSE, TRUE, TRUE, FALSE, NA, TRUE, TRUE, TRUE, ~
$ fecha   <chr> "20/10/2020", "20/10/2020", "20/10/2020", "05/11/2020", "05/11~
```

Esta función nos informa que la tabla contiene, por ejemplo, 74 observaciones y 7 variables, mostrando el tipo de dato de cada una y los primeros valores que aparecen.

Entre los tipos de datos más comunes que podemos encontrar se incluyen:

- `int` (*integer*): números enteros.
- `dbl` (*double*): números reales.
- `lgl` (*logical*): valores lógicos (`TRUE`, `FALSE`).
- `chr` (*character*): texto o cadenas de caracteres.
- `Date`: fechas.
- `fct` (*factor*): variables categóricas con niveles.
- `dttm` (*date-time*): fechas y horas.

Esta primera revisión de la estructura suele complementarse con el **diccionario de datos**, un recurso fundamental que describe el significado, tipo, unidad y codificación de cada variable. Este diccionario puede acompañar tanto a bases de datos generadas en investigaciones propias (fuentes primarias) como a datos provenientes de fuentes secundarias.

Es importante tener en cuenta que el tipo de dato en R no siempre coincide con la naturaleza estadística de la variable. Por ejemplo:

- Una variable codificada como `dbl` puede representar una medida cuantitativa continua, como la edad o el peso.
- Pero también puede representar una variable cualitativa codificada con números. Por ejemplo, si una variable que registra respuestas "Sí" y "No" fue codificada como `1` y `0`, su tipo de dato será numérico (`dbl` o `int`), aunque conceptualmente sea una variable categórica.

Por esta razón, además de inspeccionar el tipo de datos en R, es importante revisar el significado y el uso previsto de cada variable dentro del contexto del análisis.

6.3. Detectar observaciones incompletas

Los valores perdidos o faltantes (conocidos como *missing* en inglés), representados en R por el valor especial `NA`, constituyen un desafío importante en el análisis de datos. Su presencia puede afectar la calidad del análisis y condicionar las decisiones estadísticas posteriores.

Existen numerosos enfoques para el tratamiento de valores faltantes, incluyendo técnicas de imputación y modelado específico. Sin embargo, en este curso nos enfocaremos exclusivamente en cómo **detectar**, **contabilizar** y, en algunos casos, **excluir** valores faltantes utilizando funciones del lenguaje R.

Una forma sencilla de detectar valores faltantes es mediante la función `count()` del paquete `dplyr`. Al aplicarla a una variable, la salida incluye una fila adicional que informa cuántos valores `NA` hay:

```
datos |>
  count(trabaja)
```

```
# A tibble: 3 x 2
  trabaja     n
  <lgl>   <int>
1 FALSE      26
2 TRUE       39
3 NA          9
```

Una alternativa más completa es la función `find_na()` del paquete `dlookr` (Ryu 2025):

```
find_na(datos, rate = T)
```

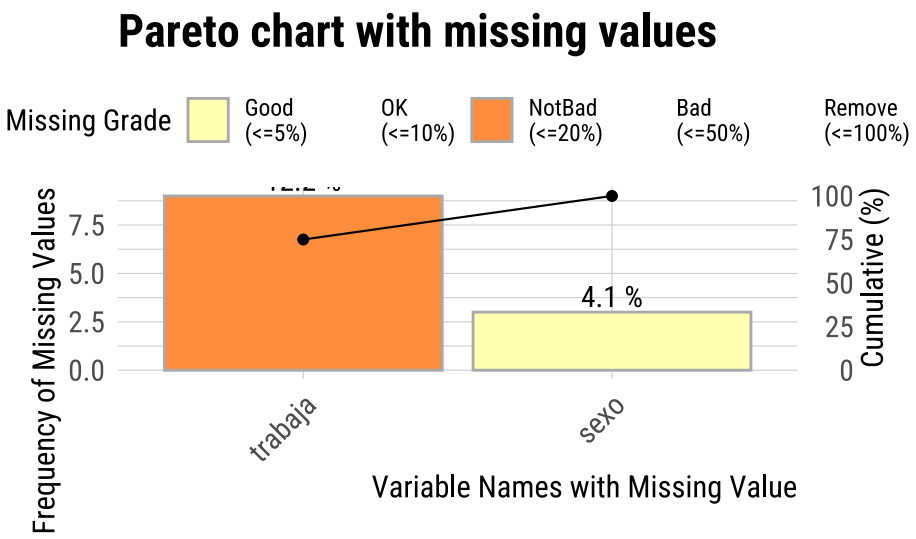
```
     id    sexo    edad    peso   talla trabaja   fecha
  0.000   4.054   0.000   0.000   0.000  12.162   0.000
```

Esta función se puede aplicar al conjunto de datos completo y devuelve, para cada variable, la cantidad y el porcentaje de valores `NA`. Por ejemplo, podríamos observar que la variable `sexo` tiene alrededor de un 4 % de valores faltantes, y la variable `trabaja`, algo más del 12 %.

Estos porcentajes pueden ayudarnos a decidir si una variable debe incluirse en un análisis o si es conveniente excluir ciertas observaciones con datos incompletos, siempre que los `NA` sean el resultado de una ausencia real de información.

El mismo paquete trae una función gráfica llamada `plot_na_pareto()`, que genera un gráfico de barras ordenados por frecuencia de valores faltantes:

```
plot_na_pareto(datos, only_na = T)
```



Finalmente, para un diagnóstico más integral de la calidad de las variables, puede utilizarse la función `diagnose()`:

```r
diagnose(datos)
```

```
# A tibble: 7 x 6
  variables types     missing_count missing_percent unique_count unique_rate
  <chr>     <chr>             <int>           <dbl>        <int>       <dbl>
1 id        numeric               0            0              74      1
2 sexo      character             3            4.05            3      0.0405
3 edad      numeric               0            0              45      0.608
4 peso      numeric               0            0              56      0.757
5 talla     numeric               0            0              38      0.514
6 trabaja   logical               9           12.2             3      0.0405
7 fecha     character             0            0              11      0.149
```

Esta función ofrece un resumen detallado que incluye el tipo de variable, la cantidad de valores faltantes, la proporción de valores únicos, entre otros indicadores de utilidad para la exploración inicial.

6.4. Conocer la distribución de las variables de interés

6.4.1. Resumir variables cuantitativas

La instalación básica de R tiene incorporadas múltiples funciones estadísticas que permiten calcular medidas resumen para variables cuantitativas. Estas funciones pueden integrarse a la función `summarise()` de `tidyverse`.

Medidas de tendencia central

Las medidas de tendencia central forman parte del grupo de medidas de posición o localización, pero su objetivo principal es resumir la información en torno a un valor que representa el "centro" de la distribución. Es decir, un valor respecto al cual tienden a agruparse los demás valores.

Podemos obtener la media y la mediana de nuestros datos con el siguiente código:

```r
datos |>
  summarise(
    # Media
    media = mean(edad),
    # Mediana
    mediana = median(edad)
  )
```

```
# A tibble: 1 x 2
  media mediana
  <dbl>   <dbl>
1  48.1    52.5
```

En cambio, R base no incluye una función específica para calcular la **moda**. Para obtenerla, debemos escribir una función propia o utilizar algún paquete adicional que la implemente (por ejemplo, `modeest::mlv()`).

Medidas de posición

Las medidas de posición dividen los datos en grupos con igual número de observaciones. Entre las más utilizadas se encuentran los **cuartiles** y **percentiles**.

La función `quantile()` del paquete base `stats` permite calcular cuartiles u otros percentiles. Por ejemplo, para calcular los cuartiles Q1 y Q3, indicamos en el argumento `probs` los valores 0.25 y 0.75:

```r
datos |>
  summarise(
    # Primer cuartil
    cuartil1 = quantile(edad, probs = 0.25),
    # Tercer cuartil
    cuartil3 = quantile(edad, probs = 0.75)
  )
```

```
# A tibble: 1 x 2
  cuartil1 cuartil3
     <dbl>    <dbl>
1       28       64
```

Para obtener el mínimo y máximo de estos valores numéricos usamos el siguiente código:

```r
datos |>
  summarise(
    # Mínimo
    minimo = min(edad),
    # Máximo
    maximo = max(edad)
  )
```

```
# A tibble: 1 x 2
  minimo maximo
   <dbl>  <dbl>
1     13     86
```

Medidas de dispersión

Las medidas de dispersión nos permiten conocer cuán dispersos o variables son los valores dentro del conjunto de datos.

Entre las más clásicas se encuentran la **varianza** y el **desvío estándar**, que se calculan fácilmente con las funciones `var()` y `sd()`:

```
datos |>
  summarise(
    # Varianza
    varianza = var(edad),
    # Desvío estándar
    desvio = sd(edad)
  )
```

```
# A tibble: 1 x 2
  varianza desvio
     <dbl>  <dbl>
1     405.   20.1
```

También puede ser útil calcular el **rango**, que se obtiene como la diferencia entre el valor máximo y el mínimo, y el **rango intercuartílico (RIC)**, mediante `IQR()`:

```
datos |>
  summarise(
    # Rango
    rango = max(edad) - min(edad),
    # Rango intercuartílico
    ric = IQR(edad)
  )
```

```
# A tibble: 1 x 2
  rango   ric
  <dbl> <dbl>
1    73    36
```

El paquete `dlookr` ofrece la función `describe()` para generar un resumen completo de las variables numéricas:

```
describe(datos, -id)
```

```
# A tibble: 3 x 26
  described_variables     n    na  mean    sd se_mean   IQR skewness kurtosis
  <chr>               <int> <int> <dbl> <dbl>   <dbl> <dbl>    <dbl>    <dbl>
1 edad                   74     0  48.1 20.1    2.34  36      -0.211  -1.11
2 peso                   74     0  72.5 13.2    1.54  17.7     0.145   0.215
3 talla                  74     0 165.   8.12   0.944  9.75    0.256   0.0363
# i 17 more variables: p00 <dbl>, p01 <dbl>, p05 <dbl>, p10 <dbl>, p20 <dbl>,
#   p25 <dbl>, p30 <dbl>, p40 <dbl>, p50 <dbl>, p60 <dbl>, p70 <dbl>,
#   p75 <dbl>, p80 <dbl>, p90 <dbl>, p95 <dbl>, p99 <dbl>, p100 <dbl>
```

Esta función puede aplicarse directamente sobre todo el conjunto de datos. Si bien selecciona automáticamente las variables numéricas, en este caso estamos excluyendo explícitamente la variable `id`, ya que un identificador no tiene interés estadístico.

El resumen que devuelve incluye:

- `na`: cantidad de observaciones con datos y con `NA`.
- `mean`: media aritmética.
- `sd`: desvío estándar de la media.
- `se_mean`: error estándar de la media.
- `IQR`: rango intercuartílico.
- Medidas de forma como la simetría (`skewness`) y la curtosis (`kurtosis`).
- Percentiles, incluyendo la mediana (`P50`) y los cuartiles (`P25` y `P75`).

6.4.2. Resumir variables cualitativas

Las variables cualitativas o categóricas pueden encontrarse en R bajo los tipos de dato `character` o `factor`. En ocasiones será necesario convertirlas a `factor`, ya que este tipo permite aplicar ciertos procedimientos específicos para variables categóricas.

Frecuencias

Podemos resumir individualmente variables cualitativas mediante las frecuencias absolutas y relativas de sus categorías. La función `count()` de `dplyr` nos muestra el conteo absoluto:

```
datos |>
  count(sexo)
```

```
# A tibble: 3 x 2
  sexo      n
  <chr> <int>
1 F        27
2 M        44
3 <NA>      3
```

En la salida se incluirán, además de las categorías presentes, las observaciones con valores faltantes (`NA`).

La inclusión o no de los valores faltantes dependerá del propósito del análisis. Para excluirlos, podemos utilizar `drop_na()`:

```
datos |>
  count(sexo) |>
  # Saltea los valores NA
  drop_na()
```

```
# A tibble: 2 x 2
  sexo      n
  <chr> <int>
1 F        27
2 M        44
```

Para obtener frecuencias relativas en porcentaje:

```
datos |>
  count(sexo) |>
  # Saltea los valores NA
  drop_na() |>
  # Transforma a porcentajes
  mutate(porc = 100 * n / sum(n))
```

```
# A tibble: 2 x 3
  sexo      n  porc
  <chr> <int> <dbl>
1 F        27  38.0
2 M        44  62.0
```

Redondeamos el valor del porcentaje con `round()`:

```
datos |>
  count(sexo) |>
  # Saltea los valores NA
  drop_na() |>
  # Transforma a porcentajes y redondea decimales
  mutate(
    porc = 100 * n / sum(n),
    porc = round(porc, digits = 2)
  )
```

```
# A tibble: 2 x 3
  sexo      n  porc
  <chr> <int> <dbl>
1 F        27  38.0
2 M        44  62.0
```

El paquete `janitor` (Firke 2024) ofrece una alternativa más completa mediante la función `tabyl()`:

```
datos |>
  tabyl(sexo)
```

```
 sexo  n    percent valid_percent
    F 27 0.36486486     0.3802817
    M 44 0.59459459     0.6197183
 <NA>  3 0.04054054            NA
```

Esta función muestra tanto frecuencias absolutas como relativas, incluyendo y excluyendo los valores `NA` (porcentaje sobre el total de valores válidos).

Podemos mejorar la presentación combinando otras funciones del paquete:

```r
datos |>
  # Excluímos valores NA
  tabyl(sexo, show_na = F) |>
  # Añadimos totales por fila
  adorn_totals(where = "row") |>
  # Redondea porcentajes a 2 decimales
  adorn_pct_formatting(digits = 2)
```

```
  sexo  n percent
     F 27  38.03%
     M 44  61.97%
 Total 71 100.00%
```

Tablas de contingencia

La forma más adecuada de describir la relación entre dos variables cualitativas es a través de una **tabla de contingencia**, en la cual:

- Las filas representan las categorías de una variable.
- Las columnas representan las categorías de otra variable.
- Las celdas muestran el número de observaciones correspondientes a cada combinación de categorías.

La función `tabyl()` también permite crear este tipo de tablas. A continuación, un ejemplo entre `sexo` y `trabaja` (aunque `trabaja` sea lógica, puede tratarse como categórica):

```r
datos |>
  tabyl(sexo, trabaja)
```

```
 sexo FALSE TRUE NA_
    F     8   15   4
    M    17   22   5
 <NA>     1    2   0
```

Recordemos que el orden dentro de los paréntesis de la función es igual al de los índices, el primer argumento es la variable que aparecerá en las filas y el segundo la variable de las columnas. Por ese motivo, en la tabla de contingencia absoluta tenemos `sexo` en las filas y `trabaja` en las columnas.

Se puede mejorar la tabla excluyendo los valores `NA` y agregando totales por fila:

```
datos |>
  # Excluímos valores NA
  tabyl(sexo, trabaja, show_na = F) |>
  # Añadimos totales por fila
  adorn_totals(where = "row")
```

```
  sexo FALSE TRUE
     F     8   15
     M    17   22
 Total    25   37
```

Para calcular frecuencias relativas porcentuales por columna usamos el siguiente código:

```
datos |>
  # Excluímos valores NA
  tabyl(sexo, trabaja, show_na = F) |>
  # Añadimos totales
  adorn_totals(where = "row") |>
  # Añadimos porcentajes por columna
  adorn_percentages(denominator = "col") |>
  # Redondea porcentajes a 2 decimales
  adorn_pct_formatting(digits = 2)
```

```
  sexo   FALSE    TRUE
     F  32.00%  40.54%
     M  68.00%  59.46%
 Total 100.00% 100.00%
```

Calculamos frecuencias relativas porcentuales por fila:

```
datos |>
  # Excluímos valores NA
  tabyl(sexo, trabaja, show_na = F) |>
  # Añadimos totales por columna
  adorn_totals(where = "col") |>
  # Añadimos porcentajes por fila
  adorn_percentages(denominator = "row") |>
  # Redondea porcentajes a 2 decimales
  adorn_pct_formatting(digits = 2)
```

```
 sexo  FALSE   TRUE   Total
    F 34.78% 65.22% 100.00%
    M 43.59% 56.41% 100.00%
```

Cambiando el argumento `denominator` por `"all"` se calculan frecuencias relativas al total:

```
datos |>
  # Excluímos valores NA
  tabyl(sexo, trabaja, show_na = F) |>
  # Añadimos totales por columna
  adorn_totals(where = "col") |>
  # Añadimos porcentajes al total
  adorn_percentages(denominator = "all") |>
  # Redondea porcentajes a 2 decimales
  adorn_pct_formatting(digits = 2)
```

```
 sexo  FALSE   TRUE  Total
    F 12.90% 24.19% 37.10%
    M 27.42% 35.48% 62.90%
```

6.4.3. Explorar variables mediante gráficos

Uno de los aportes más importantes de John Tukey al análisis de datos es la incorporación de los gráficos como herramienta exploratoria. A través de representaciones visuales podemos detectar rápidamente patrones, anomalías, valores extremos, asimetrías o relaciones entre variables.

En R, los gráficos más útiles para explorar la distribución **univariada** de las variables son:

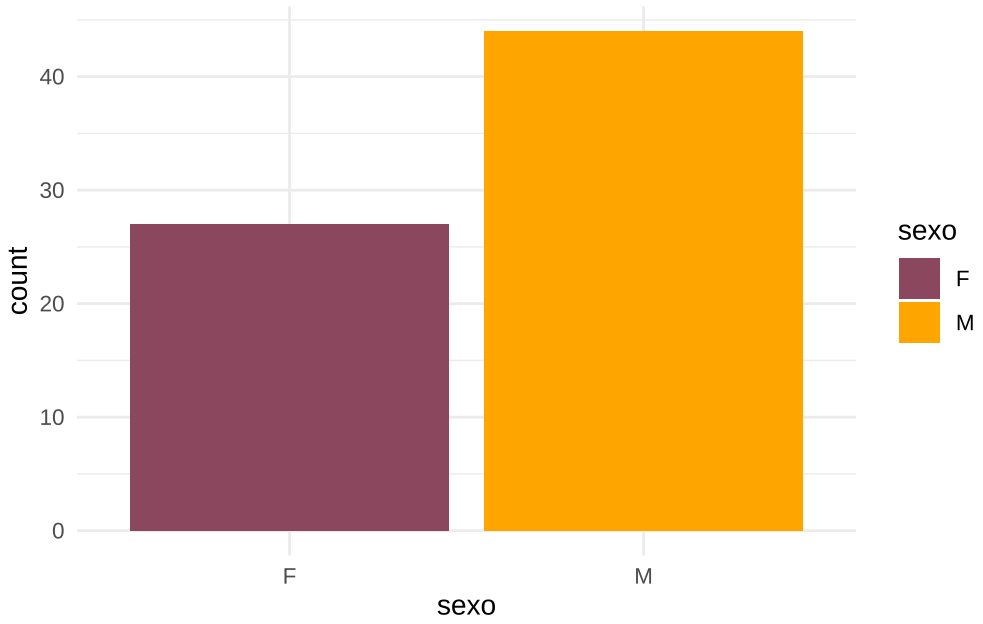
- Para variables **cualitativas**: gráficos de barras
- Para variables **cuantitativas**: histogramas, gráficos de densidad, boxplots y violin plots
- Cuando queremos explorar la relación entre **dos o más variables**, los tipos de gráficos más comunes incluyen:
    - Diagramas de dispersión (puntos)
    - Gráficos de líneas
    - Gráficos de mosaico para variables categóricas cruzadas

El lenguaje R soporta una serie de sistemas gráficos asociados a paquetes como `graphics`, `lattice`, `ggplot2`, etc. que sirven de base incluso para otros paquetes con funciones más específicas. Actualmente el estándar gráfico en R es `ggplot2`.

Barras (univariado)

El gráfico de barras permite visualizar la frecuencia de las categorías de una variable cualitativa:

```
datos |>
  # Omitimos los NA de sexo
  drop_na(sexo) |>
  # Generamos histograma
  ggplot(aes(x = sexo, fill = sexo)) +
  geom_bar() +
  scale_fill_manual(values = c("palevioletred4", "orange")) +
  theme_minimal()
```

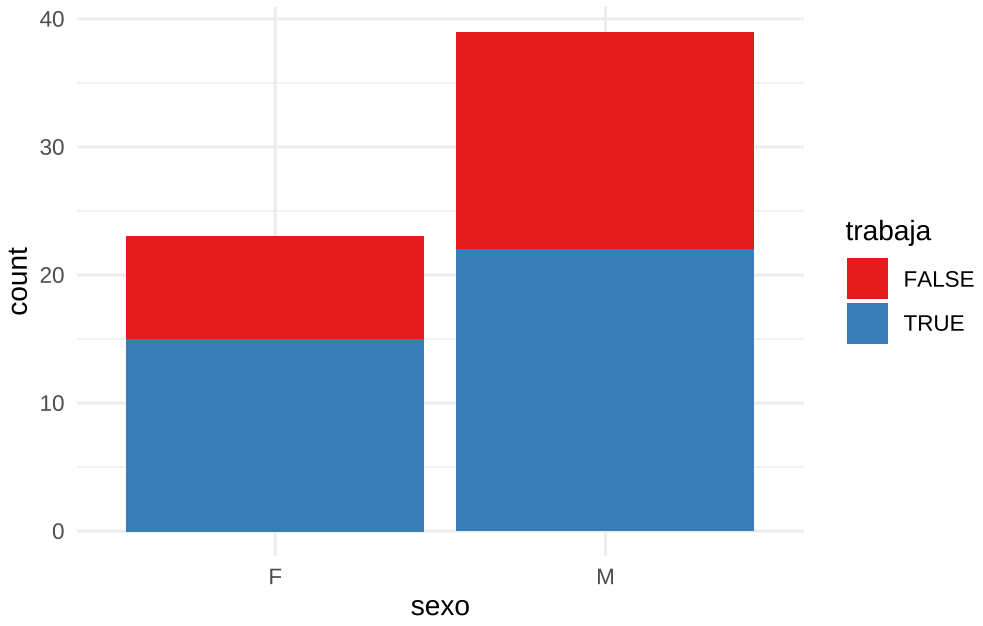


Barras (bivariado)

Cuando cruzamos dos variables categóricas, podemos representar la relación entre ambas modificando el argumento `position` de `geom_bar()`.

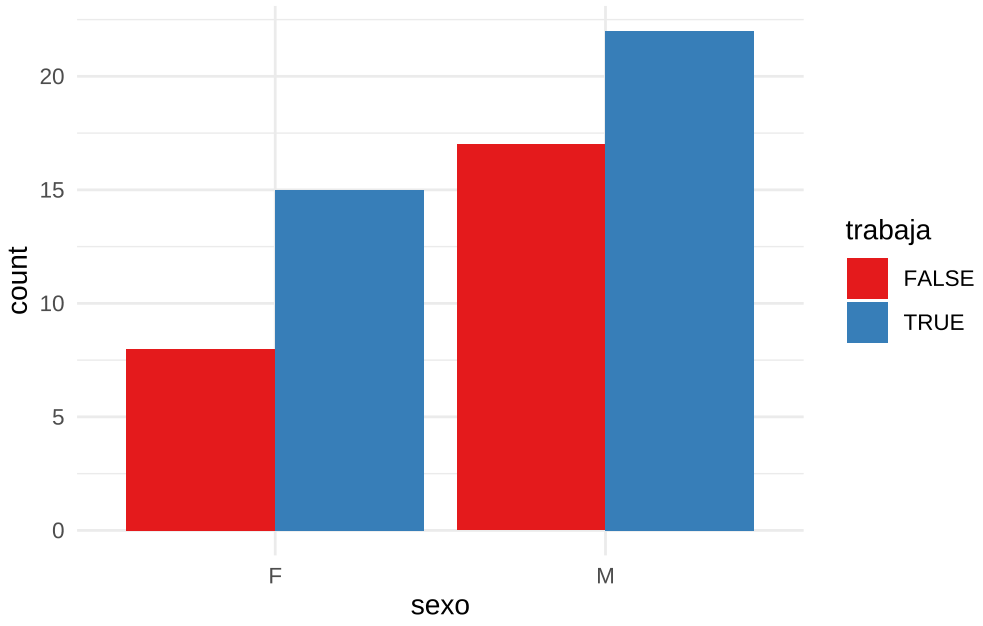
El argumento `position = "stack"` nos muestra los valores absolutos acumulados:

```r
datos |>
  # Omitimos los NA de sexo y trabaja
  drop_na(sexo, trabaja) |>
  # Generamos gráfico de barras
  ggplot(aes(x = sexo, fill = trabaja)) +
  geom_bar(position = "stack") +
  scale_fill_brewer(palette = "Set1") +
  theme_minimal()
```



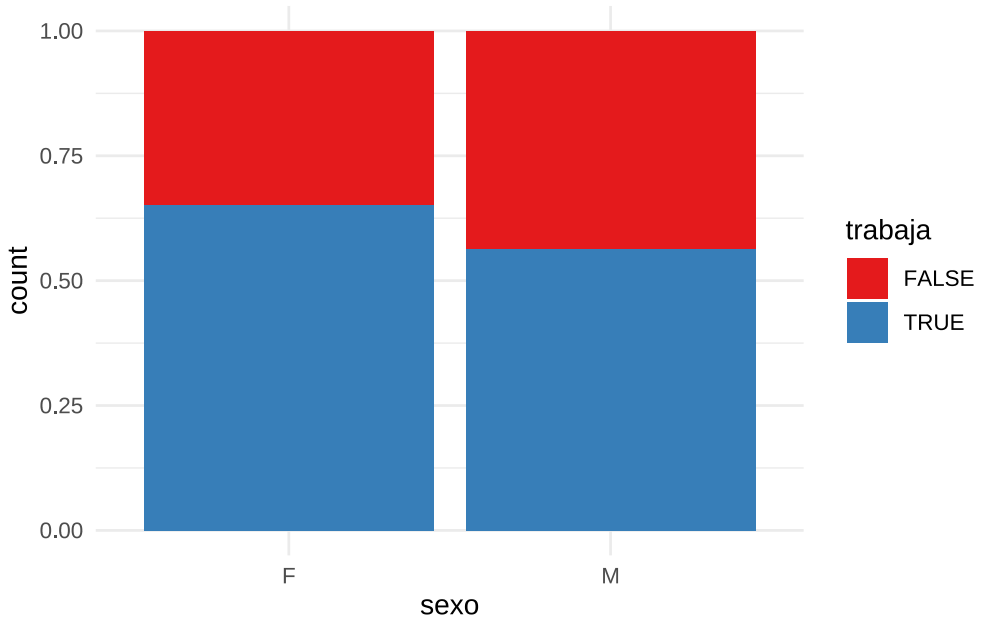
Por otro lado, el argumento `position = "dodge"` muestra las barras lado a lado, permitiendo comparar proporciones entre grupos:

```r
datos |>
  # Omitimos los NA de sexo y trabaja
  drop_na(sexo, trabaja) |>
  # Generamos gráfico de barras
  ggplot(aes(x = sexo, fill = trabaja)) +
  geom_bar(position = "dodge") +
  scale_fill_brewer(palette = "Set1") +
  theme_minimal()
```



Finalmente, `position = "fill"` convierte las alturas en proporciones sobre el total por grupo:

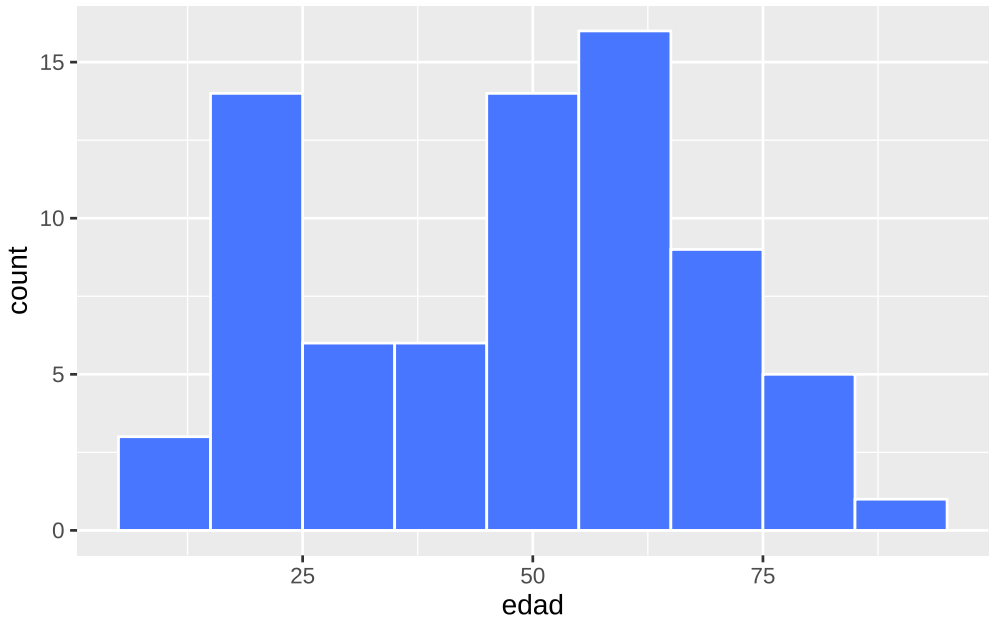
```
datos |>
  # Omitimos los NA de sexo y trabaja
  drop_na(sexo, trabaja) |>
  # Generamos gráfico de barras
  ggplot(aes(x = sexo, fill = trabaja)) +
  geom_bar(position = "fill") +
  scale_fill_brewer(palette = "Set1") +
  theme_minimal()
```



Histograma

Representa la frecuencia de valores en intervalos definidos. Útil para observar la forma general de la distribución:

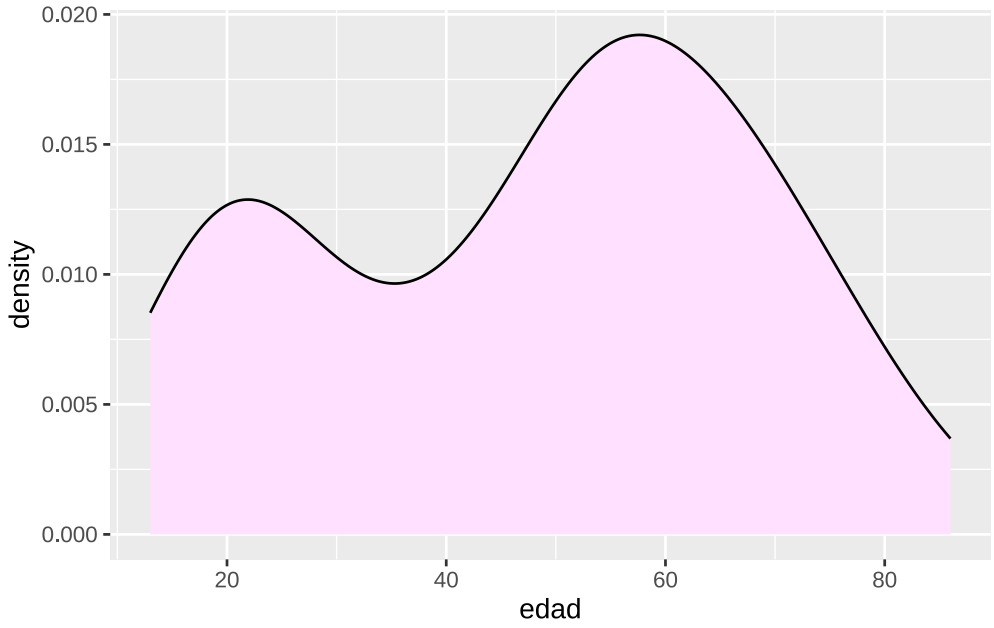
```
datos |>
  # Genera histograma
  ggplot(aes(x = edad)) +
  geom_histogram(binwidth = 10, fill = "royalblue1", color = "white")
```



Densidad

Es una estimación suave de la distribución de frecuencias:

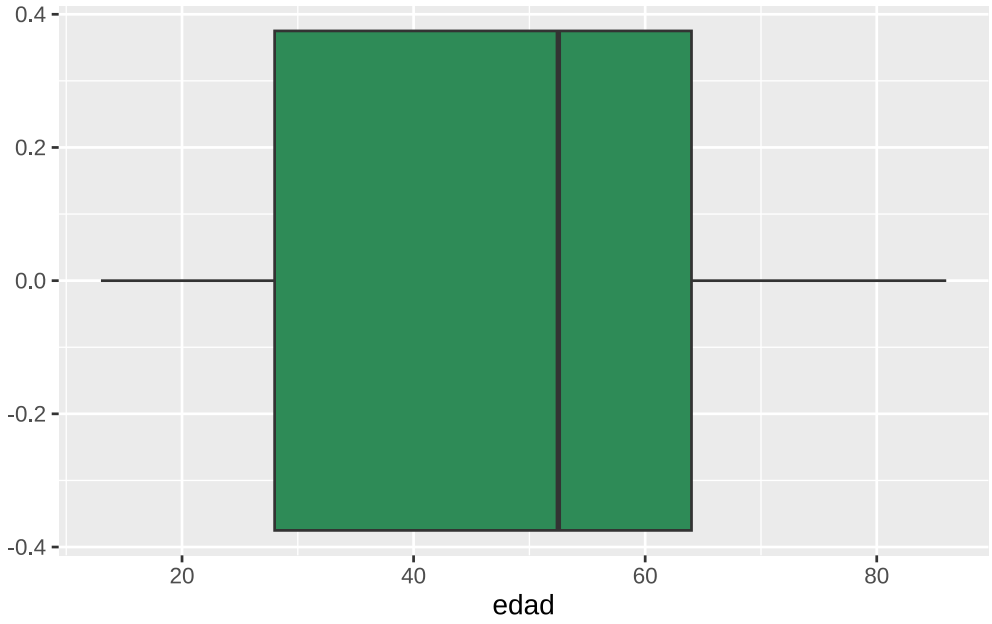
```
datos |>
  ggplot(aes(x = edad)) +
  geom_density(fill = "thistle1")
```



Boxplot

Muestra el rango intercuartílico, la mediana y los valores atípicos. Ideal para detectar asimetrías y *outliers*:

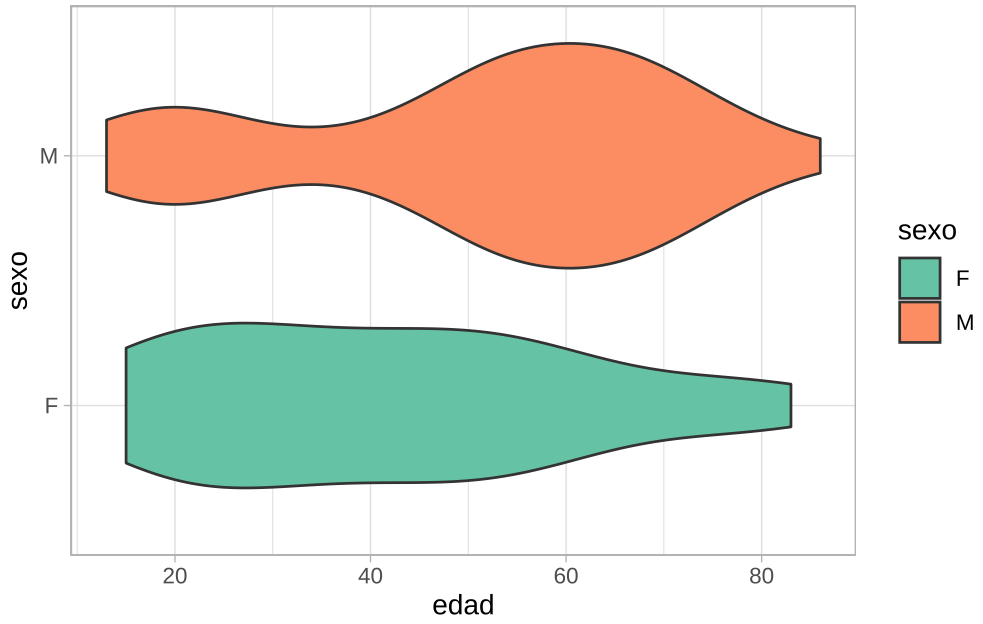
```
datos |>
  # Genera boxplot
  ggplot(aes(x = edad)) +
  geom_boxplot(fill = "seagreen4")
```



Violinplot

Combina el *boxplot* con una curva de densidad reflejada. Permite visualizar tanto la forma de la distribución como los cuantiles:

```
datos |>
  # Omitimos los NA de sexo
  drop_na(sexo) |>
  # Genera violinplot
  ggplot(aes(x = edad, y = sexo, fill = sexo)) +
  geom_violin() +
  scale_fill_brewer(palette = "Set2") +
  theme_light()
```

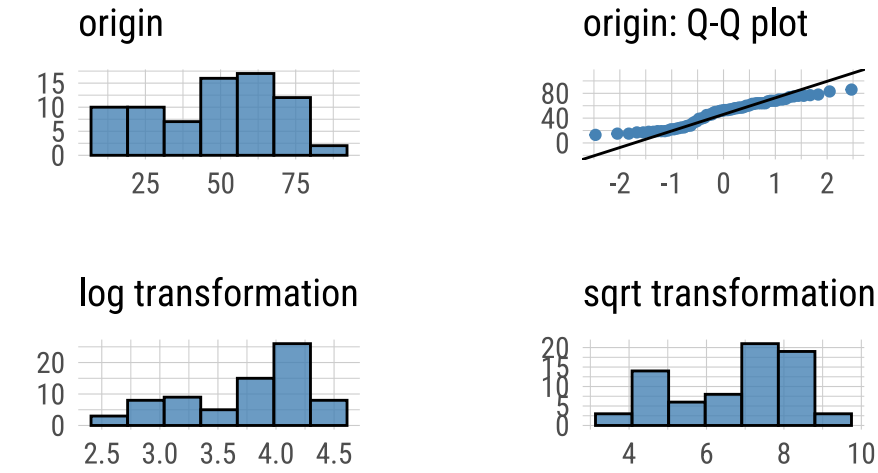


Q-Q Plot

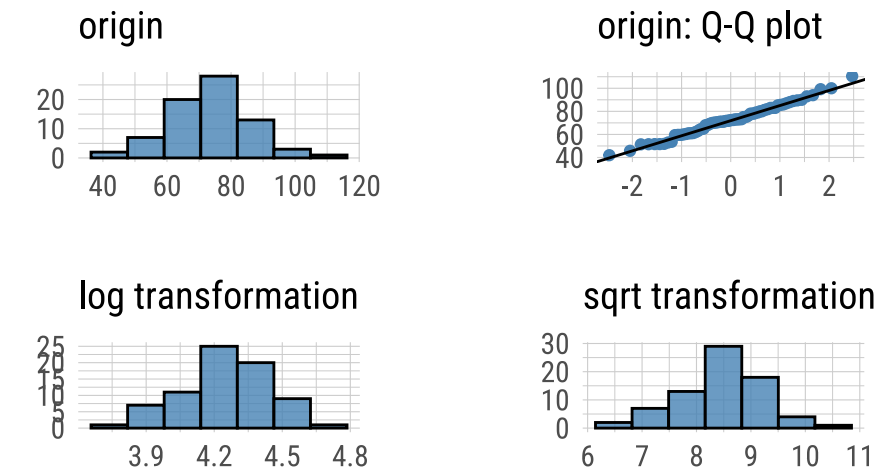
Los gráficos Q-Q (cuantil-cuantil) permiten evaluar visualmente si una variable sigue una distribución teórica, como la normal. Suelen usarse como método gráfico para analizar "normalidad", es decir cuanto se asemeja la distribución de la variable a la distribución normal o gaussiana.

La función `plot_normality()` de `dlookr` muestra un diagnóstico gráfico de normalidad de una variable usando histogramas y Q-Q plot. Además muestra otros histogramas con conversiones de datos (logarítmico y raíz cuadrada por defecto, pero también "Box-Cox" y otras):

```
# Sobre la variable edad
datos |>
  plot_normality(edad)
```




```
# Sobre la variable peso
datos |>
  plot_normality(peso)
```



Podemos decir que la variable `peso` se ajusta mejor a una distribución normal, ya que los puntos del Q-Q plot se alinean más cercanamente a la diagonal teórica.

> **Nota**: Este análisis gráfico de normalidad suele complementarse con pruebas estadísticas específicas, que abordaremos en la próxima unidad.

6.5. Detección de valores atípicos

Un valor atípico (*outlier*) es una observación que se encuentra numéricamente alejada del resto de los datos. Su presencia puede tener diferentes causas, y su tratamiento dependerá del contexto:

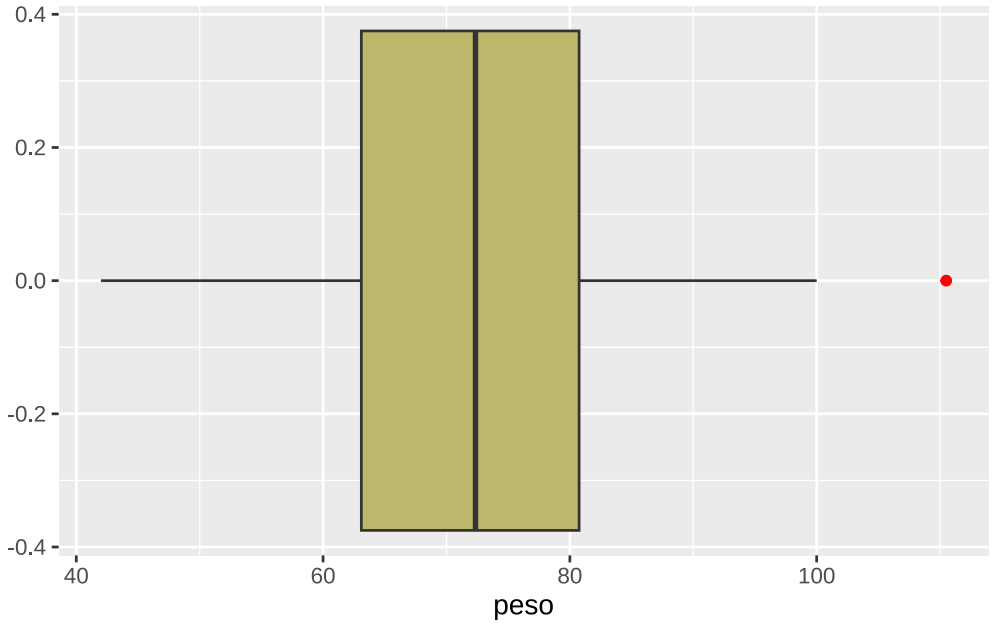
- **Errores de carga o procedimiento**: deben corregirse si se detectan.
- **Valores extremos plausibles**: pueden ser válidos, pero conviene evaluarlos en detalle.
- **Eventos extraordinarios o causas desconocidas**: si no se pueden justificar, suelen excluirse del análisis.

Estos valores pueden afectar sensiblemente ciertos estadísticos como la media, distorsionando su interpretación.

Una forma gráfica común de detectar valores atípicos es mediante los *boxplots*. Los puntos situados fuera de los "bigotes" representan posibles *outliers*.

A continuación, se presenta un ejemplo con la variable `peso`, donde se observa un valor extremo en el límite superior de la distribución (punto rojo):

```
datos |>
  ggplot(aes(x = peso)) +
  geom_boxplot(fill = "darkkhaki", outlier.color = "red")
```



Este valor coincide con el máximo observado:

```
max(datos$peso)
```

```
[1] 110.5
```

El paquete `dlookr` incluye la función `diagnose_outlier()` para la detección automatizada de valores atípicos en todas las variables numéricas de un conjunto de datos:

```
diagnose_outlier(datos)
```

```
# A tibble: 4 x 6
  variables outliers_cnt outliers_ratio outliers_mean with_mean without_mean
  <chr>            <int>          <dbl>         <dbl>     <dbl>        <dbl>
1 id                   0           0             NaN       37.5         37.5
2 edad                 0           0             NaN       48.1         48.1
3 peso                 1           1.35          110.      72.5         72.0
4 talla                0           0             NaN      165.         165.
```

Esta función devuelve una tabla que incluye, para cada variable: cantidad y proporción de *outliers* detectados, media de la variable incluyendo los *outliers*, media de la variable excluyendo los *outliers*. En función de estos dos estadísticos se puede comparar el efecto de los valores atípicos en la media.

El paquete `skimr` (Waring et al. 2026) permite obtener un resumen estadístico compacto y amigable de un conjunto de datos mediante la función `skim()`:

```
skim(datos)
```

Tabla 6.1: Data summary

Name	datos
Number of rows	74
Number of columns	7
_____	
Column type frequency:	
character	2
logical	1
numeric	4
_____	
Group variables	None

Variable type: character

skim_variable	n_missing	complete_rate	min	max	empty	n_unique	whitespace
sexo	3	0.96	1	1	0	2	0
fecha	0	1.00	10	10	0	11	0

Variable type: logical

skim_variable	n_missing	complete_rate	mean	count
trabaja	9	0.88	0.6	TRU: 39, FAL: 26

Variable type: numeric

skim_variable	n_missing	complete_rate	mean	sd	p0	p25	p50	p75	p100	hist
id	0	1	37.50	21.51	1	19.25	37.50	55.75	74.0	▇▇▇▇▇
edad	0	1	48.07	20.12	13	28.00	52.50	64.00	86.0	▃▆▇▆▅
peso	0	1	72.49	13.23	42	63.10	72.35	80.75	110.5	▃▇▇▂▁
talla	0	1	164.85	8.12	148	160.00	164.00	169.75	185.0	▂▅▇▃▂

Además, puede integrarse fácilmente con la gramática `tidyverse`. Por ejemplo, podemos explorar estadísticas descriptivas de variables numéricas agrupadas por `sexo`:

```
datos |>
  # Excluye NAs de sexo
  drop_na(sexo) |>
  # Agrupa por sexo
  group_by(sexo) |>
  # Solo variables numéricas - id
  select(where(is.numeric), -id) |>
  # Explora outliers
  skim()
```

Tabla 6.2: Data summary

Name	select(…)
Number of rows	71
Number of columns	4
_____	
Column type frequency:	
numeric	3
_____	
Group variables	sexo

Variable type: numeric

skim_variable	sexo	n_missing	complete_rate	mean	sd	p0	p25	p50	p75	p100	hist
edad	F	0	1	41.89	19.64	15.0	26.00	39.0	54.50	83.0	▆▇▆▅▂
edad	M	0	1	51.59	19.56	13.0	40.75	55.5	64.75	86.0	▂▅▆▇▅
peso	F	0	1	63.88	12.02	42.0	53.30	61.0	72.00	85.6	▇▆▅▃▃
peso	M	0	1	77.97	11.49	51.8	71.00	77.2	83.75	110.5	▂▇▆▂▁
talla	F	0	1	158.57	6.08	148.0	155.00	159.5	162.25	172.0	▂▅▇▆▂
talla	M	0	1	168.85	6.58	158.0	164.00	167.0	173.00	185.0	▂▇▆▃▁

En este ejemplo, mostramos resultados de variables numéricas menos de `id` agrupados por `sexo` (sin considerar valores `NA` en las categorías de sexo).

Referencias

7 Hoja de Estilo del lenguaje R

7.1. Convenciones de estilo R

R es bastante indulgente con la forma en que escribimos código, a diferencia de otros lenguajes como Python, donde un espacio mal puesto puede arruinar el script. Sin embargo, adoptar una guía de estilo mejora la legibilidad, facilita la colaboración y reduce errores.

Las siguientes líneas de código producen el mismo resultado, pero no todas son igual de claras:

```
# Opción 1
mpg |>
  filter(cty > 10, class == "compact")

# Opción 2
mpg |> filter(cty > 10, class == "compact")

# Opción 3
mpg |>
  filter(cty > 10, class == "compact")

# Opción 4
mpg |> filter(cty > 10, class == "compact")

# Opción 5
filter(mpg, cty > 10, class == "compact")

# Opción 6
mpg |>
  filter(cty > 10, class == "compact")

# Opción 7
filter(mpg, cty > 10, class == "compact")
```

Las tres primeras versiones son más legibles. El resto, aunque válidas, son difíciles de seguir, especialmente en trabajos colaborativos o materiales docentes.

7.2. Guía de estilo de `tidyverse`

Para ayudar a mejorar la legibilidad y facilitar el compartir código con otros, el equipo de Tidyverse publicó una guía concisa con ejemplos claros de buenas y malas formas de escribir código, nombres de variables, sangría, líneas largas, y más:

 https://style.tidyverse.org/

Además, RStudio incluye herramientas para aplicar estas convenciones automáticamente. Por ejemplo, seleccionando el código y presionando `Ctrl + i` (Windows) se puede reidentar el texto. No siempre es perfecto, pero es realmente útil para lograr la sangría correcta sin tener que presionar manualmente espacio muchas veces.

7.2.1. Espaciado

Colocar espacios después de las comas:

 Correcto

```
filter(mpg, cty > 10)
```

 Incorrecto

```
filter(mpg,cty>10)

filter(mpg,cty>10)

filter(mpg,cty>10)
```

Colocar espacios después de comas y alrededor de operadores (+, −, >, =, etc.) mejora la legibilidad. También se deben evitar espacios innecesarios dentro de paréntesis:

 Correcto

```
filter(mpg, cty > 10)
```

 Incorrecto

```
filter(mpg,  cty > 10)

filter(mpg, cty > 10)

filter(mpg,  cty > 10)
```

No colocar espacios alrededor de paréntesis que sean parte de funciones:

Correcto

```
filter(mpg, cty > 10)
```

Incorrecto

```
filter(mpg, cty > 10)

filter(mpg, cty > 10)

filter(mpg, cty > 10)
```

7.2.2. Líneas largas

En general, es una buena práctica no tener líneas de código muy largas. Se recomienda limitar las líneas a **80 caracteres**. Para visualizarlo, en RStudio vamos a `Tools > Global Options > Code > Displayy` seleccionamos la casilla **Show margin.**

Se sugiere agregar saltos de línea dentro de las líneas de código más largas, los mismos deben colocarse luego de las comas y los argumentos se deben alinear dentro de la función:

Correcto

```
filter(mpg, cty > 10, class == "compact")


filter(mpg, cty > 10, class == "compact")


filter(mpg, cty > 10, class == "compact")

filter(
  mpg,
  cty > 10,
  class %in%
    c("compact", "pickup", "midsize", "subcompact", "suv", "2seater", "minivan")
)
```

Incorrecto

```
filter(
  mpg,
  cty > 10,
  class %in%
    c("compact", "pickup", "midsize", "subcompact", "suv", "2seater", "minivan")
)
```

7.2.3. Tuberías y capas ggplot2

Colocar cada paso de la tubería (%>% ó |>) en una línea separada, con sangría de dos espacios debajo del operador:

Correcto

```
mpg |>
  filter(cty > 10) |>
  group_by(class) |>
  summarize(avg_hwy = mean(hwy))
```

Incorrecto

```
# Mal
mpg |> filter(cty > 10) |> group_by(class) |>
  summarize(avg_hwy = mean(hwy))

# Muy mal
mpg |> filter(cty > 10) |> group_by(class) |> summarize(avg_hwy = mean(hwy))

# Tan mal que no funciona
mpg |>
  filter(cty > 10)
  |> group_by(class)
  |> summarize(avg_hwy = mean(hwy))
```

Lo mismo aplica para las capas de gráficos de ggplot2, usando el conector + al final de la línea y debajo sangría de dos espacios:

Correcto

```r
ggplot(mpg, aes(x = cty, y = hwy, color = class)) +
  geom_point() +
  geom_smooth() +
  theme_bw()

# Mal
ggplot(mpg, aes(x = cty, y = hwy, color = class)) +
  geom_point() +
  geom_smooth() +
  theme_bw()

# Muy mal
ggplot(mpg, aes(x = cty, y = hwy, color = class)) +
  geom_point() +
  geom_smooth() +
  theme_bw()

# Tan mal que no funciona
ggplot(mpg, aes(x = cty, y = hwy, color = class))
+geom_point()
+geom_smooth()
+theme_bw()
```

Incorrecto

```r
# Mal
ggplot(mpg, aes(x = cty, y = hwy, color = class)) +
  geom_point() +
  geom_smooth() +
  theme_bw()

# Muy mal
ggplot(mpg, aes(x = cty, y = hwy, color = class)) +
  geom_point() +
  geom_smooth() +
  theme_bw()

# Tan mal que no funciona
ggplot(mpg, aes(x = cty, y = hwy, color = class))
+geom_point()
+geom_smooth()
+theme_bw()
```

7.2.4. Comentarios

Los comentarios deben comenzar con # seguido de un espacio:

Correcto

```
# Bien
```

Incorrecto

```
#Mal

#Mal
```

Si el comentario es corto, se puede incluir en la misma línea, separado por al menos dos espacios para mejorar la legibilidad:

```
mpg |>
  filter(cty > 10) |> # filtro filas donde cty es 10 o más
  group_by(class) |> # estratifica por class
  summarize(avg_hwy = mean(hwy)) # resume la media de hwy por cada grupo
```

Se puede agregar espacios adicionales para alinear los comentarios en línea, si lo deseamos:

```
mpg |>
  filter(cty > 10) |> # filtro filas donde cty es 10 o más
  group_by(class) |> # estratifica por class
  summarize(avg_hwy = mean(hwy)) # resume la media de hwy por cada grupo
```

Si el comentario es muy largo, podemos dividirlo en varias líneas. RStudio incluye una herramienta útil para comentarios largos: `Code > Reflow Comment` los ajusta automáticamente al ancho deseado.

7.2.5. Nombres de columnas

Se recomienda utilizar nombres de columnas en un formato consistente y compatible con el enfoque del ecosistema `tidyverse`. Esto facilita la manipulación de datos y reduce errores en el código.

En particular, se sugiere evitar:

- Nombres excesivamente largos
- Uso de mayúsculas

- Signos de puntuación, acentos y caracteres especiales
- Espacios en blanco

En su lugar, es recomendable utilizar:

- Nombres breves pero descriptivos
- Letras minúsculas

palabras separadas por guiones bajos (_) o puntos (.)

Correcto

```
names(mpg)
```

```
 [1] "manufacturer" "model"        "displ"        "year"         "cyl"
 [6] "trans"        "drv"          "cty"          "hwy"          "fl"
[11] "class"
```

Incorrecto

En bases de datos existentes:

```
names(iris)
```

```
[1] "Sepal.Length" "Sepal.Width"  "Petal.Length" "Petal.Width"  "Species"
```

Al crear un nuevo dataset:

```
datos <- tibble(
  Jurisdicción = c(rep("Buenos Aires", 10), rep("CABA", 10)),
  CasosTotales = sample(100, 20),
  POBLACIÓN = c(rep(16626374, 10), rep(3054237, 10)),
)

names(datos)
```

```
[1] "Jurisdicción" "CasosTotales" "POBLACIÓN"
```

Al crear una nueva variable o una tabla resumen:

```
# Tan mal que da error
datos  |>
summarise(tasa-incidencia(100k) = CasosTotales/POBLACIÓN * 100000)
```

```
Error in parse(text = input): <text>:3:30: unexpected symbol
2: datos  |>
3: summarise(tasa-incidencia(100k
                               ^
```

En caso de que nuestros datos tengan nombres de columnas no compatibles con `tidyverse`, se recomienda usar la función `clean_names()` del paquete `janitor` para estandarizarlas:

```
library(janitor)

iris |>
  clean_names() |>
  head(10)
```

```
   sepal_length sepal_width petal_length petal_width species
1           5.1         3.5          1.4         0.2  setosa
2           4.9         3.0          1.4         0.2  setosa
3           4.7         3.2          1.3         0.2  setosa
4           4.6         3.1          1.5         0.2  setosa
5           5.0         3.6          1.4         0.2  setosa
6           5.4         3.9          1.7         0.4  setosa
7           4.6         3.4          1.4         0.3  setosa
8           5.0         3.4          1.5         0.2  setosa
9           4.4         2.9          1.4         0.2  setosa
10          4.9         3.1          1.5         0.1  setosa
```

```
datos <- datos |>
  clean_names()

names(datos)
```

```
[1] "jurisdiccion"  "casos_totales" "poblacion"
```

Bache, Stefan Milton, y Hadley Wickham. 2026. *magrittr: A Forward-Pipe Operator for R*. https://doi.org/10.32614/CRAN.package.magrittr.

Firke, Sam. 2024. *janitor: Simple Tools for Examining and Cleaning Dirty Data*. https://doi.org/10.32614/CRAN.package.janitor.

Horikoshi, Masaaki, y Yuan Tang. 2018. *ggfortify: Data Visualization Tools for Statistical Analysis Results*. https://CRAN.R-project.org/package=ggfortify.

Kassambara, Alboukadel. 2026. *ggpubr: 'ggplot2' Based Publication Ready Plots*. https://CRAN.R-project.org/package=ggpubr.

Kassambara, Alboukadel, Marcin Kosinski, y Przemyslaw Biecek. 2026. *survminer: Drawing Survival Curves using 'ggplot2'*. https://doi.org/10.32614/CRAN.package.survminer.

Lüdecke, Daniel, Indrajeet Patil, Mattan S. Ben-Shachar, Brenton M. Wiernik, Philip Waggoner, y Dominique Makowski. 2021. «see: An R Package for Visualizing Statistical Models». *Journal of Open Source Software* 6 (64): 3393. https://doi.org/10.21105/joss.03393.

Meyer, Fanny, y Victor Perrier. 2025. *esquisse: Explore and Visualize Your Data Interactively*. https://doi.org/10.32614/CRAN.package.esquisse.

Müller, Kirill, y Hadley Wickham. 2026. *tibble: Simple Data Frames*. https://doi.org/10.32614/CRAN.package.tibble.

Pedersen, Thomas Lin. 2025. *patchwork: The Composer of Plots*. https://doi.org/10.32614/CRAN.package.patchwork.

Pedersen, Thomas Lin, y Fabio Crameri. 2023. *scico: Colour Palettes Based on the Scientific Colour-Maps*. https://doi.org/10.32614/CRAN.package.scico.

Ryu, Choonghyun. 2025. *dlookr: Tools for Data Diagnosis, Exploration, Transformation*. https://CRAN.R-project.org/package=dlookr.

Schloerke, Barret, Di Cook, Joseph Larmarange, et al. 2025. *GGally: Extension to 'ggplot2'*. https://doi.org/10.32614/CRAN.package.GGally.

Tennekes, Martijn, y Marco J. H. Puts. 2023. «cols4all: a color palette analysis tool». *EuroVis (Short Papers)*. https://doi.org/10.2312/evs.20231040.

Waring, Elin, Michael Quinn, Amelia McNamara, Eduardo Arino de la Rubia, Hao Zhu, y Shannon Ellis. 2026. *skimr: Compact and Flexible Summaries of Data*. https://doi.org/10.32614/CRAN.package.skimr.

Wickham, Hadley. 2011. «The Split-Apply-Combine Strategy for Data Analysis». *Journal of Statistical Software* 40 (1): 1-29. https://www.jstatsoft.org/v40/i01/.

Wickham, Hadley. 2016. *ggplot2: Elegant Graphics for Data Analysis*. Springer-Verlag New York. https://ggplot2.tidyverse.org.

Wickham, Hadley, Mara Averick, Jennifer Bryan, et al. 2019. «Welcome to the tidyverse». *Journal of Open Source Software* 4 (43): 1686. https://doi.org/10.21105/joss.01686.

Wickham, Hadley, y Jennifer Bryan. 2026. *readxl: Read Excel Files*. https://doi.org/10.32614/CRAN.package.readxl.

Wickham, Hadley, Romain François, Lionel Henry, Kirill Müller, y Davis Vaughan. 2026. *dplyr: A Grammar of Data Manipulation*. https://doi.org/10.32614/CRAN.package.dplyr.

Wickham, Hadley, Jim Hester, y Jennifer Bryan. 2026. *readr: Read Rectangular Text Data*. https://doi.org/10.32614/CRAN.package.readr.